

A Simple and Efficient One-Shot Signature Scheme

Andrew Huang*
MIT

Vinod Vaikuntanathan†
MIT

October 13, 2025

Abstract

One-shot signatures (OSS) are a powerful and uniquely quantum cryptographic primitive which allows anyone, given common reference string, to come up with a public verification key pk and a secret signing state $|sk\rangle$. With the secret signing state, one can produce the signature of *any one* message, but no more. In a recent breakthrough work, Shmueli and Zhandry (CRYPTO 2025) constructed one-shot signatures, either unconditionally in a classical oracle model or assuming post-quantum indistinguishability obfuscation and the hardness of Learning with Errors (LWE) in the plain model.

In this work, we address the inefficiency of the Shmueli-Zhandry construction which signs messages bit-by-bit, resulting in signing keys of $\Theta(\lambda^4)$ qubits and signatures of size $\Theta(\lambda^3)$ bits for polynomially long messages, where λ is the security parameter. We construct a new, simple, direct, and efficient one-shot signature scheme which can sign messages of any polynomial length using signing keys of $\Theta(\lambda^2)$ qubits and signatures of size $\Theta(\lambda^2)$ bits. We achieve corresponding savings in runtimes, in both the oracle model and the plain model. In addition, unlike the Shmueli-Zhandry construction, our scheme achieves perfect correctness.

Our scheme also achieves strong signature incompressibility, which implies a public-key quantum fire scheme with perfect correctness among other applications, correcting an error in a recent work of Çakan, Goyal and Shmueli (QCrypt 2025) and recovering their applications.

*E-mail: ahuang@mit.edu

†E-mail: vinodv@mit.edu. Research supported in part by NSF CNS-2154149, a Simons Investigator award and a Ford Foundation Chair at MIT.

Contents

1	Introduction	1
2	Technical Overview	3
2.1	Our Construction	5
2.2	Application to Quantum Fire	6
2.3	On the Use of the [DS22] Reduction in [SZ25]	7
2.4	Open Problems	8
3	Preliminaries	8
3.1	One-Shot Signatures	8
4	Our Construction in the Oracle Model	9
4.1	The Construction	9
4.2	Perfect Correctness	12
4.3	Strong Security	13
5	Public-Key Quantum Fire and Other Applications	16
A	Cryptographic Definitions and Lemmas	24
B	Our Construction in the Plain Model	27
B.1	Perfect Correctness	27
B.2	Strong Security	28
C	Signing Polynomial-Length Messages	30
C.1	Oracle Model Construction	30
C.2	Plain Model Construction	31
D	Subspace Hiding Lemmas	32
E	Missing Proofs from Section 4 and 5	44
F	Missing Proofs from Section B	49
G	Improved Analysis of the [SZ25] Distinguisher	56

1 Introduction

One-shot signatures (OSS), first defined in the work of Amos, Georgiou, Kiayias and Zhandry [AGKZ20], are a uniquely quantum form of digital signatures [GMR88] where, given some public parameters, anyone can sample a classical verification key together with a quantum signing key. The signing key allows anyone to produce the signature, a classical bit-string, of any message of their choice, but only one. In particular, the signing key self-destructs once a single signature is produced. Security requires that given the public parameter, no quantum polynomial-time algorithm can produce valid signatures for two different messages. (One could strengthen this definition to allow the adversary to produce a different signature of the same message; for more discussion, see Section 2.)

One-shot signatures are very powerful: they immediately imply a number of exotic quantum cryptographic primitives, including quantum money, quantum lightning, non-collapsing hash functions, non-interactive and publicly verifiable proofs of quantumness and certified min-entropy, and more.

In a recent breakthrough work, Shmueli and Zhandry [SZ25] gave the first construction of one-shot signatures. In a classical oracle model, where all algorithms have oracle access to a (classical) function, their construction is unconditionally secure. In the plain model with no oracles, they show a construction assuming post-quantum secure indistinguishability obfuscation (pqIO), one-way functions (pqOWF), lossy functions (pqLF), and (essentially) a noisy trapdoor claw-free function (pqNTCF), the latter three of which can be instantiated assuming the post-quantum subexponential learning with errors (LWE) assumption.

Shmueli and Zhandry [SZ25] in fact construct a related primitive called a *non-collapsing hash function*, a notion that originates from the works of Ambainis, Rossmanis and Unruh [ARU14, Unr16], and use generic reductions from [AGKZ20, DS22] to boost this to a one-shot signature scheme. Unfortunately, this path leads to a large polynomial factor blowup in the sizes of the signing state and the signature itself, and the runtimes of the associated algorithms. For example, as-is, the one-shot signature scheme from [SZ25] has signing keys with $\Theta(\lambda^{12})$ qubits, public keys with $\Theta(\lambda^{12})$ bits, and signatures with $\Theta(\lambda^3)$ bits, where λ is the security parameter¹. With an improved analysis of their non-collapsing hash function (see Appendix G) and opening up the [DS22] reduction, the signing and public keys can be reduced to $\Theta(\lambda^4)$ qubits and $\Theta(\lambda^3)$ bits, respectively.

Still, several inefficiencies remain. For one, the [SZ25] construction is seemingly inherently restricted to signing one bit at a time. Secondly, the signing algorithm seems to require a Marriott-Watrous-style [MW04] alternating projections algorithm.² The first of these issues causes a λ factor blowup in the signing key size and the public-key and signature size; the second causes a further λ factor blowup in the signing key size.

In this work, we construct a new one-shot signature scheme which is:

- **Efficient:** Our scheme signs polynomially many bits in one shot. In the classical oracle model, the signing keys have size $O(\lambda^2)$ qubits, and the public keys and signatures have size $O(\lambda^2)$ bits. Signing and verification also requires significantly fewer oracle queries. We achieve analogous savings in the plain model; for more details, see Table 1.
- **Simple and Direct:** The algorithms are simply described and, for example, avoid complex Marriott-Watrous/QSVT-style circuits.
- **Perfectly Correct:** The OSS scheme of [SZ25] has a negligible correctness error, again seemingly inherently so, in the sense that the signing algorithm may fail to produce a valid signature. Our scheme, in contrast, enjoys perfect correctness.

¹As is standard in cryptography, this means that we demand security against 2^λ -time adversaries that attempt to succeed with better than $2^{-\lambda}$ advantage.

²The [DS22] reduction implements a more complicated quantum rewinding/quantum singular value transform (QSVT) circuit, but at its core, still involves an alternating projections algorithm.

		Signing Key Size (# qubits)	Public Key Size (# bits)	Signature Size (# bits)
Oracle Model	[SZ25]	$\Theta(\lambda^4)$	$\Theta(\lambda^3)$	$\Theta(\lambda^3)$
	This work	$\Theta(\lambda^2)$	$\Theta(\lambda^2)$	$\Theta(\lambda^2)$
Plain Model	[SZ25]	$\Theta(\lambda^4)$	$\Theta(\lambda^4) \cdot \lambda = \Theta(\lambda^5)$	$\Theta(\lambda^3)$
	This work	$\Theta(\lambda^2)$	$\Theta(\lambda^4)$	$\Theta(\lambda^2)$

Figure 1: OSS parameters for signing polynomially long messages. By standard collision-resistant hashing, it suffices to sign λ -bit messages. The public-key size in the plain model [SZ25] construction consists of $O(\lambda)$ repetitions of a base public key of $O(\lambda^4)$ bits, whereas our scheme has a single one.

- **Incompressible:** A simple modification of our scheme in the oracle model satisfies an incompressibility property, first defined in [ÇGS25], which requires that it is impossible to telegraph a classical bit string of any polynomial length which communicates information about too many valid signatures. This in turn leads us to the construction of a quantum fire [BNZ25] and quantum key-fire [ÇGS25] scheme, both with the added advantage of perfect correctness. In the process, we identify a gap in the work of [ÇGS25] who claimed that the λ -bit OSS scheme from [SZ25] satisfies incompressibility; indeed, we show that this scheme, a parallel repetition of a single-bit OSS scheme, cannot be incompressible. We fix this bug by showing that our multi-bit OSS scheme is indeed incompressible, recovering the claim of [ÇGS25], albeit for our OSS scheme. (For more details, we refer the reader to Section 2.2.)

Our results can be summarized by the following theorems:

Theorem 1.1 (Informal). *There exist one-shot signature schemes for polynomially long messages with perfect correctness and strong unforgeability with the following efficiency properties:*

1. *In the classical oracle model, unconditionally: the scheme achieves signing key size $O(\lambda^2)$ qubits; public-key and signature size $O(\lambda^2)$ bits; key generation and verification algorithms make a constant number of oracle calls, and signing makes $O(\lambda)$ oracle calls.*
2. *In the plain model, assuming the existence of post-quantum secure indistinguishability obfuscation (pqIO) and the post-quantum subexponential learning with errors (LWE) assumption: the scheme achieves signing key size $O(\lambda^2)$ qubits; public-key size $O(\lambda^4)$ bits; and signature size $O(\lambda^2)$ bits.*

We do not attempt to optimize the runtimes of the algorithms in the plain model construction as the limiting factor there is the use of an expensive indistinguishability obfuscation scheme. Still, we note that the signing and verification algorithms are conceptually simpler than those of [SZ25].

Theorem 1.2 (Informal). *There exists an incompressible OSS scheme for λ -bit messages with perfect correctness, strong signature security and incompressibility. As a corollary of this and [ÇGS25], there exists:*

- *a public-key quantum fire (and key-fire) scheme with perfect verification correctness, perfect strong cloning correctness, and untelegraphability security;*
- *a public-key quantum key-fire scheme for any classically-unlearnable functionality with perfect cloning correctness, perfect evaluation correctness, and key-untelegraphability security; and*
- *a public-key encryption scheme with perfect correctness, perfectly clonable yet unbounded-leakage-resilient secret keys, and classical public keys and ciphertexts.*

Moreover, assuming the existence of post-quantum one-way functions, all of these schemes can be made efficient (with computational security).

2 Technical Overview

We start by reviewing the one-shot signature (OSS) scheme of Shmueli and Zhandry [SZ25] (SZ, henceforth), taking care to point out the sources of inefficiency along the way. We then describe our ideas to improve the bandwidth and runtimes of the algorithms.

The SZ work first constructs a primitive called a *non-collapsing collision-resistant hash function* [ARU14, Unr16, SZ25] and invokes the general results of [DS22, AGKZ20] to turn it into a one-shot signature scheme. A non-collapsing hash function is one where the public parameters CRS define a function $H(\text{CRS}, \cdot)$. The novel and inherently quantum property of this hash function is that there should exist a QPT state sampler that, given CRS, generates a state $|\psi\rangle$ and an auxiliary state aux ; and a QPT distinguisher (that we will call the collapsing distinguisher) that distinguishes between the following two experiments. In the first experiment, the challenger measures $|\psi\rangle$ and returns the result to the adversary; in the second, the challenger computes $H(\text{CRS}, \cdot)$ on $|\psi\rangle$ in superposition and measures only the result. In addition, the hash function is collision-resistant in the standard cryptographic sense against quantum polynomial-time adversaries.

To construct such a hash function, SZ uses a bijection (or permutation) $\Pi : \{0, 1\}^n \rightarrow \{0, 1\}^n$ where $n = \Theta(\lambda^2)$, as well as a family of subspaces and cosets $\{(\mathbf{A}_y, \mathbf{b}_y)\}_{y \in \{0, 1\}^r}$ where $\mathbf{A}_y \in \mathbb{Z}_2^{n \times (n-r)}$ and $\mathbf{b}_y \in \mathbb{Z}_2^n$. With these in place, SZ defines

$$H(x) := \Pi(x)_{[1:r]},$$

namely the hash function simply outputs the first r bits of $\Pi(x)$. SZ also defines the following auxiliary functions, which are treated as oracles in the construction and the proof:

- $\mathcal{P}(x) = (y, \mathbf{A}_y \cdot \mathbf{w} + \mathbf{b}_y)$ where $\Pi(x) = y\|\mathbf{w}$: the function $\mathcal{P}$ outputs the first r bits of $\Pi(x)$ as-is, and outputs the last $n - r$ bits encoded into the affine subspace indexed by y .
- $\mathcal{P}^{-1}(y, \mathbf{u}) = \begin{cases} \Pi^{-1}(y\|\mathbf{w}) & \text{if } \exists \mathbf{w} \in \mathbb{Z}_2^{n-r} \text{ such that } \mathbf{A}_y \cdot \mathbf{w} + \mathbf{b}_y = \mathbf{u} \\ \perp & \text{otherwise.} \end{cases}$
- $\mathcal{D}$ is the function that checks if a given vector $\mathbf{v}$ belongs to the dual of the subspace indexed by y . That is,

$$\mathcal{D}(y, \mathbf{v}) = \begin{cases} 1 & \text{if } \mathbf{v}^T \cdot \mathbf{A}_y = 0^{n-r} \\ 0 & \text{otherwise.} \end{cases}$$

Given oracle access to $\mathcal{P}$, $\mathcal{P}^{-1}$ and $\mathcal{D}$, the construction of the collapsing distinguisher is straightforward: the state sampler produces the uniform superposition state $|\psi\rangle = \sum_x |x\rangle$ (no auxiliary information is necessary). In the first experiment, the adversary will have access to the basis state $|x\rangle$ for some x , while in the second experiment, the adversary holds $|\psi_y\rangle := \sum_{x: H(x)=y} |x\rangle$ for some y . Distinguishing between the two states is straightforward: one can apply $\mathcal{P}$ in the forward direction (and use $\mathcal{P}^{-1}$ to uncompute x) to get either

$$|y, \mathbf{z}\rangle \quad \text{or} \quad |y\rangle \otimes \sum_{\mathbf{w} \in \mathbb{Z}_2^{n-r}} |\mathbf{A}_y \mathbf{w} + \mathbf{b}_y\rangle$$

where $\mathbf{z} \in \mathbb{Z}_2^{n-r}$ is just some string. Doing a Hadamard transform on the second register results in either a full superposition over $\mathbb{Z}_2^n$ (with phases depending on $\mathbf{z}$) or a uniform superposition over the dual subspace of $\mathbf{A}_y$ (again, with phases depending on $\mathbf{b}_y$). Now, applying $\mathcal{D}$ in superposition on y together with the second register results in 0 in the first case with overwhelming probability, and 1 in the second case with probability 1, distinguishing between the two cases.

Demonstrating collision-resistance of H is significantly more difficult: we do not go into detail about the proof and note only that the proof of collision resistance of our new scheme closely follows the proof

strategy of [SZ25]. We now explain how [DS22] and [AGKZ20] produce a one-shot signature scheme from a non-collapsing and collision-resistant hash function.

The template of the OSS construction, essentially from [AGKZ20], is to let the public key of the OSS scheme be an element $y \in \mathbb{Z}_2^r$ in the image of the hash function H . The signature of a bit b with respect to the public key y is $x \in \mathbb{Z}_2^n$ such that $H(x) = y$ and $x[1] = b$, i.e. a hash preimage of y whose first bit is b . All one needs now to sign a bit b is to produce a hash preimage of y that starts with the bit b ; this is where one needs an equivocator for the hash function. The key innovation is a way to use the collapsing distinguisher, together with the pre-image state $|\psi_y\rangle$ and the oracles, to build an equivocator; this is done using the reduction of Dall'Agnol and Spooner [DS22].

While the [DS22] reduction from equivocating to distinguishing only considers non-collapsing commitment schemes, we outline how a similar reduction might work for non-collapsing and collision-resistant hash functions.³ The first step of the [DS22] reduction is to use the collapsing distinguisher for the hash function to construct an algorithm that breaks sum binding, that is, an adversary that can produce preimages of a hash that begin with either 0 or 1 on demand. The sum of the probabilities p_0 and p_1 that it succeeds in producing preimages that start with 0 and 1 respectively, should be non-negligibly larger than 1. This may only succeed with a small, but non-negligible, probability. Therefore, the second step of [DS22] is an amplification step which uses the sum binding algorithm in parallel on many states $|\psi_{y_i}\rangle$.

The first step of [DS22], the reduction from a collapsing distinguisher D (which we will think of as a projective measurement) to sum-binding algorithm S , works roughly as follows: S will measure the first bit of the preimage x . Upon being asked to open to bit b , if the measured bit is b , the algorithm is all set: it can measure all of x . Otherwise, it will try to use the projective measurement $(D, I - D)$ before measuring x . The hope is that when $x_1 \neq b$, the collapsing distinguisher D will allow S to project the partially measured state back to a subspace with non-trivial weight on preimages with $x_1 = b$. This reduction is mildly lossy: if D distinguishes with probability $\frac{1}{2} + \epsilon$, then S succeeds with probability roughly $\frac{1}{2} + O(\epsilon^2)$.

At this stage, one needs to amplify the sum-binding algorithm S into an equivocator E , which must succeed with overwhelming probability. A naïve attempt might be as follows: after measuring a hash value, E may try to equivocate by running an alternating projectors/Marriott-Watrous style rewinding algorithm using S . Unfortunately, for technical reasons that we will explain in more detail in Section 2.3, this requires multiple copies of the state. Roughly speaking, with only one copy, we may get unlucky and require a super-polynomial amount of time to equivocate.

Dall'Agnol and Spooner [DS22] deal with this issue by using many preimage states $|\psi_{y_i}\rangle$ and a QSVT/quantum rewinding circuit to produce an equivocator. This step introduces a fair bit of overhead; in particular, if S succeeds with probability $\frac{1}{2} + \epsilon$, equivocating to a single bit with probability $1 - 2^{-\lambda}$ requires roughly λ/ϵ^2 preimage states, and the same number of hash functions. Overall, the [DS22] reduction will produce an equivocator with $\Theta(\lambda)$ overhead in signing.⁴

Finally, to produce a one-shot signature scheme for λ -bit messages, one needs to repeat this scheme in parallel, which introduces another λ overhead. This again seems unavoidable with the [SZ25] construction as we explain in Section 2.3.

Thus, even with a collapsing disuguisher that succeeds with probability a constant larger than $1/2$, the signature scheme that results from [SZ25] ends up with $\Theta(\lambda) \cdot \lambda \cdot \Theta(\lambda^2) = \Theta(\lambda^4)$ qubit signing keys, $\lambda \cdot \Theta(\lambda^2) = \Theta(\lambda^3)$ -bit signatures, and $\lambda \cdot \Theta(\lambda^2) = \Theta(\lambda^3)$ -bit public keys. The runtime of the signing algorithm will likely also require close to $\Omega(\lambda^4)$ queries, where an $\Omega(\lambda^2)$ overhead comes from the QSVT circuit, to the underlying programs/oracles $\mathcal{P}$, $\mathcal{P}^{-1}$ and $\mathcal{D}$, while the key generation and verification algorithms will require $\Theta(\lambda)$ queries.

³Technically speaking, the [SZ25] distinguisher is not actually enough for the [DS22] reduction to go through. There is an easy fix to this, as we explain in Section 2.3.

⁴One could move this overhead into the key generation algorithm, but ultimately either the key generation algorithm or signing algorithm will incur $\Theta(\lambda)$ overhead.

2.1 Our Construction

As noted earlier, [SZ25] technically only provides a distinguisher for the complete measurement of the preimage state, which is not enough to apply the [DS22] reduction. Fortunately, in Appendix G, we observe that their distinguisher has a constant advantage on a known index: the first bit. Using this fact allows us to break sum binding with probability $\frac{1}{2} + c$ for some constant $c > 0$, thus allowing the [DS22] reduction to go through (see Section 2.3 for more details). However, the final $\Theta(\lambda)$ overhead of the [DS22] reduction seems unavoidable without significant modification. Additionally, there is no clear way to boost the [SZ25] scheme to one which has perfect correctness, since the collapsing distinguisher already begins with imperfect advantage.

Our construction avoids the [DS22] and [AGKZ20] reductions altogether by constructing a one-shot signature scheme in one go. Our main idea is simple: instead of signing a bit b by looking for it in the hash preimage, we will look for b in the *image*. That is to say, to sign a bit b with respect to a public key y , we will produce a vector $\vec{\sigma}$ such that $\vec{\sigma}[1] = b$ and $\mathcal{P}(\mathbf{x}) = (y, \vec{\sigma})$ for some $\mathbf{x}$.

To see that this new scheme has an equivocator, we rely on intuition from [AGKZ20]: we can easily implement a reflection oracle across valid coset vectors $\vec{\sigma}$ such that $\vec{\sigma}[1] = b$, while the dual oracle/program D (essentially) provides a reflection oracle across the state

$$|\psi_y\rangle := \sum_{\mathbf{x} : H(\mathbf{x})=y} |\mathbf{A}_y \cdot \mathbf{w}_x + \mathbf{b}_y\rangle .$$

where $\mathbf{w}_x$ is the last $n - r$ bits of $\Pi(\mathbf{x})$. Note that $|\psi_y\rangle$ is the uniform superposition over the affine subspace $(\mathbf{A}_y, \mathbf{b}_y)$. This naturally leads us to attempt to implement a Grover search algorithm to sign a bit. Because we have switched to the setting where the bit to be signed is in the image of $\mathcal{P}$ rather than the pre-image, we know that with overwhelming probability, the coset $(\mathbf{A}_y, \mathbf{b}_y)$ contains vectors that begin with b , in which case precisely half of them do (unless the first row of $\mathbf{A}_y$ is the zero vector, a case we will deal with shortly) and so Grover search will succeed. Producing a signature directly is now simple: given $|\psi_y\rangle$, we can use Grover search using D to transform it into

$$|\psi_{y,b}\rangle := \sum_{\mathbf{x} : \mathcal{P}(\mathbf{x})_{[1:r+1]}=y||b} |\mathbf{A}_y \cdot \mathbf{w}_x + \mathbf{b}_y\rangle .$$

Once we have done so, we can simply measure $|\psi_{y,b}\rangle$ to get a coset vector $\vec{\sigma}$ such that $\vec{\sigma}[1] = b$ and $\mathcal{P}(\mathbf{x}) = (y, \vec{\sigma})$ for some $\mathbf{x}$. Verification simply consists of applying $\mathcal{P}^{-1}$ to a supposed coset vector $\vec{\sigma}$ and checking this condition.

By construction, any two message-signature pairs which are valid with respect to the same public key/hash y (after an invocation of $\mathcal{P}^{-1}$) give us $\mathbf{x}_0 \neq \mathbf{x}_1$ such that $\mathcal{P}(\mathbf{x}_0)_{[1:r]} = \mathcal{P}(\mathbf{x}_1)_{[1:r]}$ – but this is exactly a collision in the hash function of [SZ25]! Thus, in this simple setting, we get collision resistance essentially for free. As a corollary, this also allows us to easily achieve the stronger notion of security for our one-shot signature scheme, where it is not only hard to produce a signature of a different message, but also to produce a new signature of the same message.

There are still two undesirable properties in this basic construction: imperfect correctness and short messages. First, observe that our equivocation/signing algorithm will only work if the coset $(\mathbf{A}_y, \mathbf{b}_y)$ has vectors that start with the bit we want to sign, which does not happen with probability 1. Our next idea, therefore, is to force this to be the case. Specifically, note that the bad event happens when all the column vectors of $\mathbf{A}_y$ begin with 0. With this in mind, we will instead sample $\mathbf{A}_y$ by first sampling a random column vector that has a 1 in the first coordinate, followed by $n - r - 1$ linearly independent vectors with 0's in the first coordinate, thereby guaranteeing that our cosets are “good”.

The second undesirable property of this basic scheme is that it only allows us to sign single-bit messages, which is the same issue that we encountered with [SZ25]. Consider for a moment the most natural extension

to allow us to sign λ -bit messages: simply equivocate to the first λ bits of the coset register. Of course, this will break our signing algorithm, because we only have a dual oracle for A_y ! However, there is a crucial difference between our signature scheme and that of [SZ25]: unlike in the [SZ25] construction, where the set of preimages whose first λ bits have been fixed are essentially a random collection of points, our images are very structured: indeed, the coset (A_y, b_y) , after fixing the first i bits to arbitrary values, is *still a coset*. Thus, we can modify D to provide a series of oracles for the dual of the (A_y, b_y) cosets restricted to the first i bits being fixed; note that the duals do not depend on the particular fixing of the bits. Once we have done so, the signing algorithm can once again use D to essentially sign bit-by-bit *before* measuring an image, allowing us to compress λ signatures into a single signature.

With some work, we show that even with this set of oracles/programs, our scheme remains secure. To do so, we prove several lemmas about random subspaces (see Appendix D) and modify the security proof of [SZ25] as necessary (see Appendices E and F for more details). This gives us a signature scheme with $\Theta(\lambda^2)$ qubit signing keys (which consists of just one preimage state), $\Theta(\lambda^2)$ -bit public keys and signatures for λ -bit messages, and a simple, iterative signing algorithm that makes one call to D per bit of the message being signed. Verification is also more efficient, as we can now verify a signature for a λ -bit message with a single call to P^{-1} .

To sign arbitrary polynomial-length messages, we use a random oracle to compress the message to λ bits before using our scheme to sign; this hash-and-sign transformation maintains the stronger security notion we want as well as perfect correctness.

Our construction in the plain model looks largely the same as our oracle model construction, where we replace our permutation with a pseudorandom permutation (PRP) and random functions with pseudorandom functions (PRFs) before obfuscating all our programs instead of providing oracle access, similar to [SZ25]. One difference between the oracle and plain model constructions is that the public key becomes larger due to padding; this is also a feature/bug of the [SZ25] construction. We note that in our construction, the public key used to verify a λ -bit signature is the same size as the public keys used to verify a single bit in [SZ25], which we estimate to be around $\Theta(\lambda^4)$ bits. In the plain model, we can instantiate the random oracle with any arbitrary-domain collision-resistant hash function to extend the scheme to polynomial length messages.

2.2 Application to Quantum Fire

Our new OSS construction also allows us to construct public-key quantum fire, a notion first formalized in [BNZ25]. The recent result of [ÇGS25] constructed such a scheme (along with other applications of quantum fire) with respect to a classical oracle using the OSS scheme of [SZ25]. However, their proof relies on the existence of a one-shot signature scheme with strong incompressibility security, which does *not* hold for the [SZ25] construction.⁵

Let us briefly recall the strong incompressibility guarantee: we say that a one-shot signature scheme $(\text{Gen}, \text{Sign}, \text{Ver})$ relative to a tuple of classical oracles $(\mathcal{O}_{\text{Gen}}, \mathcal{O}_{\text{Sign}}, \mathcal{O}_{\text{Ver}})$ respectively, has strong signature incompressibility if for all query-bounded quantum adversaries $\mathcal{A}_1$, the following is true with overwhelming probability: given oracle access to $(\mathcal{O}_{\text{Gen}}, \mathcal{O}_{\text{Sign}}, \mathcal{O}_{\text{Ver}})$, for any polynomial-sized string L produced by $\mathcal{A}_1$, there is a list $\mathcal{L}_L$ of size $|\mathcal{L}_L| \leq \text{poly}(|L|, \lambda)$ such that no query-bounded quantum adversary $\mathcal{A}_2$ with access to only $\mathcal{O}_{\text{Sign}}$ and $\mathcal{O}_{\text{Ver}}$ can produce valid signatures with respect to verification keys outside of $\mathcal{L}_L$ (with noticeable probability).

We observe that the [SZ25] construction is *not incompressible* for at least two reasons:

1. First, the signing algorithm uses both the P and P^{-1} oracles, and given these two oracles, $\mathcal{A}_2$ can

⁵It is potentially possible that the *one-bit* [SZ25] construction satisfies a weaker version of incompressibility where the oracles the adversary gets are quantum rather than classical, but we leave the question of whether this is the case to future work.

ignore the string L it receives and simply run the key generation algorithm before measuring a random valid key-message-signature tuple. Each verification key appears with equal (but exponentially small) probability, so for any fixed list $\mathcal{L}_L$, $\mathcal{A}_2$ will almost always produce signatures with respect to verification keys which are outside of $\mathcal{L}_L$. [ÇGS25] incorrectly claim that the [SZ25] signing algorithm does not require access to $\mathcal{P}$ and $\mathcal{P}^{-1}$.

2. Second, the [SZ25] construction for λ -bit messages (which is what is needed for [ÇGS25]) is simply a parallel repetition of the one-bit OSS scheme. However, schemes based on parallel repetition are generically *highly compressible*: the adversary $\mathcal{A}_1$ can generate two valid key-message-signature tuples $(pk_{i,0}, 0, \sigma_{i,0})$, $(pk_{i,1}, 1, \sigma_{i,1})$ for each bit and store this as the string L . The adversary $\mathcal{A}_2$ can then output a valid signature with respect to an exponential number of verification keys, by mixing and matching the signatures for each bit.

Nevertheless, we show that a simple modification of our many-bit one-shot signature scheme *does* have strong signature incompressibility and thereby recover the results of [ÇGS25]. In fact, since this modified one-shot signature scheme has perfect correctness, the corresponding public-key quantum fire scheme (and all other related schemes) will also have perfect correctness. In Section 5, we elaborate on how to tweak our scheme from Section 4 to produce an OSS scheme with signature incompressibility.

2.3 On the Use of the [DS22] Reduction in [SZ25]

As noted earlier, [SZ25] provides a collapsing distinguisher between the case of measuring the hash value alone versus measuring the entire preimage. On the other hand, the [DS22] reduction is only able to construct equivocal hashes/commitments from non-collapsing *bit* commitments/hash functions. Thus, applying their reduction would require a proof that there is a distinguisher with good advantage between measuring the hash value alone versus measuring the hash value together with *some specific bit* of the preimage. It turns out that this is not hard to show; we include a proof that the [SZ25] distinguisher itself has this property in Appendix G.

We alluded to the possibility of using a simple Marriott-Watrous style algorithm for “amplifying” the success probability of a sum-binding adversary and turn it into a equivocal adversary. We argue that this is unlikely to work, at least without significant further analysis of the specifics of the [SZ25] scheme. Recall that to analyze the runtime of the alternating projections algorithm between projections Π_A and Π_B , we need to consider the Jordan decomposition between Π_A and Π_B . Specifically, Jordan’s lemma states that there is an orthogonal decomposition of the Hilbert space $\mathcal{H}$ upon which Π_A and Π_B act into one-dimensional or two-dimensional subspaces S_j , such that S_j is invariant under both Π_A and Π_B . This decomposition also has the property that on two-dimensional subspaces S_j , Π_A and Π_B are rank-one projectors onto the states $|v_j\rangle$ and $|w_j\rangle$, respectively, which defines the values $p_j = \langle w_j | \Pi_A | w_j \rangle$, the cosine of the angle between v_j and w_j . The runtime of an average execution of the Marriott-Watrous algorithm starting with the state $|\psi\rangle = \sum_j \alpha_j |w_j\rangle$ can be equivalently described as first sampling j with probability $|\alpha_j|^2$ and then flipping a p_j -biased coin repeatedly until a head appears; thus, the expected runtime is $\sum_j |\alpha_j|^2 / p_j$. However, the only clear guarantee about the state $|\psi\rangle$ is that the *average* probability $\sum_j |\alpha_j|^2 \cdot p_j$ is good, i.e. some small constant. It might be the case that one of the p_j ’s is very small (potentially exponentially so), in which case the average runtime may become superpolynomial or exponential in the unlucky scenario where the j sampled corresponds to this bad angle p_j . This motivates the need to consider multiple copies of the preimage state, as [DS22] does, which introduces overhead into the key generation and/or signing algorithms.

While it is potentially possible that a more detailed analysis of the Jordan decomposition could show that one could avoid using a rewinding/QSVT circuit, such an equivocator would still not be able to sign more than $O(\log \lambda)$ bits at a time, since after $\omega(\log \lambda)$ bits of the preimage state have been measured, the

state does not admit any subspace/coset structure, and the advantage of the [SZ25] distinguisher vanishes completely.

2.4 Open Problems

We end this section with two open problems. First, the elephant in the room: can we improve the size of all parameters in the oracle model to $\Theta(\lambda)$ bits? For the reader familiar with the [SZ25] proof, we offer a brief explanation why this is likely to require completely new techniques: in the oracle model, the bottleneck currently lies in analyzing the dual-free case, which requires collision resistance despite a coset structure. [SZ25] sidesteps this issue by using the parallel repetition of a 2-to-1 function and taking advantage of the fact that any pair of preimages is a coset for the 2-to-1 and noting that repetition preserves coset structure; this causes a λ factor blowup. A more direct analysis could yield better parameters: in particular, the recent frameworks for compressed permutation oracles [MMW25, Car25] may prove useful in deriving better query lower bounds. This could eliminate the λ factor blowup and result in (optimal) $\Theta(\lambda)$ -bit public keys, signatures, and signing states.

Secondly, one can ask if the parameters of the plain model construction can be improved. Here, the blowup due to the parallel repetition of a 2-to-1 function shows up as well, and it is less clear how to deal with this issue. Additionally, in the [SZ25] proof, the base 2-to-1 function needs more properties (essentially, the function needs to be a simplified version of a NTCF), which forces a blowup in the public-key size. We suspect that entirely new techniques will be needed to significantly improve the parameters of our construction (and that of [SZ25]) in the plain model.

3 Preliminaries

Notations. We will let PPT denote classical probabilistic polynomial-time algorithms, and QPT quantum polynomial-time algorithms.

For a matrix $A \in \mathbb{Z}_2^{n \times m}$, let $A^{[i:j]}$ denote the i th through j th columns of A , and let $A^{[k]}$ refer to the k th column of A . For a matrix A as above, we will let $\text{ColSpan}(A)$ denote the subspace generated by the columns of A . Throughout this work, we will slightly abuse notation and refer to spans (denoted by $\text{span}\{\cdot\}$) of sets of vectors as well as sets of vectors and subspaces. For example, $\text{span}\{\text{ColSpan}(A), \mathbf{e}_1\}$ refers to the set of all vectors which can be written as the linear combination of the unit vector $\mathbf{e}_1$ and the column vectors of A .

3.1 One-Shot Signatures

Definition 1 (One-Shot Signatures, adapted from [AGKZ20, SZ25]). A one-shot signature (OSS) scheme is a tuple of algorithms ($\text{Setup}, \text{Gen}, \text{Sign}, \text{Ver}$) together with message-space $(\mathcal{M}_\lambda)_{\lambda \in \mathbb{N}}$ with the following interface:

- $\text{CRS} \leftarrow \text{Setup}(1^\lambda)$: A PPT algorithm that, given the security parameter λ , samples a classical common reference string CRS.
- $(\text{pk}, |\text{sk}\rangle) \leftarrow \text{Gen}(\text{CRS})$: A QPT algorithm that takes the classical CRS and samples a classical public key pk and a quantum secret key $|\text{sk}\rangle$.
- $\sigma \leftarrow \text{Sign}(\text{CRS}, |\text{sk}\rangle, m)$: A QPT algorithm that, given CRS, the quantum key $|\text{sk}\rangle$ and any message $m \in \mathcal{M}_\lambda$, produces a classical signature σ .
- $\text{Ver}(\text{CRS}, \text{pk}, m, \sigma) \in \{0, 1\}$: A classical deterministic polynomial-time algorithm which verifies a message m and signature σ relative to a public key pk and a common reference string CRS.

The algorithms satisfy the following correctness and security properties:

- **Perfect Correctness:** For any $\lambda \in \mathbb{N}$ and $m \in \mathcal{M}_\lambda$,

$$\Pr \left[\begin{array}{l} \text{Ver}(\text{CRS}, \text{pk}, m, \sigma_m) = 1 \\ \text{CRS} \leftarrow \text{Setup}(1^\lambda) \\ (\text{pk}, |\text{sk}\rangle) \leftarrow \text{Gen}(\text{CRS}) \\ \sigma_m \leftarrow \text{Sign}(\text{CRS}, |\text{sk}\rangle, m) \end{array} \right] = 1.$$

- **Strong Security:** For any QPT algorithm $\mathcal{A}$ and for all $\lambda \in \mathbb{N}$,

$$\Pr \left[\begin{array}{l} (m_0, \sigma_0) \neq (m_1, \sigma_1) \wedge \\ \text{Ver}(\text{CRS}, \text{pk}, m_0, \sigma_0) = 1 \wedge \\ \text{Ver}(\text{CRS}, \text{pk}, m_1, \sigma_1) = 1 \\ \text{CRS} \leftarrow \text{Setup}(1^\lambda) \\ (\text{pk}, m_0, m_1, \sigma_0, \sigma_1) \leftarrow \mathcal{A}(\text{CRS}) \end{array} \right] = \text{negl}(\lambda).$$

Remark 3.1. Both the correctness and the security properties in this definition are stronger than [SZ25].

Our perfect correctness property guarantees that for any message, the signing algorithm produces a valid signature with probability 1 (over the CRS and the internal randomness of Gen and Sign). One could consider a weaker notion of correctness where, for every message m , for most CRS, the (strict) quantum polynomial-time signing algorithm produces a correct signature with prob $1 - \text{negl}(\lambda)$. The construction of [SZ25] only achieves this weaker form of correctness, and seemingly inherently so.

Additionally, one may consider a weaker security notion, where an adversary may be able to produce multiple valid signatures for the same message, but not for different messages. Both our construction and the construction of [SZ25] (the latter, along with our proof in Appendix G) achieve the strong security notion. We note, however, that a generically non-collapsing and binding commitment scheme would only give rise to a one-shot signature scheme with the weaker form of security via the [DS22, AGKZ20] reduction.

4 Our Construction in the Oracle Model

We begin by describing our one-shot signature scheme that allows us to sign λ -bit messages in the oracle model.

4.1 The Construction

Let $\lambda \in \mathbb{N}$ be the security parameter and ℓ denote the length of the messages we sign. We use the following parameters:

$$s := 16 \cdot \lambda, r := s \cdot (\lambda - 1), \ell = \ell(\lambda) := \lambda, \text{ and } n := r + \ell + \frac{3}{2} \cdot s.$$

Let $\Pi : \{0, 1\}^n \rightarrow \{0, 1\}^n$ be a random permutation and let $F : \{0, 1\}^r \rightarrow \{0, 1\}^{n-(r+1)}$ be a random function. For each $\mathbf{y} \in \{0, 1\}^r$, let $\mathbf{B}_\mathbf{y} \in \mathbb{Z}_2^{(n-\ell) \times \ell}$ be a random matrix, $\mathbf{C}_\mathbf{y} \in \mathbb{Z}_2^{(n-\ell) \times (n-r-\ell)}$ a random matrix with full column-rank, and $\mathbf{b}_\mathbf{y} \in \mathbb{Z}_2^n$ be a uniformly random vector, all generated by the output randomness of $F(\mathbf{y})$. Define

$$\mathbf{A}_\mathbf{y} := \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{B}_\mathbf{y} & \mathbf{C}_\mathbf{y} \end{bmatrix} \in \mathbb{Z}_2^{n \times (n-r)}.$$

Then, let $\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}$ be functions (treated as oracles by all algorithms) defined as follows:

- $\mathcal{P} : \{0, 1\}^n \rightarrow (\{0, 1\}^r \times \mathbb{Z}_2^n)$ is defined to be

$$\mathcal{P}(x) = (\mathbf{y}, \mathbf{A}_\mathbf{y} \cdot \mathbf{w} + \mathbf{b}_\mathbf{y}) \text{ where } \Pi(x) = \mathbf{y} \parallel \mathbf{w}.$$

- $\mathcal{P}^{-1} : (\{0, 1\}^r \times \mathbb{Z}_2^n) \rightarrow \{0, 1\}^n \cup \{\perp\}$ is defined to be

$$\mathcal{P}^{-1}(\mathbf{y}, \mathbf{u}) = \begin{cases} \Pi^{-1}(\mathbf{y} \parallel \mathbf{w}) & \text{if } \exists \mathbf{w} \in \mathbb{Z}_2^{n-r} \text{ such that } \mathbf{A}_y \cdot \mathbf{w} + \mathbf{b}_y = \mathbf{u} \\ \perp & \text{otherwise} \end{cases}$$

- $\mathcal{D} : ([\ell + 1] \times \{0, 1\}^r \times \mathbb{Z}_2^n) \rightarrow \{0, 1\}$ is defined to be

$$\mathcal{D}(j, \mathbf{y}, \mathbf{v}) = \begin{cases} 1 & \text{if } \mathbf{v}^T \cdot \mathbf{A}_y^{[j:n-r]} = 0^{n-r-j+1} \text{ and } j \in [\ell + 1] \\ 0 & \text{otherwise} \end{cases}$$

The above distribution of oracles is denoted $\mathcal{O}_{n,r,\ell}$. We define our hash function $H(\mathbf{x})$ to be the first r bits of $\Pi(\mathbf{x})$, so

$$H(\mathbf{x}) = \Pi(\mathbf{x})_{[1:r]}$$

We will take the message space $\mathcal{M}_\lambda$ to be the set of all strings $m \in \{0, 1\}^{\ell(\lambda)}$. (In Appendix C, we show how to sign messages of arbitrary length, so we consider for now only messages with one fixed length for a given security parameter.)

Our OSS Scheme. With this setup, we are ready to construct our one-shot signature scheme; the scheme is described in Figure 1. Since we are working in the oracle model, we omit Setup and let $\mathcal{O}_{n,r,\ell}$ play the role of the CRS.

The key generation and verification algorithms are self-explanatory, but the signing algorithm, which follows the general framework of [AGKZ20], requires a bit of elaboration. For simplicity, consider the process of signing a single bit, say 1. We start with the signing state $|\mathbf{sk}\rangle$ which is a superposition of all vectors in the affine subspace $(\mathbf{A}_y, \mathbf{b}_y)$. Indeed,

$$|\mathbf{sk}\rangle = \frac{1}{\sqrt{2}}(|\mathbf{sk}_0\rangle + |\mathbf{sk}_1\rangle)$$

where $|\mathbf{sk}_b\rangle$ is the set of all vectors in the affine subspace $(\mathbf{A}_y, \mathbf{b}_y)$ whose first coordinate is the bit b . Note that the support of $|\mathbf{sk}_b\rangle$ is the set of all valid signature of the bit b . Applying the oracle $\mathcal{O}_1$ from Figure 2 with $m = 1$ to $|\mathbf{sk}\rangle$ results in the state $\frac{1}{\sqrt{2}}(|\mathbf{sk}_0\rangle + i|\mathbf{sk}_1\rangle)$.

Denote the result of the Hadamard transform applied to $\frac{1}{\sqrt{2}}(|\mathbf{sk}_0\rangle + |\mathbf{sk}_1\rangle)$ as $|\overline{\mathbf{sk}_0}\rangle$, and that applied to $\frac{1}{\sqrt{2}}(|\mathbf{sk}_0\rangle - |\mathbf{sk}_1\rangle)$ as $|\overline{\mathbf{sk}_1}\rangle$. The key observation is that the unitary $(i - 1)\mathcal{D}(\text{iter}, \mathbf{y}, \cdot) + I$ transforms $|\overline{\mathbf{sk}_0}\rangle$ to $i|\overline{\mathbf{sk}_0}\rangle$ and leaves $|\overline{\mathbf{sk}_1}\rangle$ unchanged.

Now, applying the oracle $\mathcal{O}_2$ results in the following sequence of transformations:

$$\begin{aligned} & \frac{1}{\sqrt{2}}(|\mathbf{sk}_0\rangle + i|\mathbf{sk}_1\rangle) \xrightarrow{H^{\otimes n}} \frac{1}{2}(|\overline{\mathbf{sk}_0}\rangle + |\overline{\mathbf{sk}_1}\rangle) + \frac{i}{2}(|\overline{\mathbf{sk}_0}\rangle - |\overline{\mathbf{sk}_1}\rangle) \\ & \xrightarrow{(i-1)\mathcal{D}(\text{iter}, \mathbf{y}, \cdot) + I} \frac{i(1+i)}{2}|\overline{\mathbf{sk}_0}\rangle + \frac{(1-i)}{2}|\overline{\mathbf{sk}_1}\rangle = \frac{(i-1)}{2}(|\overline{\mathbf{sk}_0}\rangle - |\overline{\mathbf{sk}_1}\rangle) \xrightarrow{H^{\otimes n}} \frac{(i-1)}{\sqrt{2}}|\mathbf{sk}_1\rangle \end{aligned}$$

This final state is simply the superposition over all valid signatures of the bit 1, so measuring it in the standard basis does the job.

Remark 4.1. Each iteration of the Grover search will add a global phase to the signing token; this does not affect the correctness of the algorithm as this phase simply accumulates over iterations and the final step of the algorithm is to measure the token state, thereby erasing the phase. For our applications, we will not need to implement a controlled version of the signing algorithm, for which a global phase could

Our OSS Scheme in the Classical Oracle Model

Gen ^{$\mathcal{P}, \mathcal{P}^{-1}$} : Prepare the state $|+\rangle^{\otimes n} = \frac{1}{2^{n/2}} \cdot \sum_{\mathbf{x} \in \{0,1\}^n} |\mathbf{x}\rangle$ and apply $\mathcal{P}$. Measure H to get a string $\mathbf{pk} := \mathbf{y}$ and the state

$$\propto \sum_{\mathbf{x}: H(\mathbf{x})=\mathbf{y}} |\mathbf{x}\rangle |A_{\mathbf{y}} \cdot \mathbf{w}_{\mathbf{x}} + \mathbf{b}_{\mathbf{y}}\rangle .$$

where $\mathbf{w}_{\mathbf{x}} = \Pi(\mathbf{x})_{[r+1:n]}$. Apply $\mathcal{P}^{-1}(\mathbf{y}, \cdot)$ to uncompute $\mathbf{x}$, resulting in the state

$$|\mathbf{sk}\rangle \propto \sum_{\mathbf{x}: H(\mathbf{x})=\mathbf{y}} |A_{\mathbf{y}} \cdot \mathbf{w}_{\mathbf{x}} + \mathbf{b}_{\mathbf{y}}\rangle .$$

Output $(\mathbf{pk}, |\mathbf{sk}\rangle)$.

Ver ^{$\mathcal{P}^{-1}$} ($\mathbf{pk}, m, \vec{\sigma}$): Output 1 if $\vec{\sigma}_{[1:\ell]} = m$ and $\mathcal{P}^{-1}(\mathbf{pk}, \vec{\sigma}) \neq \perp$; else, output 0.

Sign ^{$\mathcal{D}$} ($\mathbf{pk}, |\mathbf{sk}\rangle, m$): Initialize $|\psi_0\rangle := |\mathbf{sk}\rangle$ and parse $\mathbf{pk} = \mathbf{y}$.

For $\text{iter} = 1$ to ℓ :

- Define

$$O_1(\text{iter}, m, \mathbf{y}) := (i - 1) \sum_{\vec{\sigma}[1:\text{iter}] = m[1:\text{iter}]} |\vec{\sigma}\rangle \langle \vec{\sigma}| + I$$

to be the first Grover reflection oracle that, on input $\vec{\sigma} \in \mathbb{Z}_2^n$, adds an imaginary phase i to every $\vec{\sigma}$ such that $\vec{\sigma}[1:\text{iter}] = m[1:\text{iter}]$.

- Define

$$O_2^{\mathcal{D}}(\text{iter}, m, \mathbf{y}) := H^{\otimes n}((i - 1)\mathcal{D}(\text{iter}, \mathbf{y}, \cdot) + I)H^{\otimes n}$$

to be the second Grover reflection oracle that, on input $\mathbf{v} \in \mathbb{Z}_2^n$, adds an imaginary phase i to every $\mathbf{v}$ in the dual of the submatrix $A_{\mathbf{y}}^{[\text{iter}:n-r]}$ generated by all but the first $\text{iter} - 1$ columns of $A_{\mathbf{y}}$.

- Compute

$$|\psi_{\text{iter}}\rangle := O_2^{\mathcal{D}}(\text{iter}, m, \mathbf{y})O_1(\text{iter}, m, \mathbf{y})|\psi_{\text{iter}-1}\rangle .$$

Given $|\psi_{\ell}\rangle$, measure in the standard basis to get a string $\vec{\sigma} \in \{0,1\}^n$. Output $\vec{\sigma}$.

Figure 2: Our OSS scheme in the classical oracle model using oracles $\mathcal{P}$, $\mathcal{P}^{-1}$ and $\mathcal{D}$. For ease of exposition, we choose to give the signing algorithm the public key $\mathbf{pk}$ in addition to the state $|\mathbf{sk}\rangle$; this is without loss of generality.

make a difference [Zha25a]. However, if that were to be desired in a future application, one can easily eliminate the global phase either by (a) extending our scheme to only sign messages whose lengths are the nearest multiple of 8; this will erase the global phase as $((i - 1)/\sqrt{2})^8 = 1$, or (b) applying another phase gate, similar to $\mathcal{O}_1$, at the end of each iteration.

Remark 4.2. We note that in our oracle (and plain model) construction, the signing algorithm only needs to call $\mathcal{D}$ with indices j where $1 \leq j \leq \ell$, even though we also allow access to $j = \ell + 1$. This is purely for notational convenience in the security proof and not necessary for correctness.

4.2 Perfect Correctness

In this section, we show that our signing algorithm always produces a valid signature for any message.

Lemma 4.3. *For all security parameters $\lambda \in \mathbb{N}$ and messages $m \in \mathcal{M}_\lambda$,*

$$\Pr \left[\text{Ver}^{\mathcal{P}^{-1}}(\text{pk}, m, \sigma_m) = 1 \quad : \quad \begin{array}{l} (\text{pk}, |\text{sk}\rangle) \leftarrow \text{Gen}^{\mathcal{P}, \mathcal{P}^{-1}}, \\ \sigma_m \leftarrow \text{Sign}^{\mathcal{D}}(|\text{sk}\rangle, m) \end{array} \right] = 1.$$

where the probability is over the experiments and the choice of the oracles $\mathcal{O}_{n,r,\ell}$.

Proof. First, observe that for any $\mathbf{y}$, the coset

$$W_{\mathbf{y}} := \{\mathbf{A}_{\mathbf{y}} \cdot \mathbf{w} + \mathbf{b}_{\mathbf{y}}\}_{\mathbf{w} \in \{0,1\}^{n-r}}$$

contains vectors which start with every prefix in $\{0,1\}^\ell$ by construction. For $m \in \{0,1\}^\ell$ and $0 \leq j \leq \ell$, define the sets

$$W_{\mathbf{y},m,j} := \{\mathbf{w} \in \{0,1\}^n : \mathbf{w}_{[1:j]} = m_{[1:j]} \wedge \mathcal{P}^{-1}(\mathbf{y}, \mathbf{w}) \neq \perp\}$$

to be the set of all valid signatures of m w.r.t. the public key $\mathbf{y}$. We will now fix m .

Since $W_{\mathbf{y}}$ contains vectors that have all prefixes, this also means that it is split equally among all prefixes, i.e. $|W_{\mathbf{y},m,j}| = 2^{n-r-j}$ for all $m \in \{0,1\}^\ell$, $0 \leq j \leq \ell$. Indeed, this follows from the structure of the first ℓ column vectors of $\mathbf{A}_{\mathbf{y}}$ for all $\mathbf{y}$.

For $1 \leq \text{iter} \leq \ell$, the idea is to run a phase-matched variant of Grover search [BHMT00] to transform

$$|W_{\mathbf{y},m,\text{iter}-1}\rangle := \sum_{\mathbf{w} \in W_{\mathbf{y},m,\text{iter}-1}} |\mathbf{w}\rangle \longrightarrow |W_{\mathbf{y},m,\text{iter}}\rangle := \sum_{\mathbf{w} \in W_{\mathbf{y},m,\text{iter}}} |\mathbf{w}\rangle$$

perhaps up to some global phase, so that we can eventually sign m .

In order to run Grover search, we need two reflection oracles: the first is an oracle which reflects marked items, which we have in $O_1(\text{iter}, m, \mathbf{y})$. The second is $U(\text{iter}, m, \mathbf{y}) := (i-1)|W_{\mathbf{y},m,\text{iter}}\rangle\langle W_{\mathbf{y},m,\text{iter}}| + I$; unfortunately, we do not have access to such an oracle. However, as in [AGKZ20], we do have access to $\mathcal{D}$, which we claim suffices.

Observe that since the Grover search will operate on the subspace spanned by $|W_{\mathbf{y},m,\text{iter}}\rangle$ and $|W_{\mathbf{y},m+\mathbf{e}_{\text{iter}},\text{iter}}\rangle$, it suffices to merely implement an oracle which applies a phase of i to $|W_{\mathbf{y},m,\text{iter}-1}\rangle = \frac{1}{\sqrt{2}}(|W_{\mathbf{y},m,\text{iter}}\rangle + |W_{\mathbf{y},m+\mathbf{e}_{\text{iter}},\text{iter}}\rangle)$ and acts as the identity on $|-_{\mathbf{y},m,\text{iter}-1}\rangle := \frac{1}{\sqrt{2}}(|W_{\mathbf{y},m,\text{iter}}\rangle - |W_{\mathbf{y},m+\mathbf{e}_{\text{iter}},\text{iter}}\rangle)$. It is not hard to see that $O_2^{\mathcal{D}}(\text{iter}, m, \mathbf{y})$ is precisely such a reflection oracle. Let us first consider what happens to $|W_{\mathbf{y},m,\text{iter}-1}\rangle$:

1. We begin with the state $|W_{\mathbf{y},m,\text{iter}-1}\rangle = \sum_{\mathbf{w} \in W_{\mathbf{y},m,\text{iter}-1}} |\mathbf{w}\rangle_{\mathbf{W}}$.
2. Applying $H^{\otimes n}$ gives the state $\sum_{d \in (A_{\mathbf{y}}^{[\text{iter}:n-r]})^\perp} |d\rangle_{\mathbf{W}}$.
3. Applying $((i-1)\mathcal{D}(\text{iter}, \mathbf{y}, \cdot) + I)$ adds a phase of i to the state. Applying $H^{\otimes n}$ again results in the state $i \sum_{\mathbf{w} \in W_{\mathbf{y},m,\text{iter}-1}} |\mathbf{w}\rangle_{\mathbf{W}} = i |W_{\mathbf{y},m,\text{iter}-1}\rangle$.

On the other hand, we can consider what happens to $|-_{\mathbf{y},m,\text{iter}-1}\rangle$:

1. We begin with the state

$$|-_{\mathbf{y},m,\text{iter}-1}\rangle = \sum_{\mathbf{w} \in W_{\mathbf{y},m,\text{iter}}} |\mathbf{w}\rangle_{\mathbf{W}} - \sum_{\mathbf{w} \in W_{\mathbf{y},m+\mathbf{e}_{\text{iter}},\text{iter}}} |\mathbf{w}\rangle_{\mathbf{W}}.$$

2. Applying $H^{\otimes n}$ gives the state

$$\sum_{d \in \left(A_y^{[\text{iter}+1:n-r]} \right)^\perp \setminus \left(A_y^{[\text{iter}:n-r]} \right)^\perp} (-1)^{f(d,m)} |d\rangle_W,$$

where f is some Boolean function.

3. Now, applying $((i-1)D(\text{iter}, y, \cdot) + I)$ acts as the identity. Thus, applying $H^{\otimes n}$ again results in the state

$$\sum_{w \in W_{y,m,\text{iter}}} |w\rangle_W - \sum_{w \in W_{y,m+\mathbf{e}_{\text{iter}},\text{iter}}} |w\rangle_W = |_{-y,m,\text{iter}-1}\rangle.$$

Since $|W_{y,m,\text{iter}}| = 2^{n-r-\text{iter}}$ is exactly half of the size of $W_{y,m,\text{iter}-1}$, we will succeed in rotating our state with certainty in just one iteration (up to some global phase) by the correctness of Grover search. Iterating for $1 \leq \text{iter} \leq \ell$ results in the state $|W_{y,m,\ell}\rangle = \sum_{w \in W_{y,m,\ell}} |w\rangle$. All that remains is to measure this state, which gives some string σ_m such that $(\sigma_m)_{[1:\ell]} = m$ and $\mathcal{P}^{-1}(y, \sigma_m) \neq \perp$; this is precisely the condition for the signature verification to succeed. $\square$

4.3 Strong Security

We now show that our new one-shot signature scheme is strongly secure. We use the same general proof strategy as [SZ25], modifying components as needed. As such, we defer most proofs to Appendix E.

We begin by considering what happens when we “bloat” the dual oracle. Let $\mathcal{O}'_{n,r,\ell,s}$ denote the following distribution over oracles $\mathcal{P}$, $\mathcal{P}^{-1}$, and $\mathcal{D}'$. The oracles $\mathcal{P}$ and $\mathcal{P}^{-1}$ are defined identically to the oracles in $\mathcal{O}_{n,r,\ell}$. However, our new dual oracle $\mathcal{D}'$ will be a relaxed version of the original dual oracle $\mathcal{D}$:

$$\mathcal{D}'(j, y, \mathbf{v}) = \begin{cases} 1 & \text{if } \mathbf{v}^T \cdot \begin{bmatrix} \mathbf{A}_y^{[j:\ell]} & \mathbf{A}_y^{[\ell+1+s:n-r]} \end{bmatrix} = 0^{n-r-s-j+1} \text{ and } j \in [\ell+1], \\ 0 & \text{otherwise.} \end{cases}$$

The following lemma states that any adversary $\mathcal{A}$ that finds a collision given access to $\mathcal{P}$, $\mathcal{P}^{-1}$, and $\mathcal{D}$, is also likely to find a collision⁶ given access to $\mathcal{P}$, $\mathcal{P}^{-1}$, and $\mathcal{D}'$.

Lemma 4.4 (The Bloating Lemma). *Let $n, r, s, \ell \in \mathbb{N}^+$ such that $r + s + \ell \leq n$. Suppose there is an oracle-aided q -query quantum algorithm $\mathcal{A}$ such that*

$$\Pr \left[\begin{array}{l} (\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}) \leftarrow \mathcal{O}_{n,r,\ell} \\ (y_0 = y_1) \wedge (x_0 \neq x_1) : \quad \begin{array}{l} (x_0, x_1) \leftarrow \mathcal{A}^{\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}} \\ (y_b, u_b) \leftarrow \mathcal{P}(x_b) \end{array} \end{array} \right] \geq \varepsilon.$$

Define $t := n - r - s$. If $q^3 \cdot n^3 \cdot \frac{1}{\varepsilon^2} \cdot (2^{t-s} + 2^{-t+\ell}) = o(1)$ and $\frac{q^7 \cdot n^{10} \cdot \frac{1}{\varepsilon^4}}{\sqrt{2^{n-r-s-\ell}}} = o(1)$, then

$$\Pr \left[\begin{array}{l} (y_0=y_1 := y) \wedge \\ (u_0-u_1) \notin \text{ColSpan} \left(\begin{bmatrix} \mathbf{A}_y^{[1:\ell]} & \mathbf{A}_y^{[\ell+s+1:n-r]} \end{bmatrix} \right) : \quad \begin{array}{l} (\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}') \leftarrow \mathcal{O}'_{n,r,\ell,s} \\ (x_0, x_1) \leftarrow \mathcal{A}^{\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}'} \\ (y_b, u_b) \leftarrow \mathcal{P}(x_b) \end{array} \end{array} \right] \geq \frac{\varepsilon}{256n^2}.$$

We now show that given an adversary which succeeds with a bloated dual oracle, one can actually entirely simulate the dual with only access to the primal oracles.

⁶Technically, the lemma states that $\mathcal{A}$ produces something which is stronger than a collision.

Lemma 4.5 (The “Dual is Useless” Lemma). *Suppose that there is an oracle aided q -query algorithm $\mathcal{A}$ such that*

$$\Pr \left[\begin{array}{l} (y_0=y_1 := y) \wedge \\ (\mathbf{u}_0-\mathbf{u}_1) \notin \text{ColSpan} \left(\begin{bmatrix} \mathbf{A}_y^{[1:\ell]} & \mathbf{A}_y^{[\ell+s+1:n-r]} \end{bmatrix} \right) : \begin{array}{l} (\mathcal{P}, \mathcal{P}^{-1}, D') \leftarrow \mathcal{O}'_{n,r,\ell,s} \\ (\mathbf{x}_0, \mathbf{x}_1) \leftarrow \mathcal{A}^{\mathcal{P}, \mathcal{P}^{-1}, D'} \\ (\mathbf{y}_b, \mathbf{u}_b) \leftarrow \mathcal{P}(\mathbf{x}_b) \end{array} \end{array} \right] \geq \varepsilon.$$

Then there is an oracle aided q -query algorithm $\mathcal{B}$ such that

$$\Pr \left[\begin{array}{l} (\bar{\mathbf{y}}_0 = \bar{\mathbf{y}}_1) \wedge (\bar{\mathbf{x}}_0 \neq \bar{\mathbf{x}}_1) : \begin{array}{l} (\bar{\mathcal{P}}, \bar{\mathcal{P}}^{-1}, \bar{D}) \leftarrow \mathcal{O}_{r+s,r,0} \\ (\bar{\mathbf{x}}_0, \bar{\mathbf{x}}_1) \leftarrow \mathcal{B}^{\bar{\mathcal{P}}, \bar{\mathcal{P}}^{-1}} \\ (\bar{\mathbf{y}}_b, \bar{\mathbf{u}}_b) \leftarrow \bar{\mathcal{P}}(\bar{\mathbf{x}}_b) \end{array} \end{array} \right] \geq \varepsilon.$$

The proofs of Lemmas 4.4 and Lemma 4.5 closely follow the proofs of Lemmas 30 and 31 in [SZ25], so we include their proofs for completeness but defer them to Appendix E.

Let us now recall the definition of a coset partition function.

Definition 2 (Coset Partition Functions, [SZ25]). For $n, k \in \mathbb{N}$ such that $k \leq n$ we say a function $Q : \{0, 1\}^n \rightarrow \{0, 1\}^m$ is a (n, m, k) -coset partition function if, for each y in the image of Q , the pre-image set $Q^{-1}(y)$ has size 2^k and is a coset of a linear space of dimension k . We allow different preimage sets to be cosets of different linear spaces.

Having reduced our collision resistance to the dual-free case, we can use the reduction in [SZ25] from the dual-free case to finding collisions in coset-partition functions:

Theorem 4.6 (Adapted from Theorem 34, [SZ25]). *Suppose there is an oracle aided q -query quantum algorithm $\mathcal{A}$ such that*

$$\Pr \left[\begin{array}{l} (\mathcal{P}, \mathcal{P}^{-1}, D) \leftarrow \mathcal{O}_{r+s,r,0} \\ (y_0 = y_1) \wedge (\mathbf{x}_0 \neq \mathbf{x}_1) : \begin{array}{l} (\mathbf{x}_0, \mathbf{x}_1) \leftarrow \mathcal{A}^{\mathcal{P}, \mathcal{P}^{-1}} \\ (\mathbf{y}_b, \mathbf{u}_b) \leftarrow \mathcal{P}(\mathbf{x}_b) \end{array} \end{array} \right] \geq \varepsilon.$$

Then there is an oracle aided q -query quantum algorithm $\mathcal{B}$ that given any $(r+s, r, s)$ -coset partition function Q , satisfies

$$\Pr[(Q(\mathbf{w}_0) = Q(\mathbf{w}_1)) \wedge (\mathbf{w}_0 \neq \mathbf{w}_1) : (\mathbf{w}_0, \mathbf{w}_1) \leftarrow \mathcal{B}^Q] \geq \varepsilon.$$

Combining the three results gives us the following corollary:

Corollary 4.7. *Let $n, r, s, \ell \in \mathbb{N}^+$ such that $r + s + \ell \leq n$. Suppose there is an oracle-aided q -query quantum algorithm $\mathcal{A}$ such that*

$$\Pr \left[\begin{array}{l} (\mathcal{P}, \mathcal{P}^{-1}, D) \leftarrow \mathcal{O}_{n,r,\ell} \\ (y_0 = y_1) \wedge (\mathbf{x}_0 \neq \mathbf{x}_1) : \begin{array}{l} (\mathbf{x}_0, \mathbf{x}_1) \leftarrow \mathcal{A}^{\mathcal{P}, \mathcal{P}^{-1}, D} \\ (\mathbf{y}_b, \mathbf{u}_b) \leftarrow \mathcal{P}(\mathbf{x}_b) \end{array} \end{array} \right] \geq \varepsilon.$$

Define $t := n - r - s$ and suppose $q^3 \cdot n^3 \cdot \frac{1}{\varepsilon^2} \cdot (2^{t-s} + 2^{-t+\ell}) = o(1)$ and $\frac{q^7 \cdot n^{10} \cdot \frac{1}{\varepsilon^4}}{\sqrt{2^{t-\ell}}} = o(1)$. Then there is an oracle aided q -query quantum algorithm $\mathcal{B}$ that given any $(r+s, r, s)$ -coset partition function Q , satisfies

$$\Pr[(Q(\mathbf{w}_0) = Q(\mathbf{w}_1)) \wedge (\mathbf{w}_0 \neq \mathbf{w}_1) : (\mathbf{w}_0, \mathbf{w}_1) \leftarrow \mathcal{B}^Q] \geq \frac{\varepsilon}{256n^2}.$$

Finally, we use the collision-resistance of coset partition functions as shown in [SZ25].

Theorem 4.8 (Adapted from Theorem 35, [SZ25]). *For any $r, s \in \mathbb{N}$ such that $s \mid r$, there exists a distribution over $(r + s, r, s)$ -coset partition functions Q , such that any algorithm making q queries to Q can only find collisions in Q with probability at most $O(\frac{s^3 \cdot q^3}{2^{(r+s)/s}})$.*

Combining all these results allows us to conclude that our one-shot signature scheme is collision-resistant:

Theorem 4.9. *Let $\mathcal{A}$ be an oracle-aided q -query quantum algorithm. Then,*

$$\Pr \left[(\mathbf{x}_0 \neq \mathbf{x}_1) \wedge (H(\mathbf{x}_0) = H(\mathbf{x}_1)) : \begin{array}{l} (\mathcal{P}, \mathcal{P}^{-1}, D) \leftarrow \mathcal{O}_{n,r,\ell} \\ (\mathbf{x}_0, \mathbf{x}_1) \leftarrow \mathcal{A}^{\mathcal{P}, \mathcal{P}^{-1}, D} \end{array} \right] = O \left(\frac{\lambda^3 \cdot q^3 \cdot n^3}{2^\lambda} \right).$$

Proof. Suppose for the sake of contradiction that $\mathcal{A}$ which makes q queries which outputs a collision in H with probability $\varepsilon = \omega \left(\frac{\lambda^3 \cdot q^3 \cdot n^3}{2^\lambda} \right)$. By the choice of our parameters in the OSS Construction (Figure 2), we have that for $t := n - r - s$,

$$\begin{aligned} q^3 \cdot n^3 \cdot \frac{1}{\varepsilon^2} \cdot (2^{t-s} + 2^{-t+\ell}) &= q^3 \cdot n^3 \cdot \frac{1}{\varepsilon^2} \cdot (2^{\ell-s/2} + 2^{-s/2}) \\ &\leq q^3 \cdot n^3 \cdot \frac{2^{2\lambda}}{\lambda^6 \cdot q^6 \cdot n^6} \cdot (2^{\ell-s/2} + 2^{-s/2}) \\ &\leq \frac{1}{\lambda^6 \cdot q^3 \cdot n^3} \cdot (2^{-5\lambda} + 2^{-6\lambda}) = o(1), \end{aligned}$$

and

$$\frac{q^7 \cdot n^{10} \cdot \frac{1}{\varepsilon^4}}{\sqrt{2^{n-r-s-\ell}}} = \frac{q^7 \cdot n^{10} \cdot \frac{1}{\varepsilon^4}}{2^{s/4}} \leq \frac{1}{\lambda^{12} \cdot q^5 \cdot n^2} = o(1).$$

Thus, the conditions of Corollary 4.7 are fulfilled, so there is a q -query algorithm $\mathcal{B}$ that given oracle access to any $(r + s, r, s)$ -coset partition function Q (and thus over any distribution of such functions), finds a collision in Q with probability $\frac{\varepsilon}{256n^2}$. Since $s \mid r$, we can consider the distribution over $(r + s, r, s)$ -coset partition functions generated by Theorem 4.8, which implies that

$$\frac{\varepsilon}{256n^2} = O \left(\frac{s^3 \cdot q^3}{2^{(r+s)/s}} \right) = O \left(\frac{\lambda^3 \cdot q^3}{2^\lambda} \right),$$

and so

$$\varepsilon = O \left(\frac{\lambda^3 \cdot q^3 \cdot n^2}{2^\lambda} \right),$$

contradicting $\varepsilon = \omega \left(\frac{\lambda^3 \cdot q^3 \cdot n^3}{2^\lambda} \right)$. □

We can now see that our scheme is strongly secure:

Corollary 4.10. *Let $\mathcal{A}$ be an oracle aided q -query quantum algorithm. Then,*

$$\Pr \left[\begin{array}{l} (m_0, \sigma_0) \neq (m_1, \sigma_1) \wedge \\ \text{Ver}^{\mathcal{P}^{-1}}(\text{pk}, m_0, \sigma_0) = 1 \wedge \\ \text{Ver}^{\mathcal{P}^{-1}}(\text{pk}, m_1, \sigma_1) = 1 \end{array} : \begin{array}{l} (\mathcal{P}, \mathcal{P}^{-1}, D) \leftarrow \mathcal{O}_{n,r,\ell} \\ (\text{pk}, m_0, \sigma_0, m_1, \sigma_1) \leftarrow \mathcal{A}^{\mathcal{P}, \mathcal{P}^{-1}, D} \end{array} \right] = O \left(\frac{\lambda^3 \cdot q^3 \cdot n^3}{2^\lambda} \right).$$

Proof. Fix any adversary $\mathcal{A}$ that outputs $(\text{pk}, m_0, \sigma_0, m_1, \sigma_1)$ such that $(m_0, \sigma_0) \neq (m_1, \sigma_1)$ and

$$\text{Ver}^{\mathcal{P}^{-1}}(\text{pk}, m_0, \sigma_0) = \text{Ver}^{\mathcal{P}^{-1}}(\text{pk}, m_1, \sigma_1) = 1.$$

If $\sigma_0 = \sigma_1$, then we must have $(\sigma_i)_{[1:\ell]} = m_i$ for $i \in \{0, 1\}$ and thus $m_0 = m_1$, which is a contradiction. Thus, it must be the case that $\sigma_0 \neq \sigma_1$.

This means that $\mathcal{A}$ produces two signatures $\sigma_0 \neq \sigma_1$ and a public key pk such that $\mathcal{P}^{-1}(\text{pk}, \sigma_i) \neq \perp$. This means that there exists a $(q+2)$ -query algorithm $\mathcal{B}$ which produces $x_0 \neq x_1$ such that $H(x_0) = H(x_1)$ which first runs $\mathcal{A}$ and then queries $x_0 \leftarrow \mathcal{P}^{-1}(\text{pk}, \sigma_0)$ and $x_1 \leftarrow \mathcal{P}^{-1}(\text{pk}, \sigma_1)$ with the same success probability as $\mathcal{A}$. As $\mathcal{B}$ succeeds with probability at most $O\left(\frac{\lambda^3 \cdot (q+2)^3 \cdot n^3}{2^\lambda}\right) = O\left(\frac{\lambda^3 \cdot q^3 \cdot n^3}{2^\lambda}\right)$ by Theorem 4.9, $\mathcal{A}$ is only able to produce two valid message-signature pairs with respect to the same verification key with probability $O\left(\frac{\lambda^3 \cdot q^3 \cdot n^3}{2^\lambda}\right)$ as desired. $\square$

5 Public-Key Quantum Fire and Other Applications

To build a public-key quantum fire scheme, we first need a one-shot signature scheme with strong incompressibility security, as defined below:

Definition 3 (Strong Incompressibility Security, [ÇGS25]). Consider a one-shot signature scheme $\text{OSS} = (\text{OSS.Setup}, \text{OSS.Gen}, \text{OSS.Sign}, \text{OSS.Ver})$ relative to a tuple of classical oracles $(\mathcal{O}_{\text{OSS.Gen}}, \mathcal{O}_{\text{OSS.Sign}}, \mathcal{O}_{\text{OSS.Ver}})$ where each algorithm of OSS uses the related oracle only.

OSS is said to satisfy average-case strong (polynomial) incompressibility security if for any query-bounded adversary $\mathcal{A}_1$ with polynomial size classical output, there is a polynomial s such that

$$\Pr \left[E : \begin{array}{l} (\mathcal{O}_{\text{OSS.Gen}}, \mathcal{O}_{\text{OSS.Sign}}, \mathcal{O}_{\text{OSS.Ver}}) \leftarrow \text{OSS.Setup}(1^\lambda) \\ L \leftarrow \mathcal{A}_1^{\mathcal{O}_{\text{OSS.Gen}}, \mathcal{O}_{\text{OSS.Sign}}, \mathcal{O}_{\text{OSS.Ver}}} \end{array} \right] \geq 1 - \text{negl}(\lambda),$$

where E is the event defined as

$$\exists \mathfrak{L}_L \in (\{0, 1\}^{\text{poly}(\lambda)})^{s(\lambda)} : \forall \text{ query-bounded uniform adversaries } \mathcal{A}_2, \\ \Pr \left[\begin{array}{l} \text{pk} \notin \mathfrak{L}_L \wedge \\ \text{OSS.Ver}^{\mathcal{O}_{\text{OSS.Ver}}}(\text{pk}, m, \text{sig}) = 1 \end{array} : (\text{pk}, m, \text{sig}) \leftarrow \mathcal{A}_2^{\mathcal{O}_{\text{OSS.Sign}}, \mathcal{O}_{\text{OSS.Ver}}}(L) \right] = \text{negl}(\lambda).$$

We are now ready to describe the incompressible OSS scheme. It is nearly identical to the construction in Figure 2, but modified so that the coset vector $\mathbf{b}_y$ is never a valid signature w.r.t. the public key $\mathbf{y}$. This will help with the incompressibility proof, where we will prove a stronger statement for adversaries $\mathcal{A}_2$ that have $\mathbf{b}_y$.

Define parameters $s := 16 \cdot (\lambda + 1)$, $r := s \cdot \lambda$, $\ell = \ell(\lambda) := \lambda + 1$, and $n := r + \ell + \frac{3}{2} \cdot s$. Let the random permutation Π and the random function F be as before. For each $\mathbf{y} \in \{0, 1\}^r$, let $\mathbf{B}_y \in \mathbb{Z}_2^{(n-\ell) \times \ell}$ be a random matrix, $\mathbf{C}_y \in \mathbb{Z}_2^{(n-\ell) \times (n-r-\ell)}$ a random matrix with full column-rank. Slightly differently from before, we will choose $\mathbf{b}_y \in \mathbb{Z}_2^n$ to be a uniformly random vector whose ℓ th coordinate is 1. All these are generated by the output randomness of $F(\mathbf{y})$. Define as before

$$\mathbf{A}_y := \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{B}_y & \mathbf{C}_y \end{bmatrix} \in \mathbb{Z}_2^{n \times (n-r)}.$$

Then, let $\mathcal{P}$, $\mathcal{P}^{-1}$, $\mathcal{D}$, and $\mathcal{D}_0$ be the following oracles:

- $\mathcal{P}$, $\mathcal{P}^{-1}$, $\mathcal{D}$ are the same as in Figure 2.

$$\bullet D_0(\mathbf{y}, \mathbf{u}) = \begin{cases} 1 & \exists \mathbf{z} \in \mathbb{Z}_2^{n-r} \text{ such that } \mathbf{A}_y \cdot \mathbf{z} + \mathbf{b}_y = \mathbf{u} \\ 0 & \text{otherwise} \end{cases}$$

We will refer to the distribution of oracles $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}, \mathcal{D}_0)$ by $\tilde{\mathcal{O}}_{n,r,\ell}$. We define our incompressible OSS scheme for signing λ -bit messages in Figure 3.

Incompressible OSS Scheme in the Classical Oracle Model

OSS.Setup (1^λ) : Sample $\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}, \mathcal{D}_0$ as above. Set $\mathcal{O}_{\text{Ver}} = \mathcal{D}_0, \mathcal{O}_{\text{Sign}} = \mathcal{D}, \mathcal{O}_{\text{Gen}} = (\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}, \mathcal{D}_0)$. Output $(\mathcal{O}_{\text{Gen}}, \mathcal{O}_{\text{Sign}}, \mathcal{O}_{\text{Ver}})$.

OSS.Gen $^{\mathcal{O}_{\text{Gen}}}$: Same as in Figure 2.

OSS.Ver $^{\mathcal{O}_{\text{Ver}}}(\text{pk}, m, \sigma)$: Output 1 if $\sigma_{[1:\ell]} = m||0$ and $\mathcal{D}_0(\text{pk}, \sigma) = 1$; else, output 0.

OSS.Sign $^{\mathcal{O}_{\text{Sign}}}(\text{sk}, m)$: Run $\text{OSS.Sig}^{\mathcal{O}_{\text{Sign}}}(\text{sk}, m||0)$, where OSS.Sig is the signing algorithm in Figure 2.

Figure 3: Incompressible OSS scheme in the classical oracle model using oracles $\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}$ and $\mathcal{D}_0$.

Lemma 5.1 (Correctness and Security). *The OSS scheme in Figure 3 has perfect correctness and strong signature security.*

We defer the proof of this lemma to Appendix E since it is nearly identical to the correctness and security proofs of the construction in Figure 2. Before we continue, we introduce a result that we will use in our incompressibility proof.

Definition 4 ([BBBV97]). For an oracle algorithm A making q queries to an oracle $\mathcal{O}$ with input domain D , let $\sum_x \alpha_x^{(i)} |x\rangle$ denote the i th oracle query, and let $M_x(i) = |\alpha_x^{(i)}|^2$ be the query mass of x in the i th query. For a subset $V \subseteq [q] \times D$, let $M_V = \sum_{(i,x) \in V} M_x(i)$ be the total query mass of points in V .

Theorem 5.2 ([BBBV97]). *Let A be an algorithm which makes q queries to an oracle $\mathcal{O}$ with input domain D and let $V \subseteq [q] \times D$ be a set of time-input pairs to $\mathcal{O}$. If we modify $\mathcal{O}$ into an oracle $\mathcal{O}'$ which is identical to $\mathcal{O}$ except possibly on inputs in V , then the final quantum states of A given access to $\mathcal{O}$ or $\mathcal{O}'$ have trace distance at most $O(\sqrt{q \cdot M_V})$.*

Lemma 5.3 (Incompressibility). *The OSS scheme in Figure 3 has strong incompressibility security.*

Proof. The proof follows the general outline of [ÇGS25] closely; some parts may be cited verbatim. We point out the key differences which address the issues with the proof of [ÇGS25].

First, note that in the construction of [SZ25], the signing algorithm makes critical use of all three oracles $\mathcal{P}, \mathcal{P}^{-1}$, and $\mathcal{D}$. But if $\mathcal{A}_2$ has access to both $\mathcal{P}$ and $\mathcal{P}^{-1}$, then even without $\mathbf{b}_y$ or L , it can trivially produce signatures for many keys outside of any polynomial-sized list of keys $\mathcal{L}_L$. This is since it can always run the key generation algorithm and produce key-message-signature tuples (pk, m, σ) such that with high probability, $\text{Ver}(\text{pk}, m, \sigma) = 1$ and $\text{pk} \notin \mathcal{L}_L$. In particular, this means that the [SZ25] OSS construction does *not* have signature incompressibility. On the other hand, our construction only requires $\mathcal{D}$ to sign and $\mathcal{D}_0$ to verify, which means it is plausible that our scheme has signature incompressibility (and we will show that this is indeed the case).

We will actually prove a stronger result where $\mathcal{A}_1$ directly gets the values $\Pi, (\mathbf{A}_y)_y, (\mathbf{b}_y)_y$ and $\mathcal{A}_2$ directly gets $\Pi, (\mathbf{b}_y)_y$, and oracle access to $\mathcal{O}_{\text{OSS.Ver}}$ and $\mathcal{O}_{\text{OSS.Sign}}$. Additionally, instead of the original game, where

we ask $\mathcal{A}_2$ to output a valid key-message-signature tuple, we will ask $\mathcal{A}_2$ to output a pair (y, v) such that $v \in \text{ColSpan}(\mathbf{A}_y) \setminus \{0\}$. We explain why this is at least as easy for $\mathcal{A}_2$ here as in the original security game.

Observe that in order for sig to be a valid signature for m with respect to some key $\text{pk} = y$, it must be the case that $\text{sig} - \mathbf{b}_y \in \text{ColSpan}(\mathbf{A}_y)$ and that $\text{sig}_\ell = 0$. However, since the ℓ 'th coordinate of $\mathbf{b}_y$ is always nonzero (by construction), $\text{sig} - \mathbf{b}_y$ is also always nonzero. Thus, given $\mathbf{b}_y$, any signature sig automatically gives rise to a nonzero vector $\text{sig} - \mathbf{b}_y$ in the column span of $\mathbf{A}_y$. This is the second difference between our construction and the [SZ25] scheme; in the [SZ25] construction, $(y, \mathbf{b}_y)$ (or more accurately, y and the preimage of $\mathbf{b}_y$) is always a valid signature for some message, so $\mathcal{A}_2$ can once again trivially produce valid key-message-signature tuples (this time for *any* key) given $\mathbf{b}_y$. On the other hand, in our construction, $\mathbf{b}_y$ is never a valid signature for any message since it is (by design) nonzero on the ℓ 'th coordinate.

We now observe that for each y , there are

$$C_1 = 2^{(n-\ell)\ell} \cdot \prod_{i=0}^{n-r-\ell-1} (2^{n-\ell} - 2^i)$$

equally likely matrices for $\mathbf{A}_y$.

Now fix any vector $v \in \mathbb{Z}_2^n \setminus \{0\}$. We first consider how many matrices of the form $\mathbf{A} = \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{B} & \mathbf{C} \end{bmatrix} \in \mathbb{Z}_2^{n \times (n-r)}$, where $\mathbf{C}$ is a full-rank matrix, contain v in their column span. We can equivalently consider the probability that a random matrix of this form contains v in its column span and derive the total number of matrices accordingly.

Suppose $v_{[1:\ell]} = 0^\ell$; in this case, whether a matrix contains v in its column span or not is independent of $\mathbf{B}$. Thus, this probability is simply the probability that a random full-rank matrix in $\mathbb{Z}_2^{(n-\ell) \times (n-r-\ell)}$ contain a nonzero vector $w \in \mathbb{Z}_2^{n-\ell}$ in its column span, which is simply $\frac{2^{n-r-\ell}-1}{2^{n-\ell}-1}$ and so there are

$$C_2 = \frac{2^{n-r-\ell}-1}{2^{n-\ell}-1} \cdot C_1 = 2^{(n-\ell)\ell} \cdot (2^{n-r-\ell}-1) \cdot \prod_{i=1}^{n-r-\ell-1} (2^{n-\ell} - 2^i)$$

such matrices.

Now suppose instead that $v_{[1:\ell]} \neq 0^\ell$; in this case, $v \in \text{ColSpan}(\mathbf{A})$ if and only if $\mathbf{B}v_{[1:\ell]} + v_{[\ell+1:n]} \in \text{ColSpan}(\mathbf{C})$. Since $v_{[1:\ell]}$ is nonzero, for a random matrix $\mathbf{B}$, $\mathbf{B}v_{[1:\ell]} + v_{[\ell+1:n]}$ is a uniformly random vector in $\mathbb{Z}_2^{n-\ell}$. Thus, the probability that $v \in \text{ColSpan}(\mathbf{A})$ is

$$\frac{1}{2^{n-\ell}} \cdot 1 + \frac{2^{n-\ell}-1}{2^{n-\ell}} \cdot \frac{2^{n-r-\ell}-1}{2^{n-\ell}-1} = \frac{2^{n-r-\ell}}{2^{n-\ell}},$$

and so there are

$$C_3 = \frac{2^{n-r-\ell}}{2^{n-\ell}} \cdot C_1 = 2^{(n-\ell)(\ell-1)} \cdot 2^{n-r-\ell} \cdot \prod_{i=0}^{n-r-\ell-1} (2^{n-\ell} - 2^i) > C_2$$

such matrices.

The second thing we consider is how many matrices of the form $\mathbf{A} = \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{B} & \mathbf{C} \end{bmatrix} \in \mathbb{Z}_2^{n \times (n-r)}$, where $\mathbf{C}$ is a full-rank matrix, have the property that $v^T \cdot \mathbf{A}^{[j:n-r]} = 0^{n-r-j+1}$ for some $j \in [\ell+1]$. It is clear that if this condition holds for any j , then it must be the case that $v_{[\ell+1:n]}^T \cdot \mathbf{C} = 0^{n-r-\ell}$. In addition, for any j , if $v_{[\ell+1:n]}^T \cdot \mathbf{A}^{[j:n-r]} = 0^{n-r-j+1}$, then either $v_{[j:n]} = 0^{n-j+1}$, in which case it is always the case that $v^T \cdot \mathbf{A}^{[j:n-r]} = 0^{n-r-j+1}$, or $v_{[\ell+1:n]} \neq 0^{n-\ell}$. We will ignore (for now) the case where $v_{[j:n]} = 0^{n-j+1}$ since this reveals no information about $\mathbf{A}$. We upper bound the fraction of matrices $\mathbf{C}$ such that $w^T \cdot \mathbf{C} = 0^{n-r-\ell}$ for some nonzero $w \in \mathbb{Z}_2^{n-\ell}$, and thus the fraction of matrices $\mathbf{A}$ such that $v^T \cdot \mathbf{A}^{[j:n-r]} = 0^{n-r-j+1}$ assuming $v_{[j:n]} \neq 0^{n-j+1}$.

Clearly the dual of C has dimension $(n - \ell) - (n - r - \ell) = r$; thus, the fraction of matrices C such that $\mathbf{w}^T \cdot C = 0^{n-r-\ell}$ is at most

$$\frac{2^r - 1}{2^{n-\ell} - 1} \leq \frac{2^r}{2^{n-\ell}} = 2^{-(n-\ell)+r}.$$

The number of matrices A is thus at most

$$C_4 = 2^{-(n-\ell)+r} \cdot C_1 = 2^{-3s/2} \cdot C_1 > C_3.$$

For any string L^* from the output support of $\mathcal{A}_1$, let S_{L^*} denote the set consisting of all vectors $\mathbf{y} \in \{0, 1\}^r$ such that $H_\infty(\mathbf{A}_\mathbf{y} \mid L = L^*) \leq \log C_4 + \lambda$. We will argue that $\mathcal{A}_2$ cannot succeed with non-negligible probability even when given $\{\mathbf{A}_\mathbf{y}\}_{\mathbf{y} \in S_{L^*}}$.

We begin by considering the special case where $\mathcal{A}_2$ does not make any oracle queries. We claim that the probability that $\mathcal{A}_2(L^*)$ can output either a) $(\mathbf{y}, \mathbf{v})$ such that $\mathbf{v} \in \text{ColSpan}(\mathbf{A}_\mathbf{y}) \setminus \{0\}$ or b) $(j, \mathbf{y}, \mathbf{v})$ such that $\mathbf{v}^T \cdot \mathbf{A}_\mathbf{y}^{[j:n-r]} = 0^{n-r-j+1}$ and $\mathbf{v}_{[j:n]} \neq 0^{n-j+1}$ for some $j \in [\ell + 1]$ where $\mathbf{y} \notin S_{L^*}$ is negligible.

Suppose this is not the case; then there is some non-negligible ε such that either $\mathcal{A}_2$ can output $(\mathbf{y}, \mathbf{v})$ with probability ε or it can output $(j, \mathbf{y}, \mathbf{v})$ with probability ε . Then, whenever $\mathcal{A}_2$ outputs a valid tuple $(\mathbf{y}, \mathbf{v})$ or $(j, \mathbf{y}, \mathbf{v})$ for $\mathbf{y} \notin S_{L^*}$, there are at most $\max\{C_2, C_3, C_4\} = C_4$ options for $\mathbf{A}_\mathbf{y}$ and so $\mathcal{A}_2$ can guess $\mathbf{A}_\mathbf{y}$ with probability at least $\frac{\varepsilon}{C_4}$. However, this contradicts the fact that $H_\infty(\mathbf{A}_\mathbf{y} \mid L = L^*) \leq \log C_4 + \lambda$; thus, it must be the case that $\mathcal{A}_2$ succeeds only with negligible probability.

We now bound the size of S_{L^*} in this special case. Observe that

$$\begin{aligned} & \Pr[H_\infty((\mathbf{A}_\mathbf{y})_\mathbf{y} \mid L = L^*) \geq 2^r \cdot \log C_1 - |L| - \lambda] \\ &= \Pr\left[H_\infty((\mathbf{A}_\mathbf{y})_\mathbf{y} \mid L = L^*) \geq \sum_{\mathbf{y}} H_\infty(\mathbf{A}_\mathbf{y}) - |L| - \lambda\right] \\ &\geq \Pr[H_\infty((\mathbf{A}_\mathbf{y})_\mathbf{y} \mid L = L^*) \geq H_\infty((\mathbf{A}_\mathbf{y})_\mathbf{y} | L) - \lambda] \geq 1 - 2^{-\lambda}. \end{aligned}$$

On the other hand, for any string L^* , we have

$$\begin{aligned} H_\infty((\mathbf{A}_\mathbf{y})_\mathbf{y} \mid L = L^*) &= \sum_{\mathbf{y}} H_\infty(\mathbf{A}_\mathbf{y} \mid L = L^*) \\ &= \sum_{\mathbf{y} \in S_{L^*}} H_\infty(\mathbf{A}_\mathbf{y} \mid L = L^*) + \sum_{\mathbf{y} \notin S_{L^*}} H_\infty(\mathbf{A}_\mathbf{y} \mid L = L^*) \\ &\leq |S_{L^*}| \cdot (\log C_4 + \lambda) + (2^r - |S_{L^*}|) \cdot \log C_1. \end{aligned}$$

Thus, with probability $1 - 2^{-\lambda}$,

$$\begin{aligned} & |S_{L^*}| \cdot (\log C_4 + \lambda) + (2^r - |S_{L^*}|) \cdot \log C_1 \geq 2^r \cdot \log C_1 - |L| - \lambda \\ \implies & |L| + \lambda \geq |S_{L^*}| \cdot \left(\log \frac{C_1}{C_4} - \lambda\right) \geq |S_{L^*}| \cdot (3s/2 - \lambda) \geq |S_{L^*}|, \end{aligned}$$

and so $\Pr[|S_{L^*}| \leq |L| + \lambda] \geq 1 - \text{negl}(\lambda)$ for all λ , which completes the proof for the special case.

We now move to the general setting where $\mathcal{A}_2(L^*)$ can make any polynomial $q(\lambda)$ number of queries. Let $\frac{1}{p(\lambda)}$ be the success probability of $\mathcal{A}_2$. We show how to remove the first query of $\mathcal{A}_2$ before iterating this process to remove all queries of $\mathcal{A}_2$. Here, we clarify an ambiguity in the proof of [CGS25], which does not concretely address queries to D ; this modification is not too hard with additional calculations, but requires some care.

Suppose $\mathcal{A}_2$'s first query is to D_0 . For just the first query, consider running $\mathcal{A}_2$ with the oracle D'_0 which is a modification of D_0 as follows: on all inputs $(\mathbf{y}, \mathbf{v})$ where $\mathbf{y} \notin S_{L^*}$, output 0 if $\mathbf{v} \neq \mathbf{b}_\mathbf{y}$ and 1 otherwise; on

inputs (y, v) where $y \in S_{L^*}$, behave identically to D_0 . Clearly, it is the case that $D_0(y, b_y) = D'_0(y, b_y) = 1$, and so the two oracles differ precisely on input pairs (y, v) such that $y \notin S_{L^*}$, $v \neq b_y$, and $D_0(y, v) = 1$. The total query mass on inputs on which D_0 and D'_0 differ is at most

$$\sum_{\substack{y \notin S_{L^*} \\ v : v \neq b_y \wedge D_0(v, y) = 1}} |\alpha_{y, v}|^2.$$

We claim that this query mass is negligible. Suppose for the sake of contradiction that this query mass was non-negligible; consider the adversary $\mathcal{B}$ which runs $\mathcal{A}_2$ up until it makes its first query, measures the query that $\mathcal{A}_2$ would have made to D_0 and gets the measurement outcome (y, v) before returning $(y, v - b_y)$. By assumption, with non-negligible probability, $\mathcal{B}$ outputs a pair $(y, v - b_y)$ such that $y \notin S_{L^*}$, $v - b_y \neq 0$, but $D_0(y, v) = 1$ and so $v - b_y \in \text{ColSpan}(\mathbf{A}_y) \setminus \{0\}$. But this violates the security we proved for the special case since $\mathcal{B}$ makes no queries! Thus, it must be the case that

$$\sum_{\substack{y \notin S_{L^*} \\ v : v \neq b_y \wedge D_0(v, y) = 1}} |\alpha_{y, v}|^2 = \text{negl}(\lambda).$$

By Theorem 5.2, $\mathcal{A}_2$ still succeeds with the reprogrammed oracle D'_0 for its first query (and D_0 or D for the other queries) with probability

$$\frac{1}{p(\lambda)} - \sqrt{q \cdot \text{negl}(\lambda)} \geq \frac{1}{p(\lambda)} - \text{negl}(\lambda) \geq \frac{1}{p(\lambda)} - \frac{1}{2p(\lambda)q(\lambda)}.$$

But now, the query that $\mathcal{A}_2$ makes to D'_0 is completely unnecessary: given b_y for all y and $\mathbf{A}_y$ for $y \in S_{L^*}$, $\mathcal{A}_2$ can *itself* simulate access to D'_0 ! We can therefore remove/simulate $\mathcal{A}_2$'s first query altogether and end up with an adversary $\mathcal{A}'_2$ which makes $q - 1$ queries and still succeeds with probability at least $\frac{1}{p(\lambda)} - \frac{1}{2p(\lambda)q(\lambda)}$.

We now consider the case where $\mathcal{A}_2$'s first query is to D . For just the first query, consider running $\mathcal{A}_2$ with the oracle D' which is a modification of D as follows: on inputs (j, y, v) where $y \notin S_{L^*}$, output 0 if $v_{[j:n]} \neq 0^{n-j+1}$ and 1 otherwise; if $y \in S_{L^*}$, then behave identically to D . As argued earlier, if $v_{[j:n]} = 0^{n-j+1}$, then we always have that $v^T \cdot \mathbf{A}_y^{[j:n-r]} = 0^{n-r-j+1}$ and thus $D(j, y, v) = D'(j, y, v) = 1$. We conclude that D and D' differ precisely on inputs (j, y, v) such that $y \notin S_{L^*}$, $v_{[j:n]} \neq 0^{n-j+1}$, and $D(j, y, v) = 1$. The total query mass on inputs on which D and D' differ is thus at most

$$\sum_{\substack{y \notin S_{L^*} \\ i, v : v_{[j:n]} \neq 0^{n-j+1} \\ D(j, v, y) = 1}} |\alpha_{y, v}|^2.$$

As with the previous case, we claim that this query mass is negligible. Suppose otherwise; consider the adversary $\mathcal{B}$ which runs $\mathcal{A}_2$ up until it makes its first query, measures the query that $\mathcal{A}_2$ would have made to D and gets the measurement outcome (j, y, v) , which it returns. By assumption, with non-negligible probability, $\mathcal{B}$ outputs (j, y, v) such that $y \notin S_{L^*}$, $v_{[j:n]} \neq 0^{n-j+1}$, but $D(j, y, v) = 1$ and so $v^T \cdot \mathbf{A}_y^{[j:n-r]} = 0^{n-r-j+1}$. But this violates the security we proved for the special case since $\mathcal{B}$ makes no queries. Thus, it must be the case that

$$\sum_{\substack{y \notin S_{L^*} \\ j, v : v_{[j:n]} \neq 0^{n-j+1} \\ D(j, v, y) = 1}} |\alpha_{y, v}|^2 = \text{negl}(\lambda).$$

By Theorem 5.2, $\mathcal{A}_2$ still succeeds with the reprogrammed oracle D' for its first query (and D or D_0 for the other queries) with probability

$$\frac{1}{p(\lambda)} - \sqrt{q \cdot \text{negl}(\lambda)} \geq \frac{1}{p(\lambda)} - \text{negl}(\lambda) \geq \frac{1}{p(\lambda)} - \frac{1}{2p(\lambda)q(\lambda)}.$$

But now, the query that $\mathcal{A}_2$ makes to D' is completely unnecessary: given $\mathbf{b}_y$ for all y and $\mathbf{A}_y$ for $y \in S_{L^*}$, $\mathcal{A}_2$ can once again simulate access to D' . We can therefore remove/simulate $\mathcal{A}_2$'s first query altogether and end up with an adversary $\mathcal{A}'_2$ which makes $q - 1$ queries and still succeeds with probability at least $\frac{1}{p(\lambda)} - \frac{1}{2p(\lambda)q(\lambda)}$.

Iterating this procedure for each query that $\mathcal{A}_2$ makes, we end up with an adversary $\mathcal{A}_2^*$ which makes no queries and still succeeds with probability at least

$$\frac{1}{p(\lambda)} - q(\lambda) \cdot \frac{1}{2p(\lambda)q(\lambda)} \geq \frac{1}{2p(\lambda)},$$

which is non-negligible. But this violates the security of the special case, and so $\mathcal{A}_2$ cannot succeed with non-negligible probability even when given $\{\mathbf{A}_y\}_{y \in S_{L^*}}$. We conclude that without $\{\mathbf{A}_y\}_{y \in S_{L^*}}$, $\{\mathbf{b}_y\}_y$, and Π , and given only L^* , no (polynomially) query-bounded algorithm $\mathcal{A}_2$ can succeed with non-negligible probability in producing a valid signature for a verification key outside of S_{L^*} . Since we argued earlier that $|S_{L^*}| \leq |L| + \lambda$ with all but negligible probability, the proof is complete. $\square$

With this, we can appeal to the result of [ÇGS25]:

Theorem 5.4 (Theorems 10 and 11, [ÇGS25]). *Suppose there exists a one-shot signature scheme for λ -bit messages with perfect correctness, (weak) signature security, and strong incompressibility security. Then there exists a quantum key-fire scheme for signing with perfect signing correctness, perfect strong cloning correctness, and key-untelegraphability security. Moreover, assuming the existence of post-quantum one-way functions, the key-fire scheme can be made efficient (with computational security) if the one-shot signature scheme is efficient.*

Corollary 5.5. *Relative to a classical oracle, there exists a public-key quantum key-fire scheme for signing with perfect signing correctness, perfect strong cloning correctness, and key-untelegraphability security. Moreover, assuming the existence of post-quantum one-way functions, the key-fire scheme can be made efficient (with computational security).*

Proof. The corollary follows from combining Lemmas 5.1 and 5.3 with Theorem 5.4. The efficient version follows from observing that one can efficiently simulate the OSS scheme by replacing the permutation Π and random function F with a post-quantum PRP and PRF, which exist assuming post-quantum one-way functions exist [Zha21a, Zha25b]. $\square$

Corollary 5.6 (Combining Corollary 5.5 and [ÇGS25]). *Relative to a classical oracle, there exists:*

- *a public-key quantum fire scheme with perfect verification correctness, perfect strong cloning correctness, and untelegraphability security,*
- *a public-key quantum key-fire scheme with untelegraphability security for any classically-unlearnable functionality with perfect cloning correctness, perfect evaluation correctness, and key-untelegraphability security,*
- *a public-key encryption scheme with perfect correctness and perfectly clonable yet unbounded-leakage-resilient secret keys (and classical public keys and ciphertexts).*

Moreover, assuming the existence of post-quantum one-way functions, all of these schemes can be made efficient (with computational security).

References

- [AGKZ20] Ryan Amos, Marios Georgiou, Aggelos Kiayias, and Mark Zhandry. One-shot signatures and applications to hybrid quantum/classical authentication. In *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing*, pages 255–268, 2020. 1, 3, 4, 5, 8, 9, 10, 12
- [AGQY22] Prabhanjan Ananth, Aditya Gulati, Luowen Qian, and Henry Yuen. Pseudorandom (function-like) quantum state generators: New definitions and applications. In *Theory of Cryptography Conference*, pages 237–265. Springer, 2022. 44
- [ARU14] Andris Ambainis, Ansis Rosmanis, and Dominique Unruh. Quantum attacks on classical proof systems: The hardness of quantum rewinding. In *55th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2014, Philadelphia, PA, USA, October 18-21, 2014*, pages 474–483. IEEE Computer Society, 2014. 1, 3
- [BBBV97] Charles H Bennett, Ethan Bernstein, Gilles Brassard, and Umesh Vazirani. Strengths and weaknesses of quantum computing. *SIAM journal on Computing*, 26(5):1510–1523, 1997. 17
- [BCP14] Elette Boyle, Kai-Min Chung, and Rafael Pass. On extractability obfuscation. In *Theory of cryptography conference*, pages 52–73. Springer, 2014. 41
- [BHMT00] Gilles Brassard, Peter Hoyer, Michele Mosca, and Alain Tapp. Quantum amplitude amplification and estimation. *arXiv preprint quant-ph/0005055*, 2000. 12
- [BNZ25] John Bostanci, Barak Nehoran, and Mark Zhandry. A general quantum duality for representations of groups with applications to quantum money, lightning, and fire. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing*, pages 201–212, 2025. 2, 6
- [Car25] Joseph Carolan. Compressed permutation oracles. *arXiv preprint arXiv:2509.18586*, 2025. 8
- [ÇGS25] Alper Çakan, Vipul Goyal, and Omri Shmueli. Public-key quantum fire and key-fire from classical oracles. *arXiv preprint arXiv:2504.16407*, 2025. Contributed talk at QCrypt 2025. 2, 6, 7, 16, 17, 19, 21
- [CLTV15] Ran Canetti, Huijia Lin, Stefano Tessaro, and Vinod Vaikuntanathan. Obfuscation of probabilistic circuits and applications. In Yevgeniy Dodis and Jesper Buus Nielsen, editors, *Theory of Cryptography - 12th Theory of Cryptography Conference, TCC 2015, Warsaw, Poland, March 23-25, 2015, Proceedings, Part II*, volume 9015 of *Lecture Notes in Computer Science*, pages 468–497. Springer, 2015. 24, 25
- [DS22] Marcel Dall’Agnol and Nicholas Spooner. On the necessity of collapsing. *IACR Cryptol. ePrint Arch.*, 2022:786, 2022. i, 1, 3, 4, 5, 7, 9
- [GMR88] Shafi Goldwasser, Silvio Micali, and Ronald L. Rivest. A digital signature scheme secure against adaptive chosen-message attacks. *SIAM J. Comput.*, 17(2):281–308, 1988. 1
- [MMW25] Christian Majenz, Giulio Malavolta, and Michael Walter. Permutation superposition oracles for quantum query lower bounds. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing*, pages 1508–1519, 2025. 8
- [MW04] Chris Marriott and John Watrous. Quantum arthur-merlin games. In *19th Annual IEEE Conference on Computational Complexity (CCC 2004), 21-24 June 2004, Amherst, MA, USA*, pages 275–285. IEEE Computer Society, 2004. 1

- [SZ25] Omri Shmueli and Mark Zhandry. On one-shot signatures, quantum vs classical binding, and obfuscating permutations. *Cryptology ePrint Archive*, 2025. [i](#), [1](#), [2](#), [3](#), [4](#), [5](#), [6](#), [7](#), [8](#), [9](#), [13](#), [14](#), [15](#), [17](#), [18](#), [25](#), [26](#), [27](#), [28](#), [29](#), [30](#), [37](#), [38](#), [42](#), [49](#), [56](#), [57](#)
- [Unr16] Dominique Unruh. Computationally binding quantum commitments. In Marc Fischlin and Jean-Sébastien Coron, editors, *Advances in Cryptology - EUROCRYPT 2016 - 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8-12, 2016, Proceedings, Part II*, volume 9666 of *Lecture Notes in Computer Science*, pages 497–527. Springer, 2016. [1](#), [3](#)
- [Zha15] Mark Zhandry. A note on the quantum collision and set equality problems. *Quantum Information & Computation*, 15(7-8):557–567, 2015. [31](#)
- [Zha21a] Mark Zhandry. How to construct quantum random functions. *Journal of the ACM (JACM)*, 68(5):1–43, 2021. [21](#)
- [Zha21b] Mark Zhandry. Quantum lightning never strikes the same state twice. or: quantum money from cryptographic assumptions. *Journal of Cryptology*, 34(1):6, 2021. [41](#)
- [Zha25a] Mark Zhandry. How to model unitary oracles. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part II*, volume 16001 of *Lecture Notes in Computer Science*, pages 237–268. Springer, 2025. [11](#)
- [Zha25b] Mark Zhandry. A note on quantum-secure prps. *Quantum*, 9:1696, 2025. [21](#)

A Cryptographic Definitions and Lemmas

Definition 5 (Puncturable PRFs). A puncturable pseudorandom function (P-PRF) is a triple of efficient algorithms $(F, \text{Punc}, \text{Eval})$ with an associated output-length function $m(\lambda)$ with the following interface:

- $F : \{0, 1\}^\lambda \times \{0, 1\}^* \rightarrow \{0, 1\}^{m(\lambda)}$ is a deterministic polynomial-time algorithm.
- $\text{Punc}(k, S)$ is a PPT algorithm which takes as input a key $k \in \{0, 1\}^\lambda$ and a set of points $S \subseteq \{0, 1\}^*$. It outputs a punctured key $k\{S\}$.
- $\text{Eval}(k\{S\}, x)$ is a deterministic polynomial-time algorithm.

The algorithms satisfy the following correctness and security properties:

- **Correctness:** For any $\lambda \in \mathbb{N}$, $S \subseteq \{0, 1\}^*$, $k \in \{0, 1\}^\lambda$, $x \notin S$, and $k\{S\}$ in the support of $\text{Punc}(k, S)$, we have that

$$\text{Eval}(k\{S\}, x) = F(k, x).$$

- **(Post-Quantum) Security:** A puncturable PRF is (f, ϵ) -secure for some functions $f : \mathbb{N} \rightarrow \mathbb{N}$ and $\epsilon : \mathbb{N} \rightarrow [0, 1]$ if for every $f(\lambda)$ -time quantum algorithm $\mathcal{A}$, the following experiment with $\mathcal{A}$ outputs 1 with probability at most $\frac{1}{2} + \epsilon(\lambda)$:
 - $\mathcal{A}(1^\lambda)$ produces a set $S \subseteq \{0, 1\}^*$.
 - The experiment chooses a random $k \leftarrow \{0, 1\}^\lambda$ and computes $k\{S\} \leftarrow \text{Punc}(k, S)$. For each $x \in S$, it also sets $y_x^0 := F(k, x)$ and samples $y_x^1 \leftarrow \{0, 1\}^{m(\lambda)}$ uniformly at random. Then, it chooses a random bit b . It finally gives $k\{S\}$ and $\{(x, y_x^b)\}_{x \in S}$ to $\mathcal{A}$.
 - $\mathcal{A}$ outputs a guess b' for b . The experiment outputs 1 if and only if $b' = b$.

Concretely, a sub-exponentially secure P-PRF scheme would be one for which there exists a positive real constant $c > 0$ such that the scheme is $(2^{\lambda^c}, \frac{1}{2^{\lambda^c}})$ -secure.

Definition 6 (Indistinguishability Obfuscation (iO)). An indistinguishability obfuscator for Boolean circuits is a probabilistic polynomial-time algorithm $\text{iO}(\cdot, \cdot, \cdot)$ with the following properties:

- **Correctness:** For all $\lambda, s \in \mathbb{N}$, Boolean circuits C of size at most s , and all inputs x ,

$$\Pr[\text{O}_C(x) = C(x) : \text{O}_C \leftarrow \text{iO}(1^\lambda, 1^s, C)] = 1.$$

- **(Post-Quantum) Security:** An iO scheme is (f, ϵ) -secure for some functions $f : \mathbb{N} \rightarrow \mathbb{N}$ and $\epsilon : \mathbb{N} \rightarrow [0, 1]$ if for every $f(\lambda)$ -time quantum algorithm $\mathcal{A}$, and every two classical circuits C_0, C_1 with the same functionality and size at most s ,

$$\Pr \left[\mathcal{A}(1^\lambda, \text{O}_b) = b \quad : \quad \begin{array}{l} b \leftarrow \{0, 1\} \\ \text{O}_b \leftarrow \text{iO}(1^\lambda, 1^s, C_b) \end{array} \right] \leq \frac{1}{2} + \epsilon(\lambda)$$

Concretely, a sub-exponentially secure iO scheme would be one for which there exists a positive real constant $c > 0$ such that the scheme is $(2^{\lambda^c}, \frac{1}{2^{\lambda^c}})$ -secure. When the upper bound on the size of the circuits is clear from the context, we will omit the size parameter 1^s from the input to the obfuscator.

We will use the following lemma from [CLTV15]:

Lemma A.1 (Adapted from Theorem 2, [CLTV15]). Let $\mathcal{X}$ be a finite set and let $\mathcal{D}_0$ and $\mathcal{D}_1$ be two distribution samplers that take as input an index x and some random coins r , and output a sample. Suppose that given the samplers $\mathcal{D}_0$ and $\mathcal{D}_1$, for every $x \in \mathcal{X}$, the distributions $\mathcal{D}_0(x; \cdot)$ and $\mathcal{D}_1(x; \cdot)$ are (f_D, δ_D) -indistinguishable.

Let $(F, \text{Punc}, \text{Eval})$ and $(F', \text{Punc}', \text{Eval}')$ be (f_F, δ_F) -secure puncturable PRFs and let iO be a $(f_{\text{iO}}, \delta_{\text{iO}})$ -secure iO scheme. Let

$$E_{0,k_0}(x) := \mathcal{D}_0(x; F(k_0, x)) \quad \text{and} \quad E_{1,k_1}(x) := \mathcal{D}_1(x; F'(k_1, x)),$$

and let

$$\tilde{E}_0 \leftarrow \text{iO}(1^\lambda, 1^s, E_{0,k_0}) \quad \text{and} \quad \tilde{E}_1 \leftarrow \text{iO}(1^\lambda, 1^s, E_{1,k_1})$$

be the obfuscated programs for a sufficiently large polynomial s and uniformly random keys k_0 and k_1 . Then, $\tilde{E}_0$ and $\tilde{E}_1$ are (f, δ) -indistinguishable, where

$$f = \min(f_F, f_{\text{iO}}, f_D) \quad \text{and} \quad \epsilon = O(|\mathcal{X}| \cdot (\delta_F + \delta_{\text{iO}} + \delta_D)).$$

Strictly speaking, [CLTV15, SZ25] only shows such a lemma for one P-PRF and two different keys, but the proof easily extends to the setting of two P-PRFs which use separate keys.

Definition 7 (Lossy Functions). A lossy function (LF) family consists of classical algorithms $(\text{LF.KeyGen}, \text{LF.F})$ with the following interface:

- $\text{pk} \leftarrow \text{LF.KeyGen}(1^\lambda, b, 1^\ell)$: a PPT algorithm that takes as input the security parameter $\lambda \in \mathbb{N}$, a bit b and a lossiness parameter $\ell \in \mathbb{N}$ with $\ell \leq \lambda$. The algorithm outputs a public key pk .
- $y \leftarrow \text{LF.F}(1^\lambda, \text{pk}, x)$: a deterministic polynomial-time algorithm that takes as input the security parameter $\lambda \in \mathbb{N}$, the public key pk and an input $x \in \{0, 1\}^\lambda$ and outputs a string $y \in \{0, 1\}^m$ for some $m \geq \lambda$.

The algorithms satisfy the following guarantees:

- **Statistical Correctness for Injective Mode:** There exists a negligible function negl such that for every $\lambda, \ell \in \mathbb{N}$,

$$\Pr \left[|\text{Img}(\text{LF.F}(\text{pk}, \cdot))| = 2^\lambda \quad : \quad \text{pk} \leftarrow \text{LF.KeyGen}(1^\lambda, 0, 1^\ell) \right] \geq 1 - \text{negl}(\lambda).$$

- **Statistical Correctness for Lossy Mode:** There exists a negligible function negl such that for every $\lambda, \ell \in \mathbb{N}$,

$$\Pr \left[|\text{Img}(\text{LF.F}(\text{pk}, \cdot))| \leq 2^\ell \quad : \quad \text{pk} \leftarrow \text{LF.KeyGen}(1^\lambda, 1, 1^\ell) \right] \geq 1 - \text{negl}(\lambda).$$

- **(Post-Quantum) Security:** An LF family is (f, ϵ) -secure for some functions $f : \mathbb{N} \rightarrow \mathbb{N}$ and $\epsilon : \mathbb{N} \rightarrow [0, 1]$ if for every $f(\lambda)$ -time quantum algorithm $\mathcal{A}$,

$$\Pr \left[\mathcal{A}(1^\lambda, \text{pk}_b) = b \quad : \quad \text{pk}_b \leftarrow \text{LF.KeyGen}(1^\lambda, b, 1^\ell) \right] \leq \frac{1}{2} + \epsilon(\lambda)$$

Concretely, a sub-exponentially secure LF scheme would be for which there exists a positive real constant $c > 0$ such that the scheme is $(2^{\lambda^c}, \frac{1}{2^{\lambda^c}})$ -secure.

Definition 8 (Permutable PRPs, [SZ25]). Let $G = \{G_N\}_{N \in \mathbb{N}}$ be a collection where each G_N is a set of permutations over $[N]$. An output-permutable PRP (OP-PRP) for G is a tuple of algorithms $(\Pi, \Pi^{-1}, \text{Permute}, \text{Eval}, \text{Eval}^{-1})$ with the following interface:

- **Efficient Permutations:** For any key $k \in \{0, 1\}^\lambda$ and any desired “block size” N , $\Pi(k, \cdot)$ is an efficiently computable permutation on $[N]$ with $\Pi^{-1}(k, \cdot)$ being its efficiently computable inverse. The same is true of Eval and Eval^{-1} .
- **Output Permuted:** $\text{Permute}(k, \Gamma, c)$ is a deterministic polynomial-time procedure which takes as input a key $k \in \{0, 1\}^\lambda$, the circuit description of a permutation $\Gamma \in G_N$, and a bit c . It outputs a permuted key $k^{\Gamma, c}$.

The algorithms satisfy the following guarantees:

- **Output Permuted Correctness:** For all $\lambda \in \mathbb{N}$, $k \in \{0, 1\}^\lambda$, $c \in \{0, 1\}$, $\Gamma \in G_N$, and $x, z \in [N]$,

$$\begin{aligned} \text{Eval}(k^{\Gamma, c}, x) &= \begin{cases} \Pi(k, x) & \text{if } c = 0 \\ \Gamma(\Pi(k, x)) & \text{if } c = 1 \end{cases} \\ \text{Eval}^{-1}(k^{\Gamma, c}, z) &= \begin{cases} \Pi^{-1}(k, z) & \text{if } c = 0 \\ \Pi^{-1}(k, \Gamma^{-1}(z)) & \text{if } c = 1 \end{cases} \end{aligned}$$

with probability 1. We call $k^{\Gamma, c} \leftarrow \text{Permute}(k, \Gamma, c)$ a permuted key.

- **(Post-Quantum) Security:** An OP-PRP family is (f, ϵ) -secure for some functions $f : \mathbb{N} \rightarrow \mathbb{N}$ and $\epsilon : \mathbb{N} \rightarrow [0, 1]$ if every $f(\lambda)$ -time quantum interactive algorithm $\mathcal{A}$ outputs 1 with probability at most $\frac{1}{2} + \epsilon(\lambda)$ in the following experiment:
 - $\mathcal{A}(1^\lambda)$ produces a block size N (in binary) and the description of a permutation $\Gamma \in G_N$.
 - The experiment chooses a random $k \leftarrow \{0, 1\}^\lambda$ and a random bit $c \in \{0, 1\}$. It gives $k^{\Gamma, c} \leftarrow \text{Permute}(k, \Gamma, c)$ to $\mathcal{A}$.
 - $\mathcal{A}$ produces a guess c' for c . The experiment outputs 1 if $c' = c$.

Concretely, a sub-exponentially secure OP-PRP scheme would be one such that there exists a positive real constant $c > 0$ such that the scheme is $(2^{\lambda^c}, \frac{1}{2^{\lambda^c}})$ -secure.

Remark A.2 (From [SZ25]). We will often overload notation, and use the same symbol for both Π and Eval (and corresponding symbols for Π^{-1} and Eval^{-1}), when clear from context whether we are using a permuted or normal key. In this case, an OP-PRP would be just a triple $(\Pi, \Pi^{-1}, \text{Permute})$.

We say that a PRP is an input permutable (IP-)PRP if we apply Γ to the inputs rather than the outputs of Π (and accordingly apply Γ^{-1} to the output, and not the input, of Π^{-1}). A PRP which is simultaneously an IP-PRP and OP-PRP is called a permutable PRP (P-PRP).

Definition 9 (Decomposable Permutations, [SZ25]). Let $N \in \mathbb{N}$, Γ a permutation on $[N]$, and let $T, s : \mathbb{N} \rightarrow \mathbb{N}$. We say that Γ is $(T(N), s(N))$ -decomposable if there exists a sequence of permutations $\Gamma_0, \dots, \Gamma_{T(N)}$ such that:

1. Γ_0 is the identity, and $\Gamma_{T(N)} = \Gamma$.
2. Each Γ_i, Γ_i^{-1} has circuit size at most $s(N)$.
3. For each i , either (a) $\Gamma_i = \Gamma_{i-1}$ or (b) there exists a $z_i \in [N]$ such that $\Gamma_i = \Gamma_{i-1} \circ (z_i \ z_i + 1)$. Here $(z_i \ z_i + 1)$ is the neighbor-swap permutation for z_i , which swaps between z_i and $z_i + 1 \pmod{N}$, and acts as the identity on all other elements in $[N]$.

In the uniform setting, we will additionally ask that there is a uniform polynomial-time (quantum) algorithm which, given the description of Γ and i , constructs both z_i (if it exists) and the circuits for Γ_i, Γ_i^{-1} .

We refer the reader to [SZ25] for examples of decomposable permutations; we will only need to consider two such classes of efficiently decomposable permutations.

Lemma A.3 ([SZ25]). *The following classes of permutations are decomposable:*

1. *Affine permutations $\mathbf{x} \rightarrow \mathbf{Ax} + \mathbf{b}$, where $\mathbf{A} \in \mathbb{Z}_2^{n \times n}$ is invertible and $\mathbf{b} \in \mathbb{Z}_2^n$, are $(O(2^n \times n^2), \text{poly}(n))$ -decomposable.*
2. *If $N = N_0 \cdot N_1$, and for every $v \in [N_0]$, Γ_v is a (T, s) -decomposable permutation on domain $[N_1]$, the controlled permutation Γ with domain $[N] := [N_0] \times [N_1]$ where Γ applies Γ_v to $[N_1]$ conditioned on the element in $[N_0]$ being v is $(T \cdot N_0, s + \text{poly} \log(N))$ -decomposable.*

Theorem A.4 (Theorem 39, [SZ25]). *Let T be any exponential function and p any polynomial. Assuming the existence of sub-exponentially-secure one-way functions and sub-exponentially-secure iO, there exists an P-PRP for the class of (T, p) -decomposable permutations. Moreover, the P-PRP is itself (T, p) -decomposable.*

We also use the following lemma from [SZ25] which allows us to use OP-PRP's in conjunction with indistinguishability obfuscation:

Lemma A.5 (Lemma 40, [SZ25]). *Let Γ be a permutation, let $(\Pi, \Pi^{-1}, \text{Permute})$ be an (f_P, δ_P) -secure OP-PRP for a class of permutations which includes Γ , and let iO be a $(f_{\text{iO}}, \delta_{\text{iO}})$ -secure iO scheme. Then for a sufficiently large polynomial s ,*

$$\text{iO}(1^\lambda, 1^s, P^{\Pi(k, \cdot), \Pi^{-1}(k, \cdot)}) \approx_{\min(f_P, f_{\text{iO}}), O(\delta_P + \delta_{\text{iO}})} \text{iO}(1^\lambda, 1^s, P^{\Gamma(\Pi(k, \cdot), \Pi^{-1}(k, \Gamma^{-1}(\cdot)))}),$$

where $k \leftarrow \{0, 1\}^\lambda$ is uniformly random.

B Our Construction in the Plain Model

Like in [SZ25], to instantiate Construction 4, we replace the random permutation and functions with pseudorandom versions, and provide an obfuscation of the programs.

Let $\lambda \in \mathbb{N}$ be the statistical security parameter. We use the following parameters:

$$s := 16 \cdot \lambda, r := s \cdot (\lambda - 1), \ell = \ell(\lambda) := \lambda, \text{ and } n := r + \ell + \frac{3}{2} \cdot s.$$

Let $d := \text{poly}(\lambda)$ be the expansion parameter, $\kappa := \text{poly}(\lambda) \geq d$ be the cryptographic security parameter, and $p := \text{poly}(\kappa) = \text{poly}(\lambda)$ be the size parameter, for some sufficiently large polynomials.

Let iO be an iO scheme, $(F, \text{Punc}, \text{Eval})$ a puncturable PRF, and $(\Pi, \Pi^{-1}, \text{Permute})$ a permutable PRP for the class of all $(2^{\text{poly}(\kappa)}, \text{poly}(\kappa))$ -decomposable permutations.

B.1 Perfect Correctness

Lemma B.1. *For all security parameters $\lambda \in \mathbb{N}$ and messages $m \in \mathcal{M}_\lambda$,*

$$\Pr_{\substack{\text{CRS} \leftarrow \text{Setup}(1^\lambda) \\ (\text{pk}, \text{sk}) \leftarrow \text{Gen}(\text{CRS})}} [\text{Ver}(\text{CRS}, \text{pk}, m, \sigma_m) = 1 : \sigma_m \leftarrow \text{Sign}(\text{CRS}, \text{sk}, m)] = 1.$$

The perfect correctness of the signing algorithm follows from an identical argument to Section 4.

Incompressible OSS Scheme in the Plain Model

OSS.Setup(1^λ): Sample $k_{\text{in}}, k_{\text{out}}, k_{\text{lin}} \leftarrow \{0, 1\}^\kappa$. $\Pi(k_{\text{in}}, \cdot)$ is a permutation with domain $\{0, 1\}^n$, $\Pi(k_{\text{out}}, \cdot)$ is a permutation with domain $\{0, 1\}^d$, and $F(k_{\text{lin}}, \cdot)$ is a PRF with inputs in $\{0, 1\}^d$ that outputs some polynomial number of bits. Let $H(\cdot)$ denote the first r output bits of $\Pi(k_{\text{in}}, \cdot)$ and $J(\cdot)$ denote the remaining $n - r$ bits. For each $\mathbf{y} \in \{0, 1\}^d$, let $\mathbf{B}_{\mathbf{y}} \in \mathbb{Z}_2^{(n-\ell) \times \ell}$ be a matrix, $\mathbf{C}_{\mathbf{y}} \in \mathbb{Z}_2^{(n-\ell) \times (n-r-\ell)}$ a matrix with full column-rank, and $\mathbf{b}_{\mathbf{y}} \in \mathbb{Z}_2^n$ be a vector, all generated pseudorandomly by the output of $F(k_{\text{lin}}, \mathbf{y})$. Define $\mathbf{A}_{\mathbf{y}} := \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{B}_{\mathbf{y}} & \mathbf{C}_{\mathbf{y}} \end{bmatrix} \in \mathbb{Z}_2^{n \times (n-r)}$.

As the common reference string output $\text{CRS} = (\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D})$ where $\mathcal{P} \leftarrow \text{iO}(1^\kappa, 1^P, P)$, $\mathcal{P}^{-1} \leftarrow \text{iO}(1^\kappa, 1^P, P^{-1})$, and $\mathcal{D} \leftarrow \text{iO}(1^\kappa, 1^P, D)$ such that

- $\mathcal{P}(x) = (\mathbf{y}, \mathbf{A}_{\mathbf{y}} \cdot J(x) + \mathbf{b}_{\mathbf{y}})$ where $\mathbf{y} \leftarrow \Pi^{-1}(k_{\text{out}}, H(x) || 0^{d-r})$,
- $\mathcal{P}^{-1}(\mathbf{y}, \mathbf{u}) = \begin{cases} \Pi^{-1}(k_{\text{in}}, \mathbf{w} || \mathbf{z}) & \exists \mathbf{w}, \mathbf{z} : (\Pi(k_{\text{out}}, \mathbf{y}) = \mathbf{w} || 0^{d-r}) \\ & \wedge (\mathbf{A}_{\mathbf{y}} \cdot \mathbf{z} + \mathbf{b}_{\mathbf{y}} = \mathbf{u}) \\ \perp & \text{otherwise} \end{cases}$
- $\mathcal{D}(j, \mathbf{y}, \mathbf{v}) = \begin{cases} 1 & \text{if } \mathbf{v}^T \cdot \mathbf{A}_{\mathbf{y}}^{[j:n-r]} = 0^{n-r-j+1} \text{ and } j \in [\ell + 1] \\ 0 & \text{otherwise} \end{cases}$

Let $\text{Hash}(\text{CRS}, x)$ refer to the first d bits of $\mathcal{P}(x)$. We will take the message space $\mathcal{M}_\lambda$ to be the set of all strings $m \in \{0, 1\}^{\ell(\lambda)}$.

OSS.Gen(CRS): Prepare the state $|+\rangle^{\otimes n} = \sum_{x \in \{0, 1\}^n} |x\rangle$ and apply $\mathcal{P}$. Measure Hash to get a string $\text{pk} := \mathbf{y}$ and state $\sum_{x: \mathcal{P}(x)_{[1:d]} = \mathbf{y}} |x\rangle |\mathbf{A}_{\mathbf{y}} \cdot \mathbf{w}_x + \mathbf{b}_{\mathbf{y}}\rangle$, where $\mathbf{w}_x$ is $J(x)$ interpreted as a vector in $\mathbb{Z}_2^{n-r}$. Apply $\mathcal{P}^{-1}(\mathbf{y}, \cdot)$ to uncompute x , resulting in the state $|\text{sk}\rangle := (\mathbf{y}, \sum_{x: \mathcal{P}(x)_{[1:d]} = \mathbf{y}} |\mathbf{A}_{\mathbf{y}} \cdot \mathbf{w}_x + \mathbf{b}_{\mathbf{y}}\rangle)$. Output $(\text{pk}, |\text{sk}\rangle)$.

OSS.Sign(CRS, $|\text{sk}\rangle, m$): Identical to the signing algorithm of Section 4.

OSS.Ver(CRS, pk, m, σ): Output 1 if $\sigma_{[1:\ell]} = m$ and $\mathcal{P}^{-1}(\text{pk}, \sigma) \neq \perp$; else, output 0.

Figure 4: Incompressible OSS scheme in the plain model.

B.2 Strong Security

We now show that our one-shot signature scheme is strongly secure. We use the same proof strategy as the plain model proof in [SZ25], modifying components as needed. As such, we defer most proofs to Appendix F.

We begin by considering what happens when we “bloat” the dual oracle. Let $\widetilde{\text{Setup}}(1^\lambda, n, r, \ell, s)$ denote the following modified setup protocol. It samples a distribution over $\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}$. $\mathcal{P}$ and $\mathcal{P}^{-1}$ are defined identically; however, our new dual program $\mathcal{D}' := \text{iO}(1^\kappa, 1^P, \mathcal{D}')$ will be a relaxed version of the original dual program $\mathcal{D} := \text{iO}(1^\kappa, 1^P, \mathcal{D})$:

$$\mathcal{D}'(j, \mathbf{y}, \mathbf{v}) = \begin{cases} 1 & \text{if } \mathbf{v}^T \cdot \begin{bmatrix} \mathbf{A}_{\mathbf{y}}^{[j:\ell]} & \mathbf{A}_{\mathbf{y}}^{[\ell+1+s:n-r]} \end{bmatrix} = 0^{n-r-s-j+1} \text{ and } j \in [\ell + 1], \\ 0 & \text{otherwise.} \end{cases}$$

Observe that $\widetilde{\text{Setup}}(1^\lambda, n, r, \ell, 0)$ is precisely the distribution of programs in Construction 4. The following lemma states that any adversary $\mathcal{A}$ that finds a collision given access to $\mathcal{P}$, $\mathcal{P}^{-1}$, and $\mathcal{D}$, is also likely to find a collision⁷ given access to $\mathcal{P}$, $\mathcal{P}^{-1}$, and $\mathcal{D}'$.

Lemma B.2. *Let $\lambda, \kappa, n, r, s, \ell \in \mathbb{N}^+$ such that $r + s + \ell \leq n$, $\kappa \geq n - r - s - \ell$, and $n = \text{poly}(\lambda)$. Suppose there is a quantum algorithm $\mathcal{A}$ with complexity $T_{\mathcal{A}}$ such that*

$$\Pr \left[\begin{array}{l} (\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}) \leftarrow \widetilde{\text{Setup}}(1^\lambda, n, r, \ell, 0) \\ (y_0 = y_1) \wedge (x_0 \neq x_1) : \quad (x_0, x_1) \leftarrow \mathcal{A}(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}) \\ (y_b, \mathbf{u}_b) \leftarrow \mathcal{P}(x_b) \end{array} \right] \geq \varepsilon,$$

for some non-negligible function $\varepsilon := \varepsilon(\lambda)$.

Suppose that the primitives used in Construction 4 are $(f(\kappa), \frac{1}{f(\kappa)})$ -secure for some sub-exponential $f(\kappa) := 2^{\kappa^\delta}$ for some constant real number $\delta > 0$. Suppose there exists a constant $c > 0$ such that for $w := \kappa^{c\delta}$ and $t := (n - r - s)$, all of the following are true:

1. $\frac{(2^r + T_{\mathcal{A}}) \cdot n^2 / \varepsilon}{f(\kappa)} \leq o(1)$,
2. $2^w \cdot \frac{t}{\varepsilon} \cdot (2^{t-s} + 2^{-t+\ell}) \leq o(1)$, and
3. $\frac{2^w \cdot \text{poly}(n, \kappa) \cdot T_{\mathcal{A}} / \varepsilon}{f(n-r-s-\ell)} \leq o(1)$.

Then,

$$\Pr \left[\begin{array}{l} (y_0 = y_1 := \mathbf{y}) \wedge \\ (\mathbf{u}_0 - \mathbf{u}_1) \notin \text{ColSpan} \left(\begin{bmatrix} \mathbf{A}_y^{[1:\ell]} & \mathbf{A}_y^{[\ell+s+1:n-r]} \end{bmatrix} \right) : \quad \begin{array}{l} (\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}') \leftarrow \widetilde{\text{Setup}}(1^\lambda, n, r, \ell, s) \\ (x_0, x_1) \leftarrow \mathcal{A}(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}') \\ (y_b, \mathbf{u}_b) \leftarrow \mathcal{P}(x_b) \end{array} \end{array} \right] \geq \frac{\varepsilon}{512n^2}.$$

We now show that given an adversary which succeeds with a bloated dual program, one can actually entirely simulate the dual with only access to the primal program obfuscations.

Lemma B.3. *Let $\lambda, \kappa, n, r, s, \ell \in \mathbb{N}^+$ such that $r + s + \ell \leq n$, $\kappa \geq n - r - s - \ell$, and $n = \text{poly}(\lambda)$. Suppose there is a quantum algorithm $\mathcal{A}$ with complexity $T_{\mathcal{A}}$ such that*

$$\Pr \left[\begin{array}{l} (y_0 = y_1 := \mathbf{y}) \wedge \\ (\mathbf{u}_0 - \mathbf{u}_1) \notin \text{ColSpan} \left(\begin{bmatrix} \mathbf{A}_y^{[1:\ell]} & \mathbf{A}_y^{[\ell+s+1:n-r]} \end{bmatrix} \right) : \quad \begin{array}{l} (\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}') \leftarrow \widetilde{\text{Setup}}(1^\lambda, n, r, \ell, s) \\ (x_0, x_1) \leftarrow \mathcal{A}(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}') \\ (y_b, \mathbf{u}_b) \leftarrow \mathcal{P}(x_b) \end{array} \end{array} \right] \geq \varepsilon.$$

Suppose that the primitives used in Construction 4 are $(f(\kappa), \frac{1}{f(\kappa)})$ -secure for some sub-exponential $f(\kappa) := 2^{\kappa^\delta}$ for some constant real number $\delta > 0$, $\frac{2^r / \varepsilon}{f(\kappa)} \leq o(1)$, and $\frac{T_{\mathcal{A}} + \text{poly}(\kappa)}{f(\kappa)} \leq o(1)$. Then there is a quantum algorithm $\mathcal{B}$ running in time $T_{\mathcal{A}} + \text{poly}(\kappa)$ such that

$$\Pr \left[\begin{array}{l} (\overline{\mathcal{P}}, \overline{\mathcal{P}}^{-1}, \overline{\mathcal{D}}) \leftarrow \widetilde{\text{Setup}}(1^\lambda, r + s, r, 0, 0) \\ (\overline{y}_0 = \overline{y}_1) \wedge (\overline{x}_0 \neq \overline{x}_1) : \quad \begin{array}{l} (\overline{x}_0, \overline{x}_1) \leftarrow \mathcal{B}^{\overline{\mathcal{P}}, \overline{\mathcal{P}}^{-1}} \\ (\overline{y}_b, \overline{\mathbf{u}}_b) \leftarrow \overline{\mathcal{P}}(\overline{x}_b) \end{array} \end{array} \right] \geq \varepsilon/2.$$

The proofs of Lemmas B.2 and B.3 closely follow the proofs of Lemmas 54 and 55 in [SZ25], so we defer their proofs to Appendix F.

Having reduced to the dual-free case, we can now directly use the collision resistance argument of [SZ25].

⁷Technically, the lemma states that $\mathcal{A}$ produces something which is stronger than a collision.

Theorem B.4 (Adapted from Theorem 56, [SZ25]). Let $r, s \in \mathbb{N}$ such that $s \mid r$. Suppose the post-quantum sub-exponential hardness of LWE and that d is a sufficiently large polynomial in λ .

Then, for every QPT algorithm $\mathcal{A}$, the probability that $\mathcal{A}(\mathcal{P}, \mathcal{P}^{-1})$ outputs a collision in Hash , where $(\mathcal{P}, \mathcal{P}^{-1})$ is sampled from $(\mathcal{P}, \mathcal{P}^{-1}, D) \leftarrow \widetilde{\text{Setup}}(1^\lambda, r + s, r, 0, 0)$, is $\text{negl}((r + s)/s)$.

By combining Lemma B.2, Lemma B.3, and Theorem B.4, we see that our hash function is collision-resistant.

Corollary B.5. Let $\lambda \in \mathbb{N}$ and set all other parameters accordingly to Construction 4. Then, for every QPT algorithm $\mathcal{A}$,

$$\Pr_{\substack{\text{CRS} \leftarrow \text{Setup}(1^\lambda) \\ (x_0, x_1) \leftarrow \mathcal{A}(\text{CRS})}} [(x_0 \neq x_1) \wedge (\text{Hash}(\text{CRS}, x_0) = \text{Hash}(\text{CRS}, x_1))] \leq \text{negl}(\lambda).$$

We can now conclude that our one-shot signature scheme is strongly secure, from an identical argument as in the proof of Corollary 4.10.

Corollary B.6. Let $\lambda \in \mathbb{N}$ and set all other parameters accordingly to Construction 4. Then, for every QPT algorithm $\mathcal{A}$,

$$\Pr_{\substack{\text{CRS} \leftarrow \text{Setup}(1^\lambda) \\ (\text{pk}, m_0, \sigma_0, m_1, \sigma_1) \leftarrow \mathcal{A}(\text{CRS})}} \left[\begin{array}{l} (m_0, \sigma_0) \neq (m_1, \sigma_1) \wedge \\ \forall b \in \{0, 1\}, \text{Ver}(\text{CRS}, \text{pk}, m_b, \sigma_b) = 1 \end{array} \right] \leq \text{negl}(\lambda).$$

C Signing Polynomial-Length Messages

With the ability to sign λ -sized messages with signatures of size $O(\lambda^2)$ and a single signing key, we show how to extend this to messages of arbitrary polynomial length without blowing up the signature size. The idea is simple (in both the oracle and plain models): we will use the classic *hash-and-sign* approach, either with a random oracle or with a collision-resistant hash function.

C.1 Oracle Model Construction

In this section, we describe how to extend Construction 2 to sign arbitrary polynomial length messages using a random oracle.

Construction 5 : Oracle Model Hash-and-Sign OSS

Let $\lambda \in \mathbb{N}$ be the security parameter, and let $(\text{Gen}, \text{Sign}, \text{Ver})$ be any oracle-based one-shot signature scheme for messages of length λ which uses the oracle distribution $\mathcal{O} \leftarrow \mathcal{O}_\lambda$. Let $\mathcal{H} : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ be a random oracle drawn from the distribution $\mathcal{H}_\lambda$. We omit Setup and let a random sample from $\mathcal{H}_\lambda, \mathcal{O}_\lambda$ play the role of the CRS. Our other algorithms are as follows:

- $\widetilde{\text{Gen}}^{\mathcal{H}, \mathcal{O}} : \text{Output } (\text{pk}, |\text{sk}\rangle) \leftarrow \text{Gen}^{\mathcal{O}}$.
- $\widetilde{\text{Sign}}^{\mathcal{H}, \mathcal{O}}(|\text{sk}\rangle, m)$: Apply $\mathcal{H}(m)$ to get a message $m' \in \{0, 1\}^\lambda$. Output $\sigma \leftarrow \text{Sign}^{\mathcal{O}}(|\text{sk}\rangle, m')$.
- $\widetilde{\text{Ver}}^{\mathcal{H}, \mathcal{O}}(\text{pk}, m, \sigma)$: Apply $\mathcal{H}(m)$ to get a message $m' \in \{0, 1\}^\lambda$. Output $b \leftarrow \text{Ver}^{\mathcal{O}}(\text{pk}, m', \sigma)$.

Correctness follows directly from the equivocality of the base OSS scheme. We now show that the hash-and-sign scheme is still strongly secure.

Lemma C.1. Let $(\text{Gen}, \text{Sign}, \text{Ver})$ be any one-shot signature scheme which uses the oracle distribution $\{\mathcal{O}_\lambda\}_{\lambda \in \mathbb{N}}$ with collision-resistance $\varepsilon_{\text{OSS}}(q, \lambda)$ against q -query quantum algorithms for a security parameter λ . Let $\mathcal{A}$ be an oracle aided q -query quantum algorithm. Then, for all λ ,

$$\Pr \left[\begin{array}{l} (m_0, \sigma_0) \neq (m_1, \sigma_1) \wedge \quad \mathcal{H} \leftarrow \mathcal{H}_\lambda, \mathcal{O} \leftarrow \mathcal{O}_\lambda \\ \widetilde{\text{Ver}}^{\mathcal{H}, \mathcal{O}}(\text{pk}, m_0, \sigma_0) = 1 \wedge \quad : (\text{pk}, m_b, \sigma_b) \leftarrow \mathcal{A}^{\mathcal{H}, \mathcal{O}} \\ \widetilde{\text{Ver}}^{\mathcal{H}, \mathcal{O}}(\text{pk}, m_1, \sigma_1) = 1 \quad \text{for } b \in \{0, 1\} \end{array} \right] \leq \varepsilon_{\text{OSS}}(q, \lambda) + O\left(\frac{q^3}{2^\lambda}\right),$$

where $\mathcal{H} \leftarrow \mathcal{H}_\lambda : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ is a random oracle.

Proof. The lemma follows from a straightforward hybrid argument. Suppose that $\mathcal{A}$ produces a collision $(\text{pk}, m_0, \sigma_0, m_1, \sigma_1)$. If $\mathcal{H}(m_0) = \mathcal{H}(m_1)$ and $m_0 \neq m_1$, this produces a collision in $\mathcal{H}$. Otherwise, either $\mathcal{H}(m_0) \neq \mathcal{H}(m_1)$ or $m_0 = m_1$ and thus $\sigma_0 \neq \sigma_1$; in either case, $\mathcal{A}$ produces two valid message-signature pairs $(\mathcal{H}(m_0), \sigma_0) \neq (\mathcal{H}(m_0), \sigma_1)$ for the underlying one-shot signature scheme. Since $\mathcal{A}$ makes q total queries, it makes at most q queries to $\mathcal{O}$ and $\mathcal{H}$ each, and so it can produce two such message-signature pairs with probability at most $\varepsilon_{\text{OSS}}(q, \lambda)$ by assumption and a collision in $\mathcal{H}$ with probability at most $O\left(\frac{q^3}{2^\lambda}\right)$ by standard collision lower bounds [Zha15]. $\square$

Instantiating this hash-and-sign construction with the base one-shot signature scheme from Section 4 gives a strongly secure scheme for signing arbitrary polynomial-length messages:

Corollary C.2. Let $(\text{Gen}, \text{Sign}, \text{Ver})$ be the one-shot signature scheme from Section 4, and define the one-shot signature scheme $(\widetilde{\text{Gen}}, \widetilde{\text{Sign}}, \widetilde{\text{Ver}})$ accordingly. Let $\mathcal{A}$ be an oracle aided q -query quantum algorithm. Then,

$$\Pr \left[\begin{array}{l} (m_0, \sigma_0) \neq (m_1, \sigma_1) \wedge \quad \mathcal{H} \leftarrow \mathcal{H}_\lambda, (P, P^{-1}, D) \leftarrow \mathcal{O}_{n, r, \ell} \\ \forall b \in \{0, 1\}, \widetilde{\text{Ver}}^{\mathcal{H}, P, P^{-1}, D}(\text{pk}, m_b, \sigma_b) = 1 \quad : \quad (\text{pk}, m_0, \sigma_0, m_1, \sigma_1) \leftarrow \mathcal{A}^{\mathcal{H}, P, P^{-1}, D} \end{array} \right] \leq O\left(\frac{\lambda^3 q^3 n^3}{2^\lambda}\right),$$

where $\mathcal{H} \leftarrow \mathcal{H}_\lambda : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ is a random oracle.

Perfect correctness of $(\widetilde{\text{Gen}}, \widetilde{\text{Sign}}, \widetilde{\text{Ver}})$ follows immediately from the perfect correctness of $(\text{Gen}, \text{Sign}, \text{Ver})$.

C.2 Plain Model Construction

In this section, we describe how to extend Construction 4 to sign arbitrary polynomial length messages by instantiating the random oracle with an arbitrary domain collision-resistant hash function.

Construction 6 : Plain Model Hash-and-Sign OSS

Let λ be the security parameter. Let $(\text{Setup}_\lambda, \text{Gen}_\lambda, \text{Sign}_\lambda, \text{Ver}_\lambda)$ be any one-shot signature scheme which signs messages of length $\ell(\lambda)$, and let $\{H_\lambda\}_{\lambda \in \mathbb{N}} : \{0, 1\}^* \rightarrow \{0, 1\}^{\ell(\lambda)}$ be any family of collision-resistant hash functions. Our hash-and-sign based one-shot signature scheme is as follows:

- $\widetilde{\text{Setup}}(1^\lambda)$: Sample $\text{CRS} \leftarrow \text{Setup}(1^\lambda)$ and hash function $h \leftarrow H_\lambda$. Output $\text{CRS}' = (h, \text{CRS})$.
- $\widetilde{\text{Gen}}(\text{CRS}' = (h, \text{CRS}))$: Output $(\text{pk}, |\text{sk}\rangle) \leftarrow \text{Gen}(\text{CRS})$.
- $\widetilde{\text{Sign}}(\text{CRS}' = (h, \text{CRS}), |\text{sk}\rangle, m)$: Compute $m' := h(m) \in \{0, 1\}^{\ell(\lambda)}$. Output $\sigma \leftarrow \text{Sign}(\text{CRS}, |\text{sk}\rangle, m')$.
- $\widetilde{\text{Ver}}(\text{CRS}' = (h, \text{CRS}), \text{pk}, m, \sigma)$: Compute $m' := h(m) \in \{0, 1\}^{\ell(\lambda)}$. Output $b \leftarrow \text{Ver}(\text{CRS}, \text{pk}, m', \sigma)$.

The proof of the following lemma is identical to the proof of Lemma C.1 and follows from a straightforward hybrid argument.

Lemma C.3. Let $(\text{Setup}_\lambda, \text{Gen}_\lambda, \text{Sign}_\lambda, \text{Ver}_\lambda)$ and $\{H_\lambda\}_{\lambda \in \mathbb{N}} : \{0, 1\}^* \rightarrow \{0, 1\}^{\ell(\lambda)}$ be any one-shot signature scheme which signs messages of length $\ell(\lambda)$ and any family of hash functions with collision resistance $\varepsilon_{\text{OSS}}(\lambda)$ and $\varepsilon_{\text{CRHF}}(\lambda)$, respectively, against QPT algorithms for a security parameter λ . Then, for all QPT algorithms $\mathcal{A}$,

$$\Pr_{\substack{\text{CRS}' \leftarrow \text{Setup}(1^\lambda) \\ (\text{pk}, m_0, m_1, \sigma_0, \sigma_1) \leftarrow \mathcal{A}(\text{CRS}')}} \left[\begin{array}{l} (m_0, \sigma_0) \neq (m_1, \sigma_1) \wedge \\ \forall b \in \{0, 1\}, \widetilde{\text{Ver}}(\text{CRS}', \text{pk}, m_b, \sigma_b) = 1 \end{array} \right] \leq \varepsilon_{\text{OSS}}(\lambda) + \varepsilon_{\text{CRHF}}(\lambda).$$

Instantiating the one-shot signature scheme with the scheme in Section B and any LWE-based collision-resistant hash function gives the following corollary:

Corollary C.4. Assuming (1) subexponentially-secure indistinguishability obfuscation, (2) subexponentially-secure one-way functions, and (3) (polynomially-secure) LWE with a subexponential noise-modulus ratio, there exists a secure OSS scheme with perfect correctness and $O(\lambda^2)$ -bit signatures and $O(\lambda^2)$ -qubit signing keys for $\text{poly}(\lambda)$ -bit messages.

D Subspace Hiding Lemmas

We begin by stating two lemmas on equivalent ways to sample superspaces which will be used throughout this work.

Lemma D.1. Let $n, r, s, \ell \in \mathbb{N}^+$ such that $r + s + \ell \leq n$, and let $S_1 \subseteq \dots \subseteq S_{\ell+1} \subseteq \mathbb{Z}_2^n$ be subspaces where S_j has dimension $r + j - 1$. Fix any full column-rank matrix $\mathbf{A} \in \mathbb{Z}_2^{n \times (n-r)}$ such that $S_j^\perp := \text{ColSpan}(\mathbf{A}^{[j:n-r]})$. Then, the following distributions are statistically equivalent:

1. Sample a random vector $\mathbf{v} \in \mathbb{Z}_2^n \setminus S_{\ell+1}$ and output $(T_{1,1}, \dots, T_{\ell+1,1})$, where $T_{j,1} := S_j \cup (S_j + \mathbf{v})$ for all j .
2. Sample a random vector $\mathbf{v} \in \mathbb{Z}_2^n \setminus S_{\ell+1}$. Compute $\mathbf{y} = \mathbf{v}^T \cdot \mathbf{A} \in \mathbb{Z}_2^{n-r}$. Output $(T_{1,2}, \dots, T_{\ell+1,2})$, where $T_{j,2} := \{\mathbf{w} \in \mathbb{Z}_2^n \mid (\mathbf{w}^T \cdot \mathbf{A}^{[j:n-r]} = 0^{n-r-j+1}) \vee (\mathbf{w}^T \cdot \mathbf{A}^{[j:n-r]} = \mathbf{y}_{[j:n-r]})\}$ for all j .
3. Sample a random matrix $\mathbf{B} \in \mathbb{Z}_2^{(n-r-\ell) \times \ell}$ and compute $\mathbf{C} := \mathbf{A} \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{B} & \mathbf{I}_{n-r-\ell} \end{bmatrix}$. Sample a random vector $\mathbf{z} \in \mathbb{Z}_2^{n-r-\ell} \setminus \{0\}$. Output $(T_{1,3}, \dots, T_{\ell+1,3})$, where $T_{j,3} := \{\mathbf{w} \in \mathbb{Z}_2^n \mid (\mathbf{w}^T \cdot \mathbf{C}^{[j:n-r]} = 0^{n-r-j+1}) \vee (\mathbf{w}^T \cdot \mathbf{C}^{[j:n-r]} = 0^{\ell-j+1} \parallel \mathbf{z})\}$ for all j .

Proof. We first show that Distributions 1 and 2 are equivalent distributions. By hypothesis, we have that

$$\{\mathbf{w} \in \mathbb{Z}_2^n \mid \mathbf{w}^T \cdot \mathbf{A}^{[j:n-r]} = 0^{n-r-j+1}\} = \{\mathbf{w} \in \mathbb{Z}_2^n \mid \mathbf{w} \in \text{ColSpan}(\mathbf{A}^{[j:n-r]})^\perp\} = S_j.$$

Fix any vector $\mathbf{v}$ in $\mathbb{Z}_2^n \setminus S_{\ell+1}$. Consider any index j and any vector $\mathbf{x} \in S_j + \mathbf{v}$. By definition, $\mathbf{x} = \mathbf{s}_j + \mathbf{v}$ for some vector $\mathbf{s}_j \in S_j$. Thus,

$$\mathbf{x}^T \cdot \mathbf{A}^{[j:n-r]} = (\mathbf{s}_j + \mathbf{v})^T \cdot \mathbf{A}^{[j:n-r]} = \mathbf{s}_j^T \cdot \mathbf{A}^{[j:n-r]} + \mathbf{v}^T \cdot \mathbf{A}^{[j:n-r]} = \mathbf{v}^T \cdot \mathbf{A}^{[j:n-r]} = \mathbf{y}_{[j:n-r]},$$

so $\mathbf{x} \in \{\mathbf{w} \in \mathbb{Z}_2^n \mid \mathbf{w}^T \cdot \mathbf{A}^{[j:n-r]} = \mathbf{y}_{[j:n-r]}\}$. In other words, $T_{j,1} \subseteq T_{j,2}$ for all j . But observe that since $\mathbf{A}$ has full column-rank, $T_{j,2}$ is a $(r + j)$ -dimensional subspace, while $\mathbf{v} \in \mathbb{Z}_2^n \setminus S_{\ell+1} \subseteq \mathbb{Z}_2^n \setminus S_j$, so $T_{j,1}$ is also a $(r + j)$ -dimensional subspace. This means that for a given $\mathbf{v}$, $(T_{1,1}, \dots, T_{\ell+1,1}) = (T_{1,2}, \dots, T_{\ell+1,2})$, so Distributions 1 and 2 are equivalent distributions (over random $\mathbf{v}$).

We now show that Distributions 2 and 3 are equivalent distributions. First, observe that for *any* matrix $\mathbf{B}$, we have that

$$\begin{aligned}\mathbf{w}^T \cdot \mathbf{C}^{[j:n-r]} &= \mathbf{w}^T \cdot \left(\mathbf{A} \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{B} & \mathbf{I}_{n-r-\ell} \end{bmatrix} \right)^{[j:n-r]} \\ &= \mathbf{w}^T \cdot \left(\left[(\mathbf{A}^{[1:\ell]} + \mathbf{A}^{[\ell+1:n-r]} \cdot \mathbf{B}) \quad \mathbf{A}^{[\ell+1:n-r]} \right] \right)^{[j:n-r]} \\ &= \mathbf{w}^T \cdot \left[(\mathbf{A}^{[j:\ell]} + \mathbf{A}^{[\ell+1:n-r]} \cdot \mathbf{B}^{[j:\ell]}) \quad \mathbf{A}^{[\ell+1:n-r]} \right] \\ &= \mathbf{w}^T \cdot \mathbf{A}^{[j:n-r]} + \left[(\mathbf{w}^T \cdot \mathbf{A}^{[\ell+1:n-r]} \cdot \mathbf{B}^{[j:\ell]}) \quad 0^{n-r-\ell} \right].\end{aligned}$$

Thus, if $\mathbf{w}^T \cdot \mathbf{A}^{[j:n-r]} = 0^{n-r-j+1}$, then

$$\begin{aligned}\mathbf{w}^T \cdot \mathbf{C}^{[j:n-r]} &= \mathbf{w}^T \cdot \mathbf{A}^{[j:n-r]} + \left[(\mathbf{w}^T \cdot \mathbf{A}^{[\ell+1:n-r]} \cdot \mathbf{B}^{[j:\ell]}) \quad 0 \right] \\ &= 0^{n-r-j+1} + \left[(0^{n-r-\ell} \cdot \mathbf{B}^{[j:\ell]}) \quad 0^{n-r-\ell} \right] = 0^{n-r-j+1}.\end{aligned}$$

But the matrix $\begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{B} & \mathbf{I}_{n-r-\ell} \end{bmatrix}$ is its own inverse, so we also have that $\mathbf{A} = \mathbf{C} \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{B} & \mathbf{I}_{n-r-\ell} \end{bmatrix}$ and thus

$$\mathbf{w}^T \cdot \mathbf{A}^{[j:n-r]} = \mathbf{w}^T \cdot \mathbf{C}^{[j:n-r]} + \left[(\mathbf{w}^T \cdot \mathbf{C}^{[\ell+1:n-r]} \cdot \mathbf{B}^{[j:\ell]}) \quad 0^{n-r-\ell} \right].$$

Therefore, if $\mathbf{w}^T \cdot \mathbf{C}^{[j:n-r]} = 0^{n-r-j+1}$, it must also be the case that $\mathbf{w}^T \cdot \mathbf{A}^{[j:n-r]} = 0^{n-r-j+1}$.

Since $\mathbf{A}$ is full column-rank and $\begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{B} & \mathbf{I}_{n-r-\ell} \end{bmatrix}$ is full-rank for all $\mathbf{B}$, $\mathbf{C}$ is always full column-rank. Thus, for all $\mathbf{v}$, $T_{j,2}$ and $T_{j,3}$ are $(r+j)$ -dimensional subspaces. It therefore suffices to show the following distributions are equivalent:

- Sample a random vector $\mathbf{v} \in \mathbb{Z}_2^n \setminus S_{\ell+1}$. Compute $\mathbf{y} = \mathbf{v}^T \cdot \mathbf{A} \in \mathbb{Z}_2^{n-r}$. Output $(U_1, \dots, U_{\ell+1})$, where $U_j := \{\mathbf{w} \in \mathbb{Z}_2^n \mid \mathbf{w}^T \cdot \mathbf{A}^{[j:n-r]} = \mathbf{y}_{[j:n-r]}\}$ for all j .
- Sample a random matrix $\mathbf{B} \in \mathbb{Z}_2^{(n-r-\ell) \times \ell}$ and compute $\mathbf{C} := \mathbf{A} \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{B} & \mathbf{I}_{n-r-\ell} \end{bmatrix}$. Sample a random vector $\mathbf{z} \in \mathbb{Z}_2^{n-r-\ell} \setminus \{0\}$. Output $(U'_1, \dots, U'_{\ell+1})$, where $U'_j := \{\mathbf{w} \in \mathbb{Z}_2^n \mid \mathbf{w}^T \cdot \mathbf{C}^{[j:n-r]} = 0^{\ell-j+1} \parallel \mathbf{z}\}$ for all j .

Note that since $\mathbf{A}$ has full column-rank, the multiplication-by- $\mathbf{A}$ map $L : \mathbb{Z}_2^n \rightarrow \mathbb{Z}_2^{n-r}$, defined as taking $\mathbf{v} \mapsto \mathbf{v} \cdot \mathbf{A}$, is 2^r -to-1. Since $L(S_{\ell+1}) = \text{span}\{\mathbf{e}_1, \dots, \mathbf{e}_\ell\}$, we see that $L(\mathbb{Z}_2^n \setminus S_{\ell+1}) = \mathbb{Z}_2^{n-r} \setminus \text{span}\{\mathbf{e}_1, \dots, \mathbf{e}_\ell\}$. In other words, in the first distribution, $\mathbf{y}$ is a uniformly random vector in $\mathbb{Z}_2^{n-r}$ whose last $n-r-\ell$ coordinates are not all zero.

We now consider the second distribution: for a fixed matrix $\mathbf{B}$ and index j , we have that

$$\begin{aligned}\mathbf{w}^T \cdot \mathbf{C}^{[j:n-r]} &= [0^{\ell-j+1} \quad \mathbf{z}] \\ \Leftrightarrow \exists \mathbf{s}_j \in \mathbb{Z}_2^{j-1} : \mathbf{w}^T \cdot \mathbf{C} &= [\mathbf{s}_j \quad 0^{\ell-j+1} \quad \mathbf{z}] \\ \Leftrightarrow \exists \mathbf{s}_j \in \mathbb{Z}_2^{j-1} : \mathbf{w}^T \cdot \mathbf{A} \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{B} & \mathbf{I}_{n-r-\ell} \end{bmatrix} &= [\mathbf{s}_j \quad 0^{\ell-j+1} \quad \mathbf{z}] \\ \Leftrightarrow \exists \mathbf{s}_j \in \mathbb{Z}_2^{j-1} : \mathbf{w}^T \cdot \mathbf{A} &= [\mathbf{s}_j \quad 0^{\ell-j+1} \quad \mathbf{z}] \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{B} & \mathbf{I}_{n-r-\ell} \end{bmatrix} \\ \Leftrightarrow \exists \mathbf{s}_j \in \mathbb{Z}_2^{j-1} : \mathbf{w}^T \cdot \mathbf{A} &= [(\mathbf{s}_j \quad 0^{\ell-j+1}) + \mathbf{z} \cdot \mathbf{B} \quad \mathbf{z}] \\ \Leftrightarrow \mathbf{w}^T \cdot \mathbf{A}^{[j:n-r]} &= ((\mathbf{z} \cdot \mathbf{B} \quad \mathbf{z}))_{[j:n-r]}.\end{aligned}$$

For any fixed nonzero vector $\mathbf{z}$, over the randomness of $\mathbf{B}$, each column of $\mathbf{z} \cdot \mathbf{B}$ is a i.i.d. random bit, so the joint distribution of $\begin{bmatrix} (\mathbf{z} \cdot \mathbf{B}) & \mathbf{z} \end{bmatrix}$ for a random nonzero $\mathbf{z}$ and random matrix $\mathbf{B}$ is precisely a uniformly random vector in $\mathbb{Z}_2^{n-r}$ whose last $n-r-\ell$ coordinates are not all zero. Thus, $(U_1, \dots, U_{\ell+1}) \equiv (U'_1, \dots, U'_{\ell+1})$ which concludes the proof. $\square$

Lemma D.2. Let $n, r, s, \ell \in \mathbb{N}^+$ such that $r + s + \ell \leq n$. Fix any matrix $\mathbf{B} \in \mathbb{Z}_2^{(n-\ell) \times \ell}$ and any full column-rank matrix $\mathbf{C} \in \mathbb{Z}_2^{(n-\ell) \times (n-r-\ell)}$; this defines $\mathbf{A} := \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{B} & \mathbf{C} \end{bmatrix}$ and $S_j := \text{ColSpan}(\mathbf{A}^{[j:n-r]})^\perp$.

Then, the following distributions over subspaces are statistically equivalent:

1. Sample s random linearly independent vectors $\mathbf{v}_1, \dots, \mathbf{v}_s$ such that

$$\text{span}\{\mathbf{v}_1, \dots, \mathbf{v}_s\} \cap S_{\ell+1} = \{0\}.$$

Compute $T_j := \text{span}\{S_j, \mathbf{v}_1, \dots, \mathbf{v}_s\}$ for all j . Output $(T_1, \dots, T_{\ell+1})$.

2. Sample a random full-rank matrix $\mathbf{M} \in \mathbb{Z}_2^{(n-r-\ell) \times (n-r-\ell)}$ and random matrix $\mathbf{M}' \in \mathbb{Z}_2^{(n-r-\ell) \times \ell}$. Compute

$$\bar{\mathbf{A}} := \mathbf{A} \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{M}' & \mathbf{M} \end{bmatrix} \text{ and}$$

$$T_j := \text{ColSpan} \left(\begin{bmatrix} \bar{\mathbf{A}}^{[j:\ell]} & \bar{\mathbf{A}}^{[\ell+s+1:n-r]} \end{bmatrix} \right)^\perp.$$

Output $(T_1, \dots, T_{\ell+1})$.

Proof. We first prove the lemma for the case of $s = 1$. This means that Distribution 1 consist of sampling any vector $\mathbf{v} \notin S_{\ell+1}$ and setting $T_j = \text{span}\{S_j, \mathbf{v}\}$. We claim that for fixed S_j , there are exactly $2^{n-r} - 2^\ell$ possible sets of subspaces $T_1 \subseteq \dots \subseteq T_{\ell+1}$. This follows from two facts:

- For any two vectors $\mathbf{v}, \mathbf{w} \notin S_{\ell+1}$ such that $\text{span}\{S_1, \mathbf{v}\} = \text{span}\{S_1, \mathbf{w}\}$, we have $\mathbf{v} - \mathbf{w} \in S_1 \subseteq S_j$ for all j and so $\text{span}\{S_j, \mathbf{v}\} = \text{span}\{S_j, \mathbf{w}\}$. Thus, determining T_1 fixes all other subspaces $T_2, \dots, T_{\ell+1}$. It is also not hard to see that among all vectors $\mathbf{v} \notin S_{\ell+1}$ consistent with a superspace T_{j+1} , there are exactly two possible superspaces $T_j := \text{span}\{S_j, \mathbf{v}\}$.
- T_1 is always a dimension $(r+1)$ superspace of S_1 such that $T_1 \subsetneq S_{\ell+1}$. There are at most $2^{n-r} - 2^\ell$ such superspaces. On the other hand, there are $2^n - 2^{r+\ell}$ vectors $\mathbf{v} \notin S_{\ell+1}$, and at most $|S_1| = 2^r$ will result in the same T_1 , so there are at least $\frac{2^n - 2^{r+\ell}}{2^r} = 2^{n-r} - 2^\ell$ such superspaces.

Moreover, these two facts imply that Distribution 1 is the uniform distribution over all possible sets of subspaces $T_1 \subseteq \dots \subseteq T_{\ell+1}$ such that $T_j = \text{span}\{S_j, \mathbf{v}\}$ for some vector $\mathbf{v} \notin S_{\ell+1}$.

We now take a look at Distribution 2. We claim that for any set of subspaces $T_1 \subseteq \dots \subseteq T_{\ell+1}$ produced in Distribution 2, there exists some vector $\mathbf{v} \notin S_{\ell+1}$ such that $T_j = S_j \cup (S_j + \mathbf{v})$. Note that this is equivalent to the condition that there is some vector $\mathbf{v} \in \bigcap_{j=1}^{\ell+1} T_j \setminus S_j$.

Fix any full-rank matrix $\mathbf{M}$ and matrix $\mathbf{M}'$. We have that

$$\bar{\mathbf{A}} = \mathbf{A} \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{M}' & \mathbf{M} \end{bmatrix} = \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{B} & \mathbf{C} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{M}' & \mathbf{M} \end{bmatrix} = \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{B} + \mathbf{C} \cdot \mathbf{M}' & \mathbf{C} \cdot \mathbf{M} \end{bmatrix},$$

and $T_j = \text{ColSpan} \left(\begin{bmatrix} \bar{\mathbf{A}}^{[j:\ell]} & \bar{\mathbf{A}}^{[\ell+2:n-r]} \end{bmatrix} \right)^\perp$. This means that $\mathbf{v} \in T_j$ if and only if $\mathbf{v}^T \cdot \begin{bmatrix} \bar{\mathbf{A}}^{[j:\ell]} & \bar{\mathbf{A}}^{[\ell+2:n-r]} \end{bmatrix} = 0^{n-r-j}$. On the other hand,

$$\begin{aligned} \text{ColSpan} \left(\bar{\mathbf{A}}^{[j:n-r]} \right) &= \text{ColSpan} \left(\begin{bmatrix} \mathbf{I}_\ell^{[j:\ell]} & \mathbf{0} \\ (\mathbf{B} + \mathbf{C} \cdot \mathbf{M}')^{[j:\ell]} & \mathbf{C} \cdot \mathbf{M} \end{bmatrix} \right) \\ &= \text{ColSpan} \left(\begin{bmatrix} \mathbf{I}_\ell^{[j:\ell]} & \mathbf{0} \\ \mathbf{B}^{[j:\ell]} & \mathbf{C} \end{bmatrix} \right) = \text{ColSpan} \left(\mathbf{A}^{[j:n-r]} \right), \end{aligned}$$

since $\mathbf{M}$ is a full-rank matrix. Thus, $\mathbf{v} \in S_j$ if and only if $\mathbf{v}^T \cdot \bar{\mathbf{A}}^{[j:n-r]} = 0^{n-r-j+1}$. Therefore, an equivalent condition to $\mathbf{v} \in \bigcap_{j=1}^{\ell+1} T_j \setminus S_j$ is if there exists a vector $\mathbf{v}$ such that $\mathbf{v}^T \cdot \bar{\mathbf{A}} = 0^\ell \|1\| 0^{n-r-\ell-1}$. But since $\mathbf{A}$ and $\mathbf{M}$ are both full column-rank matrices, so must $\bar{\mathbf{A}}$, and so the multiplication-by- $\bar{\mathbf{A}}$ is surjective and thus such a vector $\mathbf{v}$ must exist. In particular, this establishes that the support of Distribution 2 is a subset of the support of Distribution 1.

We move to considering the dual spaces $T_{\ell+1}^\perp \subseteq \dots \subseteq T_1^\perp$. Note that

$$T_{\ell+1}^\perp = \text{ColSpan} \left(\bar{\mathbf{A}}^{[\ell+2:n-r]} \right) = \text{ColSpan} \left(\begin{bmatrix} \mathbf{0} \\ \mathbf{C} \cdot \mathbf{M}^{[2:n-r-\ell]} \end{bmatrix} \right),$$

while

$$S_{\ell+1}^\perp = \text{ColSpan} \left(\mathbf{A}^{[\ell+1:n-r]} \right) = \text{ColSpan} \left(\begin{bmatrix} \mathbf{0} \\ \mathbf{C} \end{bmatrix} \right),$$

and since $\mathbf{M}^{[2:n-r-\ell]}$ is a random full column-rank matrix (as $\mathbf{M}$ is random and full-rank), $T_{\ell+1}^\perp$ is a uniformly random $(n-r-\ell-1)$ -dimension subspace of $S_{\ell+1}^\perp$; note that there are $2^{n-r-\ell}-1$ such subspaces.

Now let us fix any $\mathbf{M}$, which thereby fixes the last $n-r-\ell$ columns of $\bar{\mathbf{A}}$ and $T_{\ell+1}^\perp$. We claim that over the randomness of $\mathbf{M}'$, $(T_1^\perp, \dots, T_\ell^\perp)$ take on one of 2^ℓ possible sets of subspaces with equal probability. To see why this is the case, it suffices to show that for any index $j \in [\ell]$, after fixing the $(j+1)$ 'th through ℓ 'th column of $\mathbf{M}'$ (which fixes $T_{j+1}^\perp, \dots, T_\ell^\perp$), picking the j 'th column $\mathbf{m}'_j$ of $\mathbf{M}'$ at random results in one of two possible $T_j^\perp$'s with equal probability.

How is $T_j^\perp$ defined with respect to $T_{j+1}^\perp$? Recall that

$$\begin{aligned} T_j^\perp &:= \text{ColSpan} \left(\begin{bmatrix} \bar{\mathbf{A}}^{[j:\ell]} & \bar{\mathbf{A}}^{[\ell+2:n-r]} \end{bmatrix} \right) = T_{j+1}^\perp \cup \left(T_{j+1}^\perp + \bar{\mathbf{A}}^{[j]} \right) \\ &= T_{j+1}^\perp \cup \left(T_{j+1}^\perp + \begin{bmatrix} \mathbf{I}_\ell^{[j]} \\ \mathbf{B}^{[j]} \end{bmatrix} + \begin{bmatrix} \mathbf{0} \\ \mathbf{C} \cdot \mathbf{m}'_j \end{bmatrix} \right), \end{aligned}$$

where there are $2^{n-r-\ell}$ possible vectors $\mathbf{C} \cdot \mathbf{m}'_j$. For any two vectors $\mathbf{m}'_{j,1}, \mathbf{m}'_{j,2}$, the subspaces $T_{j,1}^\perp$ and $T_{j,2}^\perp$ are equal if and only if

$$\begin{aligned} &\begin{bmatrix} \mathbf{0} \\ \mathbf{C} \cdot (\mathbf{m}'_{j,1} - \mathbf{m}'_{j,2}) \end{bmatrix} \in T_{j+1}^\perp \\ \Leftrightarrow &\begin{bmatrix} \mathbf{0} \\ \mathbf{C} \cdot (\mathbf{m}'_{j,1} - \mathbf{m}'_{j,2}) \end{bmatrix} \in T_{\ell+1}^\perp = \text{ColSpan} \left(\begin{bmatrix} \mathbf{0} \\ \mathbf{C} \cdot \mathbf{M}^{[2:n-r-\ell]} \end{bmatrix} \right) \\ \Leftrightarrow &\mathbf{m}'_{j,1} - \mathbf{m}'_{j,2} \in \text{ColSpan} \left(\mathbf{M}^{[2:n-r-\ell]} \right), \end{aligned}$$

since columns $j+1$ through ℓ of $\bar{\mathbf{A}}$ have a 1 somewhere in the first ℓ coordinates and $\mathbf{C}$ has full column-rank. Since $\mathbf{M}$ has full rank and thus $\text{ColSpan}(\mathbf{M}^{[2:n-r-\ell]})$ has dimension $n-r-\ell-1$, this partitions the vectors $\mathbf{m}'_j \in \mathbb{Z}_2^{n-r-\ell}$ evenly into exactly two cosets. Thus, $T_j^\perp$ takes on one of two possible values over the randomness of the j th column of $\mathbf{M}'$.

Thus, $(T_1^\perp, \dots, T_{\ell+1}^\perp)$ take on one of $2^\ell \cdot (2^{n-r-\ell}-1) = 2^{n-r} - 2^\ell$ possible sets of subspaces with equal probability. Consequently, the output distribution $(T_1, \dots, T_{\ell+1})$ in Distribution 2 is also the uniform distribution over a support of size $2^{n-r} - 2^\ell$. But we argued earlier that the support of Distribution 2 is a subset of the support of Distribution 1, which has size $2^{n-r} - 2^\ell$. Thus Distribution 2 is also the uniform distribution over the same $2^{n-r} - 2^\ell$ possible subspaces, which concludes the proof for the case of $s = 1$.

For $s > 1$, we assume the lemma is true for all smaller s . Consider the following recursive process:

1. First, sample a random matrix $\mathbf{D}_1 \in \mathbb{Z}_2^{(n-r-\ell) \times \ell}$ and a random full-rank matrix $\mathbf{E}_1 \in \mathbb{Z}_2^{(n-r-\ell) \times (n-r-\ell)}$. Compute the matrix $\mathbf{A}_1 := \mathbf{A} \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{D}_1 & \mathbf{E}_1 \end{bmatrix}$ and subspaces $S'_j := \text{ColSpan} \left(\begin{bmatrix} \mathbf{A}_1^{[j:\ell]} & \mathbf{A}_1^{[\ell+2:n-r]} \end{bmatrix} \right)^\perp$.

2. Continue by sampling a new random matrix $\mathbf{D}_2 \in \mathbb{Z}_2^{(n-r-\ell-1) \times \ell}$ and random full-rank matrix $\mathbf{E}_2 \in \mathbb{Z}_2^{(n-r-\ell-1) \times (n-r-\ell-1)}$. Compute the matrix

$$\mathbf{A}_2 := \begin{bmatrix} \mathbf{A}_1^{[1:\ell]} & \mathbf{A}_1^{[\ell+2:n-r]} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{D}_2 & \mathbf{E}_2 \end{bmatrix}$$

and subspaces

$$T_j := \text{ColSpan} \left(\begin{bmatrix} \mathbf{A}_2^{[j:\ell]} & \mathbf{A}_2^{[\ell+s:n-r-1]} \end{bmatrix} \right)^\perp.$$

Output $(T_1, \dots, T_{\ell+1})$.

After step 1, we compute the matrix

$$\mathbf{A}_1 = \mathbf{A} \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{D}_1 & \mathbf{E}_1 \end{bmatrix} = \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{B} & \mathbf{C} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{D}_1 & \mathbf{E}_1 \end{bmatrix} = \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{B} + \mathbf{C} \cdot \mathbf{D}_1 & \mathbf{C} \cdot \mathbf{E}_1 \end{bmatrix}.$$

Thus, $S'_j = \text{ColSpan} \left(\tilde{\mathbf{A}}_1^{[j:(n-1)-r]} \right)^\perp$ for

$$\tilde{\mathbf{A}}_1 = \begin{bmatrix} \mathbf{A}_1^{[j:\ell]} & \mathbf{A}_1^{[\ell+2:n-r]} \end{bmatrix} = \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{B} + \mathbf{C} \cdot \mathbf{D}_1 & (\mathbf{C} \cdot \mathbf{E}_1)^{[2:n-r-\ell]} \end{bmatrix}.$$

Since $\mathbf{C}$ and $\mathbf{E}_1$ have full column-rank, so does $\mathbf{C} \cdot \mathbf{E}_1$ and therefore $(\mathbf{C} \cdot \mathbf{E}_1)^{[2:n-r-\ell]}$. By our inductive hypothesis, as $r + (s-1) + \ell \leq n-1$, we have that the distribution of $(T_1, \dots, T_{\ell+1})$ can be equivalently sampled as follows:

1. Sample a random vector $\mathbf{v}_1$ such that $\mathbf{v}_1 \notin S_{\ell+1}$. Compute $S'_j := \text{span}\{S_j, \mathbf{v}_1\}$ for all j .
2. Sample $s-1$ random linearly independent vectors $\mathbf{v}_2, \dots, \mathbf{v}_s$ such that $\text{span}\{\mathbf{v}_2, \dots, \mathbf{v}_s\} \cap S'_{\ell+1} = \{0\}$. Compute $T_j := \text{span}\{S'_j, \mathbf{v}_2, \dots, \mathbf{v}_s\}$ for all j . Output $(T_1, \dots, T_{\ell+1})$.

It is easy to see that this is statistically identical to sampling $\mathbf{v}_1, \dots, \mathbf{v}_s$ simultaneously, as in Distribution 1. What remains is to show that the distribution produced by the recursive process is also statistically equivalent to Distribution 2.

Let us compute the matrix $\mathbf{A}_2$:

$$\begin{aligned} \mathbf{A}_2 &= \begin{bmatrix} \mathbf{A}_1^{[1:\ell]} & \mathbf{A}_1^{[\ell+2:n-r]} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{D}_2 & \mathbf{E}_2 \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{B} + \mathbf{C} \cdot \mathbf{D}_1 & (\mathbf{C} \cdot \mathbf{E}_1)^{[2:n-r-\ell]} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{D}_2 & \mathbf{E}_2 \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{B} + \mathbf{C} \cdot \mathbf{D}_1 + (\mathbf{C} \cdot \mathbf{E}_1)^{[2:n-r-\ell]} \cdot \mathbf{D}_2 & (\mathbf{C} \cdot \mathbf{E}_1)^{[2:n-r-\ell]} \cdot \mathbf{E}_2 \end{bmatrix}. \end{aligned}$$

Observe that $\mathbf{A}_2$ is closely related to the following matrix:

$$\begin{aligned} \tilde{\mathbf{A}}_2 &:= \mathbf{A} \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{D}_1 & \mathbf{E}_1 \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{1} & \mathbf{0} \\ \mathbf{D}_2 & \mathbf{0} & \mathbf{E}_2 \end{bmatrix} = \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{B} + \mathbf{C} \cdot \mathbf{D}_1 & \mathbf{C} \cdot \mathbf{E}_1 \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{1} & \mathbf{0} \\ \mathbf{D}_2 & \mathbf{0} & \mathbf{E}_2 \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} & \mathbf{0} \\ \mathbf{B} + \mathbf{C} \cdot \mathbf{D}_1 + (\mathbf{C} \cdot \mathbf{E}_1)^{[2:n-r-\ell]} \cdot \mathbf{D}_2 & (\mathbf{C} \cdot \mathbf{E}_1)^{[1]} & (\mathbf{C} \cdot \mathbf{E}_1)^{[2:n-r-\ell]} \cdot \mathbf{E}_2 \end{bmatrix}. \end{aligned}$$

Thus, $[\tilde{\mathbf{A}}_2^{[j:\ell]} \quad \tilde{\mathbf{A}}_2^{[\ell+s+1:n-r]}] = [\mathbf{A}_2^{[j:\ell]} \quad \mathbf{A}_2^{[\ell+s:n-r-1]}]$. So we can equivalently sample $\mathbf{D}_1, \mathbf{D}_2, \mathbf{E}_1, \mathbf{E}_2$ together and compute $\tilde{\mathbf{A}}_2$ accordingly before defining $T_j := \text{ColSpan}([\tilde{\mathbf{A}}_2^{[j:\ell]} \quad \tilde{\mathbf{A}}_2^{[\ell+s+1:n-r]}])^\perp$.

If we now compute the following matrix:

$$\mathbf{F} := \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{D}_1 & \mathbf{E}_1 \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{1} & \mathbf{0} \\ \mathbf{D}_2 & \mathbf{0} & \mathbf{E}_2 \end{bmatrix} = \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{D}_1 + \mathbf{E}_1^{[2:n-r-\ell]} \cdot \mathbf{D}_2 & \mathbf{E}_1 \cdot \begin{bmatrix} \mathbf{1} & \mathbf{0} \\ \mathbf{0} & \mathbf{E}_2 \end{bmatrix} \end{bmatrix},$$

we observe that (1) $\mathbf{E}_2$ is full-rank and $\mathbf{E}_1$ is a random full-rank matrix and (2) for each fixed $\mathbf{E}_1$ and $\mathbf{D}_2$, $\mathbf{D}_1$ is a random matrix. Thus, $\mathbf{F}$ has the same distribution as $\begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{M}' & \mathbf{M} \end{bmatrix}$ in Distribution 2, and since $\tilde{\mathbf{A}}_2 = \mathbf{A} \cdot \mathbf{F}$ and $T_j := \text{ColSpan}([\tilde{\mathbf{A}}_2^{[j:\ell]} \quad \tilde{\mathbf{A}}_2^{[\ell+s+1:n-r]}])^\perp$, this distribution is equivalent to Distribution 2. $\square$

We will also use variations of the information-theoretical and cryptographic subspace-hiding lemmas of [SZ25], stating proofs for completeness; many of these proofs will closely track the proof outlines of [SZ25].

Lemma D.3 (Adapted from [SZ25]). *Let $n, r, s, \ell \in \mathbb{N}^+$ such that $r + s + \ell \leq n$, and let $S_1 \subseteq \dots \subseteq S_{\ell+1} \subseteq \mathbb{Z}_2^n$ be subspaces where S_j has dimension $r + j - 1$. Let S_s be the distribution over subspaces T_j defined as follows: pick s random linearly independent vectors $v_1, \dots, v_s \in \mathbb{Z}_2^n$ such that $\text{span}\{v_1, \dots, v_s\} \cap S_{\ell+1} = \{0\}$ and let $T_j := \text{span}\{S_j, v_1, \dots, v_s\}$ for all j . For any subspace $S' \subseteq \mathbb{Z}_2^n$ let $\mathcal{O}_{S'}$ be the oracle that checks membership in S' (outputs 1 if and only if the input is inside S').*

Then, for every oracle-aided quantum algorithm $\mathcal{A}$ making at most q quantum queries, we have the following indistinguishability over oracle distributions:

$$\{\mathcal{O}_{S_1}, \dots, \mathcal{O}_{S_{\ell+1}}\} \approx_{O\left(\frac{q \cdot \ell \cdot s}{\sqrt{2^{n-r-s-\ell}}}\right)} \{\mathcal{O}_{T_1}, \dots, \mathcal{O}_{T_{\ell+1}} : (T_1, \dots, T_{\ell+1}) \leftarrow S_s\}.$$

Proof. We prove the claim by a hybrid argument, increasing the dimension of the random super-spaces T_j one at a time.

First, note that since $S_1 \subseteq \dots \subseteq S_{\ell+1}$, there is some full-column-rank matrix $\mathbf{A} \in \mathbb{Z}_2^{n \times (n-r)}$ such that $\mathcal{O}_{S_j}$ can be described as accepting $\mathbf{x} \in \mathbb{Z}_2^n$ if and only if $\mathbf{x}^T \cdot \mathbf{A}^{[j:n-r]} = 0^{n-r-j+1}$; fix any such matrix $\mathbf{A}$.

We now sample a random matrix $\mathbf{B} \in \mathbb{Z}_2^{(n-r-\ell) \times \ell}$ and compute $\mathbf{C} := \mathbf{A} \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{B} & \mathbf{I}_{n-r-\ell} \end{bmatrix}$. We switch to the oracles $\mathcal{O}'_{S_j}$ which accept $\mathbf{x} \in \mathbb{Z}_2^n$ if and only if $\mathbf{x}^T \cdot \mathbf{C}^{[j:n-r]} = 0^{n-r-j+1}$. As shown in the proof of Lemma D.1, $(\mathcal{O}_{S_1}, \dots, \mathcal{O}_{S_{\ell+1}})$ and $(\mathcal{O}'_{S_1}, \dots, \mathcal{O}'_{S_{\ell+1}})$ are functionally identical for all $\mathbf{B}$, so this change is indistinguishable. We can also indistinguishably switch to the oracles $\mathcal{O}''_{S_j}$ which accept $\mathbf{x} \in \mathbb{Z}_2^n$ if and only if $\mathbf{x}^T \cdot \mathbf{C}^{[\ell+1:n-r]} = 0^{n-r-\ell}$ and $\mathbf{x}^T \cdot \mathbf{C}^{[j:\ell]} = 0^{\ell-j+1}$.

Next, we additionally sample a uniformly random vector $\mathbf{z} \in \mathbb{Z}_2^{n-r-\ell} \setminus \{0\}$. Consider the oracles $\mathcal{O}_{S'_j}$ which accept $\mathbf{x} \in \mathbb{Z}_2^n$ if and only if (1) $\mathbf{x}^T \cdot \mathbf{A}^{[\ell+1:n-r]} = 0^{n-r-\ell}$ or $\mathbf{x}^T \cdot \mathbf{A}^{[\ell+1:n-r]} = \mathbf{z}$ and (2) $\mathbf{x}^T \cdot \mathbf{A}^{[j:\ell]} = 0^{\ell-j+1}$. By a simulation argument, it is easy to see that we are essentially giving $\ell + 1$ copies of an oracle which in one case accepts $\mathbf{x}$ if $\mathbf{x}^T \cdot \mathbf{A}^{[\ell+1:n-r]} = 0^{n-r-\ell}$ and in the other accepts if $\mathbf{x}^T \cdot \mathbf{A}^{[\ell+1:n-r]} = 0^{n-r-\ell}$ or $\mathbf{x}^T \cdot \mathbf{A}^{[\ell+1:n-r]} = \mathbf{z}$. By the randomness of $\mathbf{z}$ and a standard quantum lower bound, we have that

$$\{\mathcal{O}''_{S_1}, \dots, \mathcal{O}''_{S_{\ell+1}}\} \approx_{O\left(\frac{q \cdot \ell}{\sqrt{2^{n-r-\ell}}}\right)} \{\mathcal{O}_{S'_1}, \dots, \mathcal{O}_{S'_{\ell+1}}\}.$$

Finally, we move from $\mathcal{O}_{S'_j}$ to the functionally equivalent oracles $\mathcal{O}'_{S'_j}$ which accept $\mathbf{x} \in \mathbb{Z}_2^n$ if and only if $\mathbf{x}^T \cdot \mathbf{A}^{[j:n-r]} = 0^{n-r-j+1}$ or $\mathbf{x}^T \cdot \mathbf{A}^{[j:n-r]} = 0^{\ell-j+1} \|\mathbf{z}\|$; this change is indistinguishable.

By Lemma D.1, $\mathcal{O}'_{S'_j}$ are precisely membership oracles for the distribution of superspaces produced by adding a random vector outside $S_{\ell+1}$ to all S_j 's, thereby proving the claim for $s = 1$. For arbitrary s , iterating the argument s times in total gives a distinguishing advantage of

$$\sum_{j=1}^s O\left(\frac{q \cdot \ell}{\sqrt{2^{n-r-\ell-(j-1)}}}\right) = O\left(\frac{q \cdot \ell \cdot s}{\sqrt{2^{n-r-s-\ell}}}\right),$$

as desired. $\square$

A straightforward combination of a simulation argument and a hybrid argument gives the following corollary of Lemma D.3.

Corollary D.4. *Let $n, r, s, \ell \in \mathbb{N}^+$ such that $r + s + \ell \leq n$, and let $S_1 \subseteq \dots \subseteq S_{\ell+1} \subseteq \mathbb{Z}_2^n$ as in Lemma D.3. Define the distribution S_s and membership oracles $\mathcal{O}_{S'}$ for subspaces S' similarly.*

Then, for every oracle-aided quantum algorithm $\mathcal{A}$ making at most q quantum queries, we have the following indistinguishability over oracle distributions:

$$\begin{aligned} & \{(\mathcal{O}_{T_{1,1}}, \dots, \mathcal{O}_{T_{1,\ell+1}}), \dots, (\mathcal{O}_{T_{m,1}}, \dots, \mathcal{O}_{T_{m,\ell+1}}) : \forall i \in [m], (T_{i,1}, \dots, T_{i,\ell+1}) \leftarrow S_s\} \\ & \approx_{O\left(\frac{q \cdot \ell \cdot m \cdot s}{\sqrt{2^{n-r-s-\ell}}}\right)} \{(\mathcal{O}_{T_1}^1, \dots, \mathcal{O}_{T_{\ell+1}}^1), \dots, (\mathcal{O}_{T_1}^m, \dots, \mathcal{O}_{T_{\ell+1}}^m) : (T_1, \dots, T_{\ell+1}) \leftarrow S_s\}. \end{aligned}$$

Using Corollary D.4, we can now show the following subspace anti-concentration result.

Lemma D.5. *Let $n, r, s, \ell \in \mathbb{N}^+$ such that $r + s + \ell \leq n$, and let $S_1 \subseteq \dots \subseteq S_{\ell+1} \subseteq \mathbb{Z}_2^n$ as in Lemma D.3. Define the distribution S_s and membership oracles $\mathcal{O}_{S'}$ for subspaces S' similarly.*

Assume there is an oracle-aided quantum algorithm $\mathcal{A}$ making at most q quantum queries and outputting a vector $u \in \mathbb{Z}_2^n$ at the end of its execution, such that

$$\Pr[\mathcal{A}^{\mathcal{O}_{T_1}, \dots, \mathcal{O}_{T_{\ell+1}}} \in (T_1^\perp \setminus \{0\}) : (T_1, \dots, T_{\ell+1}) \leftarrow S_s] \geq \varepsilon.$$

Denote $t := n - r - s, m := \frac{n(t+1)}{\varepsilon}$ and assume (1) $\frac{t}{\varepsilon} \cdot (2^{t-s} + 2^{-t+\ell}) \leq o(1)$ and (2) $\frac{q \cdot \ell \cdot m^2 \cdot s}{2^{t/2}} \leq o(1)$. Then,

$$\Pr[\mathcal{A}^{\mathcal{O}_{T_1}, \dots, \mathcal{O}_{T_{\ell+1}}} \in (S_1^\perp \setminus T_1^\perp) : (T_1, \dots, T_{\ell+1}) \leftarrow S_s] \geq \frac{\varepsilon}{16n \cdot (t+1)}.$$

Proof. Our proof follows [SZ25] closely, but with a few modifications. We first construct a reduction $\mathcal{B}$ that uses $\mathcal{A}$ as a subroutine; $\mathcal{B}$ has access to $m := n(t+1)/\varepsilon$ samples of oracles $(\mathcal{O}^{(1,1)}, \dots, \mathcal{O}^{(1,\ell+1)}), \dots, (\mathcal{O}^{(m,1)}, \dots, \mathcal{O}^{(m,\ell+1)})$, for $t := n - r - s$. $\mathcal{B}$ executes $\mathcal{A}^{\mathcal{O}^{(i,1)}, \dots, \mathcal{O}^{(i,\ell+1)}}$ for every $i \in [m]$ and obtains m vectors $\{u_1, \dots, u_m\}$, before taking only the vectors $\{v_1, \dots, v_j\}$ that are inside $S_1^\perp$. Finally, it computes $D := \dim(\text{span}\{v_1, \dots, v_j\})$; note that $\mathcal{B}$ only makes $q \cdot m$ queries.

Consider the following oracle distribution $\mathcal{D}_1$: sample $(T_{i,1}, \dots, T_{i,\ell+1}) \leftarrow S_s$ i.i.d. m times and give $\mathcal{B}$ access to all of the associated membership oracles $(\mathcal{O}_{T_{i,1}}, \dots, \mathcal{O}_{T_{i,\ell+1}})$. Our goal is to show that $D \geq t+1$ with high probability over $\mathcal{D}_1$.

For every $i \in [m]$ we define the probability p_i as follows. For indices $i \in [n/\varepsilon]$ in the first bucket, p_i is the probability that given access to $\mathcal{O}_{T_{i,1}}, \dots, \mathcal{O}_{T_{i,\ell+1}}$, the output of $\mathcal{A}$ is a vector $u_i \in T_{i,1}^\perp \setminus \{0\} \subseteq S_1^\perp$, in which case we call the i 'th execution *successful*. We denote by $T_{(1,1)}, \dots, T_{(1,\ell+1)}$ the first set of subspaces in the first bucket where a successful execution happens (and set $(T_{(1,1)}, \dots, T_{(1,\ell+1)}) := (\perp, \dots, \perp)$ if no execution in the bucket is successful). For any i inside buckets $2 \leq b \leq t+1$, let p_i be the probability that (1) $u_i \in T_{i,1}^\perp \setminus \{0\}$ and (2) the intersection between $T_{i,1}^\perp$ and each of $T_{(1,1)}^\perp, \dots, T_{(b-1,1)}^\perp$ contains only $\{0\}$. Similarly to the first bucket, we denote by $T_{(b,1)}, \dots, T_{(b,\ell+1)}$ the first set of subspaces in bucket b with a successful execution.

Observe that a successful execution in all buckets immediately implies that $\mathcal{B}$ finds at least $t + 1$ linearly independent vectors which are in $S_1^\perp$ and thus that $D \geq t + 1$.

Consider the probability p' defined as follows. Let $T_{1,1}, \dots, T_{t,1}$ be any t subspaces, each of dimension $r + s$, and hence their duals $T_{1,1}^\perp, \dots, T_{t,1}^\perp$ have dimension t . $p'_{T_{1,1}, \dots, T_{t,1}}$ is the probability that (1) when sampling $T_1^\perp$, the intersection of $T_1^\perp$ with each of the t dual subspaces $T_{1,1}^\perp, \dots, T_{t,1}^\perp$ is only the zero vector and (2) the output of $\mathcal{A}$ is inside $T_1^\perp \setminus \{0\}$. p' is defined to be the minimal probability taken over all possible choices of t subspaces $T_{1,1}, \dots, T_{t,1}$. It is easy to see that $p' \leq p_i$ for all i and thus it suffices to give a lower bound on p' .

We first show that the probability for a nontrivial intersection is small. Fix any index id_x and consider the probability that $T_1^\perp$ intersects with $T_{\text{id}_x,1}^\perp$. How is $T_1^\perp$ (and in particular T_1) sampled? A random set of linearly independent vectors $v_1, \dots, v_s$ is chosen such that $\text{span}\{v_1, \dots, v_s\} \cap S_{\ell+1} = \{0\}$, and T_1 is computed as $\text{span}\{S_1, v_1, \dots, v_s\}$. Of course, one alternative way to think of this process is to first pick random linearly independent vectors $v_1, \dots, v_s$ such that $\text{span}\{v_1, \dots, v_s\} \cap S_1 = \{0\}$, compute $T_1 := \text{span}\{S_1, v_1, \dots, v_s\}$, and resample if $\text{span}\{v_1, \dots, v_s\} \cap S_{\ell+1} \neq \{0\}$. In other words, if we let A denote the event that a random dimension $(r + s)$ -superspace T'_1 of S_1 can be written as $T'_1 = \text{span}\{S_1, v'_1, \dots, v'_s\}$ for $v'_1, \dots, v'_s \in \mathbb{Z}_2^n$ such that $\text{span}\{v'_1, \dots, v'_s\} \cap S_{\ell+1} = \{0\}$, and B the event that a random superspace T'_1 of S_1 satisfies $(T'_1)^\perp \cap T_{\text{id}_x,1}^\perp = \{0\}$, then we want a lower bound on $\Pr[B \mid A]$.

We begin by bounding $\Pr[\neg A]$. We can think of the process of picking a random superspace of S_1 as a s -step process of picking vectors $\mathbf{v}_z \in \mathbb{Z}_2^n$ for $z = 1$ to s , where each time, we select $\mathbf{v}_z$ such that $\mathbf{v}_z \notin \text{span}\{S_1, \mathbf{v}_1, \dots, \mathbf{v}_{z-1}\}$. We observe that the probability that A does not occur is the sum over $z \in [s]$ of the following event: the z 'th step is the first step such that A becomes false. This is at most the sum over $z \in [s]$ of the probability that in the z 'th step, (1) $\mathbf{v}_z \in \text{span}\{S_{\ell+1}, \mathbf{v}_1, \dots, \mathbf{v}_{z-1}\}$, conditioned on (2) $\mathbf{v}_k \notin \text{span}\{S_{\ell+1}, \mathbf{v}_1, \dots, \mathbf{v}_{k-1}\}$ for all $k < z$ and (3) $\mathbf{v}_z \notin \text{span}\{S_1, \mathbf{v}_1, \dots, \mathbf{v}_{z-1}\}$. Observe that given condition (2), there are only $2^n - 2^{r+z-1}$ vectors which also satisfy condition (3). On the other hand, given condition (2), only $2^{r+\ell+z-1} - 2^{r+z-1}$ vectors also satisfy conditions (1) and (3). Thus, the probability for a given z is at most

$$\frac{2^{r+\ell+z-1} - 2^{r+z-1}}{2^n - 2^{r+z-1}} = \frac{2^\ell - 1}{2^{n-r-z+1} - 1} < \frac{2^\ell}{2^{n-r-z+1}} = 2^{-(n-\ell-r-z+1)}.$$

We therefore conclude that $\Pr[\neg A]$ is at most

$$\sum_{z=1}^s 2^{-(n-\ell-r-z+1)} < 2^{-(n-\ell-r-s)} = 2^{-t+\ell}.$$

Now, let us bound $\Pr[\neg B]$. Recall that if T'_1 is a random superspace of S_1 , then $(T'_1)^\perp$ is a random dimension t subspace of $S_1^\perp$, which has dimension $n - r$. On the other hand, $T_{\text{id}_x,1}^\perp$ is a fixed dimension t subspace of $S_1^\perp$. We want to determine the probability that $(T'_1)^\perp \cap T_{\text{id}_x,1}^\perp = \{0\}$. Like before, we can now think of the process of sampling a basis for $(T'_1)^\perp$ as a t -step process, where in the z 'th step, a vector $\mathbf{v}_z \in S_1^\perp$ is chosen such that $\mathbf{v}_z \notin \text{span}\{\mathbf{v}_1, \dots, \mathbf{v}_{z-1}\}$. We again bound the sum over $z \in [t]$ of the following event: conditioned on (1) $\mathbf{v}_k \notin \text{span}\{T_{\text{id}_x,1}^\perp, \mathbf{v}_1, \dots, \mathbf{v}_{k-1}\}$ for all $k < z$ and (2) $\mathbf{v}_z \notin \text{span}\{\mathbf{v}_1, \dots, \mathbf{v}_{z-1}\}$, (3) $\mathbf{v}_z \in \text{span}\{T_{\text{id}_x,1}^\perp, \mathbf{v}_1, \dots, \mathbf{v}_{z-1}\}$. Observe that condition (1) obviously implies that $\mathbf{v}_1, \dots, \mathbf{v}_{z-1}$ are linearly independent and thus given condition (1), there are precisely $2^{n-r} - 2^{z-1}$ vectors in $S_1^\perp$ which satisfy condition (2). On the other hand, given condition (1), there are only $2^{t+z-1} - 2^{z-1}$ vectors which satisfy both conditions (2) and (3). Thus, the probability for a given z is at most

$$\frac{2^{t+z-1} - 2^{z-1}}{2^{n-r} - 2^{z-1}} = \frac{2^t - 1}{2^{n-r-z+1} - 1} < \frac{2^t}{2^{n-r-z+1}} = 2^{-(n-r-t-z+1)}.$$

We therefore conclude that $\Pr[\neg B]$ is at most

$$\sum_{z=1}^t 2^{-(n-r-t-z+1)} < 2^{t-s}.$$

Thus, we have that

$$\Pr[B \mid A] \geq \Pr[B] - \Pr[\neg A] = 1 - \Pr[\neg B] - \Pr[\neg A] \geq 1 - 2^{t-s} - 2^{-t+\ell}.$$

By a union bound, this means the probability that there exists idx such that $T_1^\perp$ has a nontrivial intersection with $T_{\text{idx},1}^\perp$ is at most $t(2^{t-s} + 2^{-t+\ell})$.

If we now let C denote the event that $\mathcal{A}$ outputs a vector in $T_1^\perp \setminus \{0\}$ and D the event that $T_1^\perp \cap T_{\text{idx},1}^\perp = \{0\}$ for all $\text{idx} \in [t]$, we have that

$$\Pr[C \mid D] \geq \Pr[C] - \Pr[\neg D] \geq \varepsilon - t(2^{t-s} + 2^{-t+\ell}).$$

As $\frac{t}{\varepsilon}(2^{t-s} + 2^{-t+\ell}) \leq o(1)$ by assumption,

$$\begin{aligned} p' &= \Pr[C \wedge D] = \Pr[C \mid D] \cdot \Pr[D] \\ &\geq (\varepsilon - t(2^{t-s} + 2^{-t+\ell})) \cdot (1 - t(2^{t-s} + 2^{-t+\ell})) \\ &\geq \varepsilon/4. \end{aligned}$$

Since every bucket has n/ε indices, each of which succeeds with probability at least $\varepsilon/4$, the overall success probability in a bucket is at least $1 - e^{-\Omega(n)}$. By a union bound, $D \geq t + 1$ occurs with probability at least $1 - (t + 1)e^{-\Omega(n)} = 1 - e^{-\Omega(n)}$.

We now consider a different oracle distribution $\mathcal{D}_2$: sample $(T_1, \dots, T_{\ell+1}) \leftarrow \mathcal{S}_s$ once, then allow an m -oracle access to it. Note that $\mathcal{B}$ is a $(q \cdot m)$ -query algorithm and thus by Corollary D.4, we have that

$$\mathcal{D}_1 \approx_{O\left(\frac{q \cdot \ell \cdot m^2 \cdot s}{\sqrt{2^{n-r-s-\ell}}}\right)} \mathcal{D}_2.$$

Thus, when we execute $\mathcal{B}$ on a sample from $\mathcal{D}_2$, with probability at least

$$1 - e^{-\Omega(n)} - O\left(\frac{q \cdot \ell \cdot m^2 \cdot s}{\sqrt{2^{n-r-s-\ell}}}\right) \geq 1 - O\left(\frac{q \cdot \ell \cdot m^2 \cdot s}{\sqrt{2^{n-r-s-\ell}}}\right),$$

$\mathcal{B}$ will compute $D \geq t + 1$. As we assumed that $\frac{q \cdot \ell \cdot m^2 \cdot s}{\sqrt{2^{n-r-s-\ell}}} = O\left(\frac{q \cdot \ell \cdot m^2 \cdot s}{2^{t/2}}\right) \leq o(1)$, this means that $\mathcal{B}$ computes $D \geq t + 1$ with probability at least $1/2$ over $\mathcal{D}_2$. By a simple averaging argument, this means that with probability at least $1/4$ over the sampling of $T_1, \dots, T_{\ell+1}$, $\mathcal{B}$ computes $D \geq t + 1$ when given access to this set of superspaces with probability at least $1/4$; we refer to such sets of superspaces as “good” sets. But when $D \geq t + 1$, this means that $\mathcal{B}$ has computed at least $t + 1$ linearly independent vectors in $S_1^\perp \setminus \{0\}$ despite the fact that $T_1^\perp$ has dimension t , and thus $\mathcal{B}$ produces at least one vector in $S_1^\perp \setminus T_1^\perp$. Since $\mathcal{B}$ uses m i.i.d executions of $\mathcal{A}$, for every $(T_1, \dots, T_{\ell+1})$ which is “good”, when we prepare an oracle access to $(T_1, \dots, T_{\ell+1})$ and execute $\mathcal{A}$, we will get $\mathcal{A}^{\mathcal{O}_{T_1}, \dots, \mathcal{O}_{T_{\ell+1}}} \in S_1^\perp \setminus T_1^\perp$ with probability at least $\frac{1}{4m}$.

Thus, over random $(T_1, \dots, T_{\ell+1})$, we have that

$$\Pr[\mathcal{A}^{\mathcal{O}_{T_1}, \dots, \mathcal{O}_{T_{\ell+1}}} \in S_1^\perp \setminus T_1^\perp : (T_1, \dots, T_{\ell+1}) \leftarrow \mathcal{S}_s] \geq \frac{1}{4} \cdot \frac{1}{4m} = \frac{\varepsilon}{16 \cdot n \cdot (t + 1)},$$

as desired. $\square$

We now consider a cryptographic version of Lemma D.3.

Lemma D.6. *Let $n, r, s, \ell \in \mathbb{N}^+$ such that $r + s + \ell \leq n$, and let $S_1 \subseteq \dots \subseteq S_{\ell+1} \subseteq \mathbb{Z}_2^n$ be subspaces where S_j has dimension $r + j - 1$. Let $\mathcal{S}_s$ be the distribution over subspaces T_j defined as follows: pick s random linearly independent vectors $v_1, \dots, v_s \in \mathbb{Z}_2^n$ such that $\text{span}\{v_1, \dots, v_s\} \cap S_{\ell+1} = \{0\}$ and let $T_j := \text{span}\{S_j, v_1, \dots, v_s\}$ for*

all j . For any subspace $S' \subseteq \mathbb{Z}_2^n$ let $C_{S'}$ some canonical classical circuit that checks membership in S' , say by Gaussian elimination. Let iO be an indistinguishability obfuscation scheme that is $(f(\lambda), \varepsilon(\lambda))$ -secure, and assume that $(f(\lambda), \varepsilon(\lambda))$ -secure injective one-way functions exist, where f and $1/\varepsilon$ are sub-exponential functions.

Then, for every security parameter λ such that $\lambda \leq n - r - s - \ell$ and sufficiently large $p := p(\lambda)$ polynomial in the security parameter, we have the following indistinguishability:

$$\begin{aligned} \{\mathcal{O}_{S_1}, \dots, \mathcal{O}_{S_{\ell+1}} : \mathcal{O}_{S_j} \leftarrow \text{iO}(1^\lambda, 1^p, C_{S_j})\} &\approx_{(f(\lambda) - \ell \cdot \text{poly}(\lambda), \text{poly}(n) \cdot \varepsilon(\lambda))} \\ \{\mathcal{O}_{T_1}, \dots, \mathcal{O}_{T_{\ell+1}} : (T_1, \dots, T_{\ell+1}) \leftarrow S_s, \mathcal{O}_{T_j} \leftarrow \text{iO}(1^\lambda, 1^p, C_{T_j})\}. \end{aligned}$$

Proof. The proof of Lemma D.6 is a simple extension of the proof of Lemma D.3 to the cryptographic setting using the same proof outline as in [Zha21b]. We will use an induction argument, first considering the case where $s = 1$.

Consider the following series of hybrids:

- Hyb_0 : In this hybrid, $\mathcal{A}$ receives $\mathcal{O}_{S_1}, \dots, \mathcal{O}_{S_\ell}$. All programs are appropriately padded before obfuscating so that all the programs received by $\mathcal{A}$ in the following hybrids have the same length.
- Hyb_1 : Since $S_1 \subseteq \dots \subseteq S_{\ell+1}$, we also have that $S_{\ell+1}^\perp \subseteq \dots \subseteq S_1^\perp$. Fix any full column-rank matrix $\mathbf{A} \in \mathbb{Z}_2^{n \times (n-r)}$ such that $S_j^\perp = \text{ColSpan}(\mathbf{A}^{[j:n-r]})$; such a matrix can be computed by Gaussian elimination.

Sample a random matrix $\mathbf{B} \in \mathbb{Z}_2^{(n-r-\ell) \times \ell}$ and compute $\mathbf{C} := \mathbf{A} \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{B} & \mathbf{I}_{n-r-\ell} \end{bmatrix}$. Let $\hat{P}$ be an obfuscation under iO of the program Z that always outputs 0 on inputs in $\mathbb{Z}_2^{n-r-\ell}$. Then $\mathcal{O}_{S_j}$ is the obfuscation under iO of the function

$$Q_j(\mathbf{x}) := \begin{cases} 1 & \text{if } \mathbf{x}^T \cdot \mathbf{C}^{[j:n-r]} = 0^{n-r-j+1} \\ 1 & \text{if } \mathbf{x}^T \cdot \mathbf{C}^{[j:\ell]} = 0^{\ell-j+1} \text{ and } \hat{P}(\mathbf{x}^T \cdot \mathbf{C}^{[\ell+1:n-r]}) = 1 \\ 0 & \text{otherwise.} \end{cases}$$

As shown in the proof of Lemma D.1, the condition $\mathbf{x}^T \cdot \mathbf{C}^{[j:n-r]} = 0^{n-r-j+1}$ is equivalent to checking that $\mathbf{x} \in S_j$ for all matrices $\mathbf{B}$. Since $\hat{P}$ always outputs 0, Q_j is a membership program for S_j and thus is functionally equivalent to C_{S_j} . Thus Hyb_0 and Hyb_1 are $(f(\lambda), (\ell + 1) \cdot \varepsilon(\lambda))$ -computationally indistinguishable by the security of the iO .

- Hyb_2 : Let $\hat{P}$ now be the obfuscation under iO of the function

$$P_y(\mathbf{z}) := \begin{cases} 1 & \text{if } \text{OWF}(\mathbf{z}) = y \\ 0 & \text{otherwise,} \end{cases}$$

where OWF is an injective one-way function and $y = \text{OWF}(\mathbf{z}^*)$ for a random $\mathbf{z}^* \in \mathbb{Z}_2^{n-r-\ell}$. Since $n - r - \ell \geq \lambda$, we can invoke the security of OWF . Specifically, the only point on which Z and P_y differ is $\mathbf{z}^*$, and finding $\mathbf{z}^*$ requires inverting OWF ; thus, iO is also a differing-inputs obfuscator for these circuits [BCP14]. Since the reduction will need to embed either an obfuscation of P_y or Z in $Q_1, \dots, Q_{\ell+1}$ before running $\mathcal{A}$, it has overhead $\ell \cdot \text{poly}(\lambda)$. Therefore, Hyb_1 and Hyb_2 are $(f(\lambda) - \ell \cdot \text{poly}(\lambda), O(\ell \cdot \varepsilon(\lambda)))$ -computationally indistinguishable by the security of OWF and iO . Note that $Q_j(\mathbf{x})$ decides membership in the subspace S'_j of vectors $\mathbf{x}$ such that $\mathbf{x}^T \cdot \mathbf{C}^{[j:n-r]} \in \text{span}\{0^{\ell-j+1} \parallel \mathbf{z}^*\}$.

- Hyb_3 : In this hybrid, a random $\mathbf{B}$ and $\mathbf{z}^*$ are chosen which fixes $S'_1, \dots, S'_{\ell+1}$; the canonical membership programs $C_{S'_1}, \dots, C_{S'_{\ell+1}}$ are then obfuscated. Since all programs in both hybrids still decide membership in $S'_1, \dots, S'_{\ell+1}$, the programs being obfuscated are functionally equivalent. Thus, Hyb_2 and Hyb_3 are $(f(\lambda), (\ell + 1) \cdot \varepsilon(\lambda))$ -computationally indistinguishable by the security of the iO .

- Hyb₄: Now $\mathbf{z}^*$ is chosen at random, but restricted to being nonzero. Since $\lambda \leq n - r - \ell$, Hyb₃ and Hyb₄ are $O\left(\frac{1}{2^\lambda}\right)$ -statistically close. Observe that by Lemma D.1, Hyb₄ now contains precisely the obfuscations of C_{T_j} .

The proof follows by iterating over s and observing that $\lambda \leq n - r - \ell - s$ and $\ell, s \leq n$. $\square$

We remark that the lemma also trivially extends (with the same base security parameter λ) to the setting where we choose stronger parameters $\kappa \geq n - r - s - \ell$, $\text{poly}(\kappa) \geq p(n - r - s - \ell)$ to obfuscate the circuits.

We will also need cryptographic versions of Corollary D.4 and Lemma D.5.

Lemma D.7. *Let $n, r, s, \ell \in \mathbb{N}^+$ such that $r + s + \ell \leq n$, and let $S_1 \subseteq \dots \subseteq S_{\ell+1} \subseteq \mathbb{Z}_2^n$ be the same subspaces as in Lemma D.6. Define the distribution $\mathcal{S}_s$ and membership circuits $C_{S'}$ for subspaces S' similarly. Let iO an indistinguishability obfuscation scheme that is $(f(\lambda), \varepsilon(\lambda))$ -secure, and assume that $(f(\lambda), \varepsilon(\lambda))$ -secure injective one-way functions exist, where f and $1/\varepsilon$ are sub-exponential functions.*

Then, for every security parameter λ such that $\lambda \leq n - r - s - \ell$ and sufficiently large $p := p(\lambda)$ polynomial in the security parameter, we have the following indistinguishability:

$$\begin{aligned} & \{(\mathcal{O}_{S_1}^1, \dots, \mathcal{O}_{S_{\ell+1}}^1), \dots, (\mathcal{O}_{S_1}^m, \dots, \mathcal{O}_{S_{\ell+1}}^m) : \forall i \in [m], \mathcal{O}_{S_j}^i \leftarrow \text{iO}(1^\lambda, 1^p, C_{S_j})\} \\ & \quad \approx_{(f(\lambda) - m \cdot n \cdot \text{poly}(\lambda), m \cdot \text{poly}(n) \cdot \varepsilon(\lambda))} \\ & \{(\mathcal{O}_{T_1}^1, \dots, \mathcal{O}_{T_{\ell+1}}^1), \dots, (\mathcal{O}_{T_1}^m, \dots, \mathcal{O}_{T_{\ell+1}}^m) : (T_1, \dots, T_{\ell+1}) \leftarrow \mathcal{S}_s, \\ & \quad \forall i \in [m], \mathcal{O}_{T_j}^i \leftarrow \text{iO}(1^\lambda, 1^p, C_{T_j})\}. \end{aligned}$$

Proof. The proof is nearly identical to [SZ25], but we include one here for completeness.

As long as p is sufficiently large, then it is indistinguishable to tell whether a given circuit is obfuscated under one or two layers of obfuscation. Thus, for a sufficiently large parameter p , the distribution

$$\{(\mathcal{O}_{S_1}^1, \dots, \mathcal{O}_{S_{\ell+1}}^1), \dots, (\mathcal{O}_{S_1}^m, \dots, \mathcal{O}_{S_{\ell+1}}^m) : \forall i \in [m], \mathcal{O}_{S_j}^i \leftarrow \text{iO}(1^\lambda, 1^p, C_{S_j})\},$$

is $(f(\lambda), O(\ell \cdot m) \cdot \varepsilon(\lambda))$ -computationally indistinguishable from a distribution where C_{S_j} are swapped with obfuscations of C_{S_j} (denote this modified distribution by $\mathcal{D}_S$). Similarly, the distribution

$$\begin{aligned} & \{(\mathcal{O}_{T_1}^1, \dots, \mathcal{O}_{T_{\ell+1}}^1), \dots, (\mathcal{O}_{T_1}^m, \dots, \mathcal{O}_{T_{\ell+1}}^m) : (T_1, \dots, T_{\ell+1}) \leftarrow \mathcal{S}_s, \\ & \quad \forall i \in [m], \mathcal{O}_{T_j}^i \leftarrow \text{iO}(1^\lambda, 1^p, C_{T_j})\}, \end{aligned}$$

is $(f(\lambda), O(\ell \cdot m) \cdot \varepsilon(\lambda))$ -computationally indistinguishable from a distribution where C_{T_j} are swapped with obfuscations of C_{T_j} (denote this modified distribution by $\mathcal{D}_T$).

The indistinguishability between $\mathcal{D}_S$ and $\mathcal{D}_T$ follows from Lemma D.6: consider the reduction $\mathcal{B}$ that gets a sample $(\mathcal{O}_1, \dots, \mathcal{O}_{\ell+1})$ which is either from $\mathcal{D}_0 := \{(\mathcal{O}_{S_1}, \dots, \mathcal{O}_{S_{\ell+1}}) : \mathcal{O}_{S_j} \leftarrow \text{iO}(1^\lambda, 1^p, C_{S_j})\}$ or from $\mathcal{D}_1 := \{(\mathcal{O}_{T_1}, \dots, \mathcal{O}_{T_{\ell+1}}) : (T_1, \dots, T_{\ell+1}) \leftarrow \mathcal{S}_s, \mathcal{O}_{T_j} \leftarrow \text{iO}(1^\lambda, 1^p, C_{T_j})\}$ for random superspaces $(T_1, \dots, T_{\ell+1})$ of $(S_1, \dots, S_{\ell+1})$. $\mathcal{B}$ then generates m i.i.d sets of obfuscations

$$\{(\mathcal{O}_1^{(1)}, \dots, \mathcal{O}_{\ell+1}^{(1)}), \dots, (\mathcal{O}_1^{(m)}, \dots, \mathcal{O}_{\ell+1}^{(m)})\}$$

of $(\mathcal{O}_1, \dots, \mathcal{O}_{\ell+1})$ and executes $\mathcal{A}$ on the $m \cdot (\ell + 1)$ obfuscations. When $(\mathcal{O}_1, \dots, \mathcal{O}_{\ell+1})$ comes from $\mathcal{D}_0$ then the output sample of the reduction comes from the distribution $\mathcal{D}_S$, and when $(\mathcal{O}_1, \dots, \mathcal{O}_{\ell+1})$ comes from $\mathcal{D}_1$ then the output sample of the reduction comes from the distribution $\mathcal{D}_T$. Since the reduction executes in complexity $\ell \cdot m \cdot \text{poly}(\lambda) \leq m \cdot n \cdot \text{poly}(\lambda)$, this means that

$$\mathcal{D}_S \approx_{(f(\lambda) - m \cdot n \cdot \text{poly}(\lambda), \text{poly}(n) \cdot \varepsilon(\lambda))} \mathcal{D}_T.$$

By transitivity of computational indistinguishability, the two distributions in the statement of the lemma are $(f(\lambda) - m \cdot n \cdot \text{poly}(\lambda), m \cdot \text{poly}(n) \cdot \varepsilon(\lambda))$ -computationally indistinguishable. $\square$

Corollary D.8. Let $n, r, s, \ell \in \mathbb{N}^+$ such that $r + s + \ell \leq n$, and let $S_1 \subseteq \dots \subseteq S_{\ell+1} \subseteq \mathbb{Z}_2^n$ be the same subspaces as in Lemma D.6. Define the distribution S_s and membership circuits $C_{S'}$ for subspaces S' similarly. Let iO an indistinguishability obfuscation scheme that is $(f(\lambda), \varepsilon(\lambda))$ -secure, and assume that $(f(\lambda), \varepsilon(\lambda))$ -secure injective one-way functions exist, where f and $1/\varepsilon$ are sub-exponential functions.

Then, for every security parameter λ such that $\lambda \leq n - r - s - \ell$ and sufficiently large $p := p(\lambda)$ polynomial in the security parameter, we have the following indistinguishability:

$$\begin{aligned} & \{(\mathcal{O}_{T_1}^1, \dots, \mathcal{O}_{T_{\ell+1}}^1), \dots, (\mathcal{O}_{T_1}^m, \dots, \mathcal{O}_{T_{\ell+1}}^m) : \\ & (T_1, \dots, T_{\ell+1}) \leftarrow S_s, \forall i \in [m], \mathcal{O}_{T_j}^i \leftarrow \text{iO}(1^\lambda, 1^p, C_{T_j})\} \\ & \approx_{(f(\lambda) - m \cdot n \cdot \text{poly}(\lambda), m \cdot \text{poly}(n) \cdot \varepsilon(\lambda))} \\ & \{(\mathcal{O}_{T_{1,1}}, \dots, \mathcal{O}_{T_{1,\ell+1}}), \dots, (\mathcal{O}_{T_{m,1}}, \dots, \mathcal{O}_{T_{m,\ell+1}}) : \\ & \forall i \in [m], (T_{i,1}, \dots, T_{i,\ell+1}) \leftarrow S_s, \mathcal{O}_{T_{i,j}} \leftarrow \text{iO}(1^\lambda, 1^p, C_{T_{i,j}})\}. \end{aligned}$$

Lemma D.9. Let $n, r, s, \ell \in \mathbb{N}^+$ such that $r + s + \ell \leq n$, and let $S_1 \subseteq \dots \subseteq S_{\ell+1} \subseteq \mathbb{Z}_2^n$ be the same subspaces as in Lemma D.6. Define the distribution S_s and membership circuits $C_{S'}$ for subspaces S' similarly. Let iO an indistinguishability obfuscation scheme that is $(f(\cdot), \frac{1}{f(\cdot)})$ -secure, and assume that $(f(\cdot), \frac{1}{f(\cdot)})$ -secure injective one-way functions exist, where f is a sub-exponential function.

Let λ be any security parameter such that $\lambda \leq n - r - s - \ell$ and $p := p(\lambda)$ a sufficiently large polynomial in the security parameter. Denote by $\mathcal{O}_{\lambda,p,s}$ the distribution over obfuscated circuits that samples $(T_1, \dots, T_{\ell+1}) \leftarrow S_s$ and then $\mathcal{O}_{T_j} \leftarrow \text{iO}(1^\lambda, 1^p, C_{T_j})$.

Assume there is a quantum algorithm $\mathcal{A}$ of complexity $T_{\mathcal{A}}$ such that,

$$\Pr[\mathcal{A}(\mathcal{O}_{T_1}, \dots, \mathcal{O}_{T_{\ell+1}}) \in T_1^\perp \setminus \{0\} : (\mathcal{O}_{T_1}, \dots, \mathcal{O}_{T_{\ell+1}}) \leftarrow \mathcal{O}_{\lambda,p,s}] \geq \varepsilon.$$

Also, denote $t := n - r - s$, $m := \frac{n(t+1)}{\varepsilon}$, and assume (1) $\frac{t}{\varepsilon} \cdot (2^{t-s} + 2^{-t+\ell}) \leq o(1)$ and (2) $\frac{m \cdot \text{poly}(n) \cdot T_{\mathcal{A}}}{f(\lambda)} \leq o(1)$. Then,

$$\Pr[\mathcal{A}(\mathcal{O}_{T_1}, \dots, \mathcal{O}_{T_{\ell+1}}) \in (S_1^\perp \setminus T_1^\perp) : (\mathcal{O}_{T_1}, \dots, \mathcal{O}_{T_{\ell+1}}) \leftarrow \mathcal{O}_{\lambda,p,s}] \geq \frac{\varepsilon}{16 \cdot n \cdot (t+1)}.$$

Proof. The proof of Lemma D.9 is nearly identical to that of Lemma D.5; we will point out where the main differences lie.

As before, we consider a reduction $\mathcal{B}$ which uses $\mathcal{A}$ as a subroutine. $\mathcal{B}$ has access to $m := n(t+1)/\varepsilon$ samples of obfuscations

$$((\mathcal{O}^{(1,1)}, \dots, \mathcal{O}^{(1,\ell+1)}), \dots, (\mathcal{O}^{(m,1)}, \dots, \mathcal{O}^{(m,\ell+1)})),$$

for $t := n - r - s$. $\mathcal{B}$ executes $\mathcal{A}(\mathcal{O}^{(i,1)}, \dots, \mathcal{O}^{(i,\ell+1)})$ for every $i \in [m]$ and obtains m vectors $\{u_1, \dots, u_m\}$, before taking only the vectors $\{v_1, \dots, v_j\}$ that are inside $S_1^\perp$. Finally, it computes $D := \dim(\text{span}\{v_1, \dots, v_j\})$.

The running time of $\mathcal{B}$ is $O(m \cdot T_{\mathcal{A}} + m \cdot n^3)$, where the first term arises from running m executions of $\mathcal{A}$ and the second term comes from running Gaussian elimination to check whether the i 'th vector u_i produced by $\mathcal{A}$ is in $S_1^\perp$ and is outside of the span of the previous vectors (produced by $\mathcal{A}$) that lie in $S_1^\perp$ (recall that S_1 is known to $\mathcal{A}$ and $\mathcal{B}$ and hence checking $S_1^\perp$ also takes at most $O(n^3)$ time).

Consider the following distribution $\mathcal{D}_1$: sample $(T_{i,1}, \dots, T_{i,\ell+1}) \leftarrow S_s$ i.i.d. m times and for each set of superspaces, give $\mathcal{B}$ an obfuscation of all membership programs $((\mathcal{O}_{T_{1,1}}, \dots, \mathcal{O}_{T_{1,\ell+1}}), \dots, (\mathcal{O}_{T_{m,1}}, \dots, \mathcal{O}_{T_{m,\ell+1}}))$. By an identical argument as in Lemma D.5, we have that $D \geq t + 1$ with probability at least $1 - e^{-\Omega(n)}$ assuming $\frac{t}{\varepsilon} \cdot (2^{t-s} + 2^{-t+\ell}) \leq o(1)$.

We now execute $\mathcal{B}$ on a different distribution $\mathcal{D}_2$: sample $(T_1, \dots, T_{\ell+1}) \leftarrow S_s$ once, then sample m i.i.d. obfuscations of the same circuits $(C_{T_1}, \dots, C_{T_{\ell+1}})$, denoted by $((\mathcal{O}_{T_1}^{(1)}, \dots, \mathcal{O}_{T_{\ell+1}}^{(1)}), \dots, (\mathcal{O}_{T_1}^{(m)}, \dots, \mathcal{O}_{T_{\ell+1}}^{(m)}))$. By Corollary D.8,

$$\mathcal{D}_1 \approx_{f(\lambda) - m \cdot n \cdot \text{poly}(\lambda), \frac{m \cdot \text{poly}(n)}{f(\lambda)}} \mathcal{D}_2.$$

By assumption, we have that $\mathcal{B}$ runs in time $\leq f(\lambda) - m \cdot n \cdot \text{poly}(\lambda)$. Thus, whenever we execute $\mathcal{B}$ on a sample from $\mathcal{D}_2$, with probability at least

$$1 - e^{-\Omega(n)} - \frac{m \cdot \text{poly}(n)}{f(\lambda)} \geq 1 - \frac{m \cdot \text{poly}(n)}{f(\lambda)} \geq \frac{1}{2},$$

$\mathcal{B}$ will compute $D \geq t + 1$. By an averaging argument, this means that with probability at least $1/4$ over the sampling of $T_1, \dots, T_{\ell+1}$, $\mathcal{B}$ will compute $D \geq t + 1$ on the appropriate input with probability at least $1/4$; we refer to such sets of superspaces as “good” sets. But when $D \geq t + 1$, this means that $\mathcal{B}$ has at least $t + 1$ linearly independent vectors in $S_1^\perp \setminus \{0\}$; since $T_1^\perp$ has dimension $n - r - s = t$, $\mathcal{B}$ produces at least one vector in $S_1^\perp \setminus T_1^\perp$. As $\mathcal{B}$ uses m i.i.d. executions of $\mathcal{A}$, for every set of superspaces $(T_1, \dots, T_{\ell+1})$ which is “good”, when we prepare obfuscations of T_i , $\mathcal{A}(\mathcal{O}_{T_1}, \dots, \mathcal{O}_{T_{\ell+1}}) \in S_1^\perp \setminus T_1^\perp$ with probability at least $\frac{1}{4m}$.

Thus, over random $(T_1, \dots, T_{\ell+1})$,

$$\begin{aligned} \Pr[\mathcal{A}(\mathcal{O}_{T_1}, \dots, \mathcal{O}_{T_{\ell+1}}) \in S_1^\perp \setminus T_1^\perp : (\mathcal{O}_{T_1}, \dots, \mathcal{O}_{T_{\ell+1}}) \leftarrow \mathcal{O}_{\lambda, p, s}] &\geq \frac{1}{4} \cdot \frac{1}{4m} \\ &= \frac{\varepsilon}{16n \cdot (t + 1)}, \end{aligned}$$

as desired. $\square$

E Missing Proofs from Section 4 and 5

Proof of Lemma 4.4. We consider a series of hybrids, and show that the success probability in each pair of hybrids is statistically close.

- **Hyb₀: The original execution of $\mathcal{A}$.** By assumption, $\mathcal{A}$ succeeds with probability ε .
- **Hyb₁: Using small-range distribution.** Consider the random function F which samples the randomness used to compute the cosets $(\mathbf{A}_y, \mathbf{b}_y)$ for every $y \in \mathbb{Z}_2^r$; these cosets are then used in $\mathcal{P}$, $\mathcal{P}^{-1}$, and $\mathcal{D}$. We now swap F with a small-range version F' , which is sampled as follows: let $R := (300 \cdot q^3) \cdot \frac{2^7 \cdot n^2}{\varepsilon}$. For every $y \in \mathbb{Z}_2^r$, sample $\text{idx}_y \leftarrow [R]$; for every $\text{idx} \in [R]$, sample a coset $(\mathbf{A}_{\text{idx}}, \mathbf{b}_{\text{idx}})$. Define $F'(\mathbf{y}) := (\mathbf{A}_{\text{idx}_y}, \mathbf{b}_{\text{idx}_y})$.

By Theorem A.6 of [AGQY22], every quantum algorithm making at most q queries to F or F' has distinguishing advantage at most $\frac{300 \cdot q^3}{R} < \frac{\varepsilon}{8}$, so the outputs of Hyb₀ and Hyb₁ have statistical distance at most $\frac{\varepsilon}{8}$, and thus $\mathcal{A}$ succeeds in this hybrid succeeds with probability at least $\frac{7\varepsilon}{8}$.

- **Hyb₂: Relaxing the dual oracle.** In Hyb₁, on input (j, y, v) , the oracle $\mathcal{D}$ accepted if and only if $v \in \text{ColSpan}(\mathbf{A}_{\text{idx}_y}^{[j:n-r]})^\perp$. Consider the following modification. For each $\text{idx} \in [R]$, sample a random $(r + s + \ell)$ -dimensional superspace $T_{\text{idx}, \ell+1}^\perp$ of $\text{ColSpan}(\mathbf{A}_{\text{idx}}^{[\ell+1:n-r]})^\perp$ by sampling s random linearly independent vectors $v_1, \dots, v_s$ such that

$$\text{span}\{v_1, \dots, v_s\} \cap \text{ColSpan}(\mathbf{A}_{\text{idx}}^{[\ell+1:n-r]})^\perp = \{0\}$$

and define

$$T_{\text{idx}, k}^\perp := \text{span}\{\text{ColSpan}(\mathbf{A}_{\text{idx}}^{[k:n-r]})^\perp, v_1, \dots, v_s\}$$

for $k \in [\ell]$. Now, on input (j, y, v) , the oracle $\mathcal{D}$ will instead accept if and only if $v \in T_{\text{idx}_y, j}^\perp$.

By Lemma D.3, for each $\text{idx} \in [R]$, changing $\mathcal{D}$ is $O\left(\frac{q \cdot \ell \cdot s}{\sqrt{2^{n-r-s-\ell}}}\right)$ -indistinguishable. Since we use the above indistinguishability R times, we get $R \cdot O\left(\frac{q \cdot \ell \cdot s}{\sqrt{2^{n-r-s-\ell}}}\right)$ -indistinguishability. Thus, if we denote $\mathcal{A}$'s success probability in Hyb_2 by ε_2 , we have that

$$\begin{aligned} \varepsilon_2 &\geq \frac{7\varepsilon}{8} - R \cdot O\left(\frac{q \cdot \ell \cdot s}{\sqrt{2^{n-r-s-\ell}}}\right) \geq \frac{7\varepsilon}{8} - O\left(\frac{q^4 \cdot \ell \cdot s \cdot n^2 \cdot \frac{1}{\varepsilon}}{\sqrt{2^{n-r-s-\ell}}}\right) \\ &\geq \frac{7\varepsilon}{8} - O\left(\frac{q^4 \cdot n^4 \cdot \frac{1}{\varepsilon}}{\sqrt{2^{n-r-s-\ell}}}\right) \geq \frac{3\varepsilon}{4}, \end{aligned}$$

by assumption (2).

- **Hyb₃: Changing the success predicate.** Recall that in Hyb_2 , we say $\mathcal{A}$ is successful if it finds a collision, i.e. it produces a pair $x_0 \neq x_1$ such that $H(x_0) = H(x_1)$. We now say that $\mathcal{A}$ is successful only if $H(x_0) = H(x_1)$ and $\mathbf{u}_0 - \mathbf{u}_1 \notin T_{\text{idx}_y, 1}$. For every $\text{idx} \in [R]$, let $\varepsilon_2^{(\text{idx})}$ be the probability that $\mathcal{A}$ finds $x_0 \neq x_1$ such that $y_0 = y_1 := \mathbf{y}$ and $\text{idx}_y = \text{idx}$; by definition, $\sum_{\text{idx}} \varepsilon_2^{(\text{idx})} = \varepsilon_2$. Observe that in this case, $\mathbf{u}_0 - \mathbf{u}_1 \in \text{ColSpan}(\mathbf{A}_{\text{idx}_y}) \setminus \{0\}$. Let $\delta^{(\text{idx})}$ be the probability that $\mathcal{A}$ finds x_0, x_1 such that $y_0 = y_1 := \mathbf{y}$, $\text{idx}_y = \text{idx}$, and $\mathbf{u}_0 - \mathbf{u}_1 \in T_{\text{idx}_y, 1} \setminus \{0\}$.

Define $\varepsilon_3^{(\text{idx})}$ to be the probability that $\mathcal{A}$ finds x_0, x_1 such that $y_0 = y_1 := \mathbf{y}$, $\text{idx}_y = \text{idx}$, and $\mathbf{u}_0 - \mathbf{u}_1 \notin T_{\text{idx}_y, 1}$. Let L be the subset of indices $\text{idx} \in [R]$ such that $\varepsilon_2^{(\text{idx})} \geq \frac{\varepsilon_2}{2R}$; this means that $\sum_{\text{idx} \in L} \varepsilon_2^{(\text{idx})} \geq \varepsilon_2/2$. Our goal is to show that for every $\text{idx} \in L$, $\varepsilon_3^{(\text{idx})} \geq \frac{\varepsilon_2^{(\text{idx})}}{16n^2}$. By definition, we have that $\varepsilon_3^{(\text{idx})} \geq \varepsilon_2^{(\text{idx})} - \delta^{(\text{idx})}$, so if $\delta^{(\text{idx})} \leq \varepsilon_2^{(\text{idx})}/2$, then we are done. Thus, we assume that

$$\delta^{(\text{idx})} > \frac{\varepsilon_2^{(\text{idx})}}{2} \geq \frac{\varepsilon_2}{4R} \geq \Omega\left(\frac{\varepsilon^2}{q^3 \cdot n^2}\right).$$

Let $t := n - r - s \leq n - 1$ and $m := \frac{n(t+1)}{\delta^{(\text{idx})}} \leq O\left(\frac{q^3 \cdot n^4}{\varepsilon^2}\right)$. By assumptions (1) and (2), we have that

$$\frac{t}{\delta^{(\text{idx})}} \cdot (2^{t-s} + 2^{-t+\ell}) = O\left(\frac{q^3 \cdot n^3}{\varepsilon^2}\right) \cdot (2^{t-s} + 2^{-t+\ell}) \leq o(1),$$

and

$$\frac{q \cdot \ell \cdot m^2 \cdot s}{2^{t/2}} = O\left(\frac{q^7 \cdot n^{10}/\varepsilon^4}{2^{(t-1)/2}}\right) = O\left(\frac{q^7 \cdot n^{10}/\varepsilon^4}{\sqrt{2^{n-r-s-\ell}}}\right) \leq o(1),$$

so the conditions of Lemma D.5 are fulfilled.⁸ Thus, we have that for $\text{idx} \in L$, $\mathcal{A}$ finds x_0, x_1 such that $y_0 = y_1 := \mathbf{y}$ and $\mathbf{u}_0 - \mathbf{u}_1 \in \text{ColSpan}(\mathbf{A}_{\text{idx}_y}) \setminus T_{\text{idx}_y, 1}$, and thus $\mathbf{u}_0 - \mathbf{u}_1 \notin T_{\text{idx}_y, 1}$, with probability at least $\frac{\delta^{(\text{idx})}}{16n(t+1)} \geq \frac{\delta^{(\text{idx})}}{16n^2} \geq \frac{\varepsilon_2^{(\text{idx})}}{32n^2}$.

Therefore, the success probability of $\mathcal{A}$ in this hybrid satisfies

$$\varepsilon_3 = \sum_{\text{idx}} \varepsilon_3^{(\text{idx})} \geq \sum_{\text{idx} \in L} \varepsilon_3^{(\text{idx})} \geq \sum_{\text{idx} \in L} \frac{\varepsilon_2^{(j)}}{32n^2} \geq \frac{\varepsilon_2}{64n^2} \geq \frac{3\varepsilon}{256n^2}.$$

- **Hyb_{3.1}: Using the randomness of Π .** Recall that for every $\text{idx} \in [R]$, the process of sampling $T_{\text{idx}, k}^\perp$ involves sampling s random linearly independent vectors $\mathbf{v}_{\text{idx}, 1}, \dots, \mathbf{v}_{\text{idx}, s}$ and adding them to

⁸Strictly speaking, $\mathcal{A}$ only has access to one oracle which, when restricted to inputs $(k, \cdot, \cdot)$, behaves as the oracle $\mathcal{O}_{T_{\text{idx}_y, k}}$, but this is an even weaker model of access and so Lemma D.5 readily applies.

$S_{\text{idx},k}^\perp$. By Lemma D.2, we can equivalently sample $T_{\text{idx},k}^\perp$ by sampling a random full column-rank matrix $\mathbf{M}_{\text{idx}} \in \mathbb{Z}_2^{(n-r-\ell) \times (n-r-\ell)}$ and a random matrix $\mathbf{M}'_{\text{idx}} \in \mathbb{Z}_2^{(n-r-\ell) \times \ell}$ for each $\text{idx} \in [R]$, computing $\bar{\mathbf{A}}_{\text{idx}} := \mathbf{A}_{\text{idx}} \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{M}'_{\text{idx}} & \mathbf{M}_{\text{idx}} \end{bmatrix}$ and setting

$$T_{\text{idx},k}^\perp = \text{ColSpan} \left(\begin{bmatrix} \bar{\mathbf{A}}^{[k:\ell]} & \bar{\mathbf{A}}^{[\ell+s+1:n-r]} \end{bmatrix} \right)^\perp.$$

Since this new sampling process is statistically equivalent, the change is indistinguishable.

Define the permutation Γ over $\{0,1\}^n$ as follows: for an input $\mathbf{s} \in \{0,1\}^n$, take the left r bits denoted $\mathbf{y} \in \mathbb{Z}_2^r$, compute $\text{idx}_y \in [R]$, and then apply matrix multiplication by $\mathbf{C}_{\text{idx}_y} := \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{M}'_{\text{idx}_y} & \mathbf{M}_{\text{idx}_y} \end{bmatrix}$ to the remaining right $n-r$ bits. Since $\mathbf{C}_{\text{idx}_y}$ is full-rank for all idx_y , Γ is indeed a permutation. We now apply Γ to the output of Π inside the execution of a query to $\mathcal{P}$, and apply Γ^{-1} to the input of Π^{-1} inside the execution of a query to $\mathcal{P}^{-1}$. Since Π is random and Γ is fixed, the composition produces a random permutation as well and thus the success probability in $\text{Hyb}_{3,1}$ is identical.

- **Hyb_{3,2}: Moving Γ into the coset computation.** In $\text{Hyb}_{3,2}$, we stop applying Γ and Γ^{-1} and instead multiply the output of Π by $\mathbf{C}_{\text{idx}}$ in $\mathcal{P}$ and multiply the original input to Π^{-1} in $\mathcal{P}^{-1}$ by $\mathbf{C}_{\text{idx}}^{-1}$. By definition, the oracle distributions in $\text{Hyb}_{3,1}$ and $\text{Hyb}_{3,2}$ are identical, so the success probability of $\mathcal{A}$ in $\text{Hyb}_{3,2}$ is also ε_3 .
- **Hyb₄: Using the randomness of F' .** In Hyb_4 , we stop multiplying by $\mathbf{C}_{\text{idx}}$ and $\mathbf{C}_{\text{idx}}^{-1}$ and directly define $T_{\text{idx},k}$ to be $\text{ColSpan} \left(\begin{bmatrix} \mathbf{A}_{\text{idx}}^{[k:\ell]} & \mathbf{A}_{\text{idx}}^{[\ell+s+1:n-r]} \end{bmatrix} \right)$. Observe that for any fixed choice of Π , the distribution of $\mathbf{A}_{\text{idx}} = \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{K}_{\text{idx}} & \mathbf{L}_{\text{idx}} \end{bmatrix}$ has random matrices $\mathbf{K}_{\text{idx}}$ and random full column-rank matrices $\mathbf{L}_{\text{idx}}$, so we have that

$$\bar{\mathbf{A}}_{\text{idx}} := \mathbf{A}_{\text{idx}} \cdot \mathbf{C}_{\text{idx}} = \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{K}_{\text{idx}} & \mathbf{L}_{\text{idx}} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{M}'_{\text{idx}} & \mathbf{M}_{\text{idx}} \end{bmatrix} = \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{K}_{\text{idx}} + \mathbf{L}_{\text{idx}} \cdot \mathbf{M}'_{\text{idx}} & \mathbf{L}_{\text{idx}} \cdot \mathbf{M}_{\text{idx}} \end{bmatrix},$$

which has the same distribution as $\mathbf{A}_{\text{idx}}$ (as both $\mathbf{L}_{\text{idx}}$ and $\mathbf{M}_{\text{idx}}$ are random full column-rank matrices and $\mathbf{K}_{\text{idx}}$ is a random matrix). Thus, the success probability ε_4 of $\mathcal{A}$ in Hyb_4 is identical to the success probability of $\mathcal{A}$ in $\text{Hyb}_{3,2}$, which is simply ε_3 .

- **Hyb₅: Rewinding small-range distributions.** We undo the process of sampling an R -small range distribution by swapping F' back to a completely random function F . By an identical argument as before, $\mathcal{A}$ succeeds in this hybrid with probability at least

$$\varepsilon_4 - \frac{300q^3}{R} = \varepsilon_3 - \frac{300q^3}{R} \geq \frac{3\varepsilon}{256n^2} - \frac{300q^3}{R} = \frac{3\varepsilon}{256n^2} - \frac{\varepsilon}{128n^2} = \frac{\varepsilon}{256n^2}.$$

We then observe that the success probability of $\mathcal{A}$ in Hyb_5 is exactly the probability we want to bound, which concludes the proof. $\square$

Proof of Lemma 4.5. Given oracle access to $\bar{\mathcal{P}}$ and $\bar{\mathcal{P}}^{-1}$ which come from $(\bar{\mathcal{P}}, \bar{\mathcal{P}}^{-1}, \bar{\mathcal{D}})$, $\mathcal{B}$ acts as follows:

- Sample a random function F' that outputs sufficient (polynomial in r) randomness and a random n -bit permutation Γ . On input $\mathbf{y} \in \{0,1\}^r$, use the randomness of $F'(\mathbf{y})$ to generate a random vector $\mathbf{d}_y \in \mathbb{Z}_2^n$, random matrix $\mathbf{C}_{y,1} \in \mathbb{Z}_2^{(n-\ell) \times \ell}$, and random full-rank matrix $\mathbf{C}_{y,2} \in \mathbb{Z}_2^{(n-\ell) \times (n-\ell)}$. Denote by $\mathbf{C}_y$ the full-rank matrix $\begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{C}_{y,1} & \mathbf{C}_{y,2} \end{bmatrix} \in \mathbb{Z}_2^{n \times n}$. This defines the following oracles:

- $(\mathbf{y} \in \mathbb{Z}_2^r, \mathbf{u} \in \mathbb{Z}_2^n) \leftarrow \mathcal{P}(\mathbf{x} \in \mathbb{Z}_2^n)$:
 - $(\bar{x} \in \{0, 1\}^{r+s}, \tilde{x} \in \mathbb{Z}_2^{n-r-s-\ell}, x' \in \mathbb{Z}_2^\ell) \leftarrow \Gamma(\mathbf{x})$.
 - $(\mathbf{y} \in \mathbb{Z}_2^r, \bar{\mathbf{u}} \in \mathbb{Z}_2^{r+s}) \leftarrow \bar{\mathcal{P}}(\bar{x})$.
 - $(\mathbf{C}_y \in \mathbb{Z}_2^{n \times n}, \mathbf{d}_y \in \mathbb{Z}_2^n) \leftarrow F'(\mathbf{y})$.
 - $\mathbf{u} \leftarrow \mathbf{C}_y \cdot \begin{pmatrix} x' \\ \bar{\mathbf{u}} \\ \tilde{x} \end{pmatrix} + \mathbf{d}_y$.
- $(\mathbf{x} \in \mathbb{Z}_2^n \cup \{\perp\}) \leftarrow \mathcal{P}^{-1}(\mathbf{y} \in \mathbb{Z}_2^r, \mathbf{u} \in \mathbb{Z}_2^n)$:
 - $(\mathbf{C}_y, \mathbf{d}_y) \leftarrow F'(\mathbf{y})$.
 - $\begin{pmatrix} x' \\ \bar{\mathbf{u}} \\ \tilde{x} \end{pmatrix} \leftarrow \mathbf{C}_y^{-1} \cdot (\mathbf{u} - \mathbf{d}_y)$.
 - $(\bar{x} \in \mathbb{Z}_2^{r+s} \cup \{\perp\}) \leftarrow \bar{\mathcal{P}}^{-1}(\mathbf{y}, \bar{\mathbf{u}})$
 - Output $\mathbf{x} \leftarrow \Gamma^{-1}(\bar{x}, \tilde{x}, x')$ if $\bar{x} \neq \perp$, else output $\perp$.
- $\{0, 1\} \leftarrow \mathcal{D}'(j \in [\ell + 1], \mathbf{y} \in \mathbb{Z}_2^r, \mathbf{v} \in \mathbb{Z}_2^n)$:
 - $(\mathbf{C}_y, \mathbf{d}_y) \leftarrow F'(\mathbf{y})$.
 - Output 1 iff $\mathbf{v}^T \cdot [\mathbf{C}_y^{[j:\ell]} \quad \mathbf{C}_y^{[\ell+s+1:n-r]}] = 0^{n-r-s-j+1}$.

Finally, $\mathcal{B}$ runs $(x_0, x_1) \leftarrow \mathcal{A}^{\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}'}, (\bar{x}_b, \tilde{x}_b, x'_b) \leftarrow \Gamma(x_b)$ and outputs $(\bar{x}_0, \bar{x}_1)$.

Define

$$\begin{aligned}
 \mathbf{A}_y &:= \mathbf{C}_y \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \bar{\mathbf{A}}_y & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I}_{n-r-s-\ell} \end{bmatrix} = \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{C}_{y,1} & \mathbf{C}_{y,2} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \bar{\mathbf{A}}_y & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I}_{n-r-s-\ell} \end{bmatrix} \\
 &= \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{C}_{y,1} & \mathbf{C}_{y,2} \cdot \begin{bmatrix} \bar{\mathbf{A}}_y & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_{n-r-s-\ell} \end{bmatrix} \end{bmatrix} \in \mathbb{Z}_2^{n \times (n-r)},
 \end{aligned}$$

where $\bar{\mathbf{A}}_y \in \mathbb{Z}_2^{(r+s) \times s}$ is the matrix arising from $\bar{\mathcal{P}}$ and $\bar{\mathcal{P}}^{-1}$. As $\mathbf{C}_{y,2}$ and $\bar{\mathbf{A}}_y$ have full column-rank, so does $\mathbf{A}_y^{[\ell+1:n-r]}$; on the other hand, $\mathbf{C}_{y,1}$ remains a random matrix; thus, $\mathbf{A}_y$ has full column-rank. Now, since $(\mathbf{u}_0 - \mathbf{u}_1) \notin \text{ColSpan}([\mathbf{A}_y^{[1:\ell]} \quad \mathbf{A}_y^{[\ell+s+1:n-r]}])$, this means that if we look at each of the two coordinates vectors $\mathbf{w}_0, \mathbf{w}_1$ (which each have $n - r$ bits) of $\mathbf{u}_0$ and $\mathbf{u}_1$, they differ somewhere between the $(\ell + 1)$ 'th and $(\ell + s)$ 'th bit. However, $(\mathbf{w}_b)_{[\ell+1:\ell+s]}$ arises from applying the permutation $\bar{\Pi}$ on $\{0, 1\}^{r+s}$ defined by the oracles $\mathcal{P}, \mathcal{P}^{-1}$ to $\bar{x}_b$ and taking the last s bits of the output. As x_0, x_1 have the same $\mathbf{y}$'s and thus so do $\bar{x}_0$ and $\bar{x}_1$, it must be the case that $\bar{x}_0 \neq \bar{x}_1$ in order for $\mathbf{w}_0$ to differ from $\mathbf{w}_1$ in the appropriate indices.

If we define ε_B as the probability that $\mathcal{A}$ outputs x_0, x_1 such that $y_0 = y_1 := \mathbf{y}$ and $(\mathbf{u}_0 - \mathbf{u}_1) \notin \text{ColSpan}([\mathbf{A}_y^{[1:\ell]} \quad \mathbf{A}_y^{[\ell+s+1:n-r]}])$, we wish to lower bound the probability of ε_B . We do this through a series of hybrids.

- **Hyb₀: The distribution $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}') \leftarrow \mathcal{B}^{\bar{\mathcal{P}}, \bar{\mathcal{P}}^{-1}}$, as simulated by $\mathcal{B}$.**

This is the original execution, and thus has success probability ε_B .

- **Hyb₁: Removing the inner permutation $\bar{\Pi}$ by using Γ .**

Instead of applying $\bar{\Pi}$ and Π^{-1} inside $\bar{\mathcal{P}}$ and $\bar{\mathcal{P}}^{-1}$, respectively, only use Γ . This hybrid is statistically identical to the previous hybrid since a concatenation of a permutation with a random permutation Γ is still a random permutation.

- **Hyb₂: Taking $\mathbf{A}_y, \mathbf{b}_y$ to be the direct output of F by using F' .**

Instead of sampling $\mathbf{C}_y$ and $\mathbf{d}_y$ before computing $\mathbf{A}_y := \mathbf{C}_y \cdot \begin{bmatrix} \mathbf{I} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \bar{\mathbf{A}}_y & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I}_{n-r-s} \end{bmatrix}$ and $\mathbf{b}_y := \mathbf{C}_y \cdot \begin{pmatrix} \mathbf{0} \\ \bar{\mathbf{b}}_y \\ \mathbf{0} \end{pmatrix} + \mathbf{d}_y$,

where $\bar{\mathbf{A}}_y, \bar{\mathbf{b}}_y$ are defined by the output of the inner random function, we sample $\mathbf{A}_y$ and $\mathbf{b}_y$ directly. We observe that by the randomness of $\mathbf{d}_y, \mathbf{C}_{y,1}, \mathbf{C}_{y,2}$, and $\bar{\mathbf{A}}_y$, the joint distributions of matrices and offset vectors in both hybrids are identical. Therefore, the success probability of $\mathcal{A}$ in this hybrid is also ε_B .

Remark E.1. Note that

$$\mathbf{b}_y - \mathbf{d}_y = \mathbf{C}_y \cdot \begin{pmatrix} \mathbf{0} \\ \bar{\mathbf{b}}_y \\ \mathbf{0} \end{pmatrix} = \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{C}_{y,1} & \mathbf{C}_{y,2} \end{bmatrix} \cdot \begin{pmatrix} \mathbf{0} \\ \bar{\mathbf{b}}_y \\ \mathbf{0} \end{pmatrix} = \begin{pmatrix} \mathbf{0} \\ \mathbf{C}_{y,2} \cdot \begin{pmatrix} \bar{\mathbf{b}}_y \\ \mathbf{0} \end{pmatrix} \end{pmatrix}$$

has only zeros in the first ℓ coordinates. Thus, if $\mathbf{d}_y$ is distributed randomly, then $\mathbf{b}_y$ is distributed randomly; if $\mathbf{d}_y$ is uniformly random with the ℓ 'th coordinate set to 1, then so is $\mathbf{b}_y$.

Since the distribution generated in Hyb₂ is precisely $\mathcal{O}'_{n,r,\ell,s}$, this means that $\varepsilon_B = \varepsilon$, as desired. $\square$

Proof of Lemma 5.1. The perfect correctness follows from the perfect correctness of Construction 2 and observing that the verification algorithm **Ver** in Construction 2 does not actually need the full capability of $\mathcal{P}^{-1}$ and that having access to $\mathcal{D}_0$ suffices.

The oracle $\mathcal{D}_0$ can be implemented using $\mathcal{P}^{-1}$. Thus, it suffices to show the strong signature security of Construction 3 with respect to the oracles $\mathcal{P}, \mathcal{P}^{-1}$, and $\mathcal{D}$ (where we will abuse notation and also refer to this distribution of oracles by $\tilde{\mathcal{O}}_{n,r,\ell}$).

As with Construction 2, we can “bloat” the dual oracle. Let $\tilde{\mathcal{O}}'_{n,r,\ell,s}$ denote the following distribution over oracles $\mathcal{P}, \mathcal{P}^{-1}$, and $\mathcal{D}'$. The oracles $\mathcal{P}$ and $\mathcal{P}^{-1}$ are defined identically to the oracles in $\tilde{\mathcal{O}}_{n,r,\ell}$. However, our new dual oracle $\mathcal{D}'$ will be a relaxed version of the original dual oracle $\mathcal{D}$:

$$\mathcal{D}'(j, y, \mathbf{v}) = \begin{cases} 1 & \text{if } \mathbf{v}^T \cdot \begin{bmatrix} \mathbf{A}_y^{[j:\ell]} & \mathbf{A}_y^{[\ell+1+s:n-r]} \end{bmatrix} = 0^{n-r-s-j+1} \text{ and } j \in [\ell+1], \\ 0 & \text{otherwise.} \end{cases}$$

The analog to Lemma 4.4 remains true for these sets of oracle distributions:

Lemma E.2 (Analogous Bloating Lemma). *Suppose there is an oracle-aided q -query quantum algorithm $\mathcal{A}$ such that*

$$\Pr \left[\begin{array}{l} (y_0 = y_1) \wedge (x_0 \neq x_1) : \\ (x_0, x_1) \leftarrow \mathcal{A}^{\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}} \\ (y_b, \mathbf{u}_b) \leftarrow \mathcal{P}(x_b) \end{array} \right] \geq \varepsilon.$$

Then we have that

$$\Pr \left[\begin{array}{l} (y_0 = y_1 := y) \wedge \\ (\mathbf{u}_0 - \mathbf{u}_1) \notin \text{ColSpan} \left(\begin{bmatrix} \mathbf{A}_y^{[1:\ell]} & \mathbf{A}_y^{[\ell+s+1:n-r]} \end{bmatrix} \right) : \\ (x_0, x_1) \leftarrow \mathcal{A}^{\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}'} \\ (y_b, \mathbf{u}_b) \leftarrow \mathcal{P}(x_b) \end{array} \right] \geq \frac{\varepsilon}{256n^2}.$$

Proof. The proof of Lemma E.2 is identical to the proof of Lemma 4.4, since the oracle distribution of $\tilde{\mathcal{O}}_{n(\lambda),r(\lambda),\ell(\lambda)}$ is the same as the distribution of $\mathcal{O}_{n(\lambda+1),r(\lambda+1),\ell(\lambda+1)}$ except for the coset shift vectors $\{\mathbf{b}_y\}_y$ whose distribution is not used in the proof. $\square$

Just like with Construction 2, we can entirely simulate access to a bloated dual given only access to the primal oracles.

Lemma E.3 (Analogous Dual Simulation Lemma). *Suppose that there is an oracle aided q -query algorithm $\mathcal{A}$ such that*

$$\Pr \left[\begin{array}{l} (y_0 = y_1 := \mathbf{y}) \wedge \\ (\mathbf{u}_0 - \mathbf{u}_1) \notin \text{ColSpan} \left(\begin{bmatrix} \mathbf{A}_y^{[1:\ell]} & \mathbf{A}_y^{[\ell+s+1:n-r]} \end{bmatrix} \right) \end{array} : \begin{array}{l} (\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}') \leftarrow \tilde{\mathcal{O}}'_{n,r,\ell,s} \\ (x_0, x_1) \leftarrow \mathcal{A}^{\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}'} \\ (y_b, \mathbf{u}_b) \leftarrow \mathcal{P}(x_b) \end{array} \right] \geq \varepsilon.$$

Then there is an oracle aided q -query algorithm $\mathcal{B}$ such that

$$\Pr \left[\begin{array}{l} (\overline{\mathcal{P}}, \overline{\mathcal{P}}^{-1}, \overline{\mathcal{D}}) \leftarrow \mathcal{O}_{r+s,r,0} \\ (\overline{y}_0 = \overline{y}_1) \wedge (\overline{x}_0 \neq \overline{x}_1) : \end{array} \begin{array}{l} (\overline{x}_0, \overline{x}_1) \leftarrow \mathcal{B}^{\overline{\mathcal{P}}, \overline{\mathcal{P}}^{-1}} \\ (\overline{y}_b, \overline{\mathbf{u}}_b) \leftarrow \overline{\mathcal{P}}(\overline{x}_b) \end{array} \right] \geq \varepsilon,$$

where $s := 16 \cdot (\lambda + 1)$ and $r := s \cdot \lambda$.

Note that the new distribution $\mathcal{O}_{r+s,r,0}$ is the same dual-free oracle distribution as in Section 4, which uses fully random coset shift vectors.

Proof. The proof is identical to that of Lemma 4.5 with one difference: instead of sampling the random function F' such that on input $\mathbf{y} \in \{0, 1\}^r$, $F'(\mathbf{y})$ provides randomness to generate a random vector $\mathbf{d}_y \in \mathbb{Z}_2^n$, $\mathbf{d}_y$ will be sampled to be a random vector whose $\ell = (\lambda + 1)$ 'th coordinate is 1. It is not hard to verify that with this change, the rest of the proof goes through (see Remark E.1 for more details). $\square$

We can now apply Theorems 4.6 and 4.8 and conclude that Construction 3 has strong signature security (following the same outline as in Section 4 until Corollary 4.10). $\square$

F Missing Proofs from Section B

The proofs of the following lemmas will closely track the general outline of [SZ25].

Proof of Lemma B.2. We include a proof of Lemma B.2, which very closely follows the proof of Lemma 54 in [SZ25], for completeness. We will consider a series of hybrids, and show that the success probability in each pair of hybrids is computationally close.

- **Hyb₀: The original execution of $\mathcal{A}$.**

By definition, $\mathcal{A}$ succeeds with probability ε .

- **Hyb₁: Preparing to switch to a small-range distribution.**

Let $(\text{LF.KeyGen}, \text{LF.F})$ be a $(f(\kappa), \frac{1}{f(\kappa)})$ -secure lossy function with output length m and lossiness parameter w . We would like to apply Lemma A.1, so we will describe two PRFs and one ensemble of distributions (which will serve as both $\mathcal{D}_0$ and $\mathcal{D}_1$). The first PRF takes as input $\{0, 1\}^d$ and applies $F(k_{\text{lin}}, \cdot)$ to get randomness r . The second PRF takes as input $\{0, 1\}^d$, samples $\text{pk}_{\text{LF}} \leftarrow \text{LF.KeyGen}(1^\kappa, 0, 1^w)$, applies $\text{LF.F}(\text{pk}_{\text{LF}}, \cdot)$ to the input (padded with $\kappa - d$ zeros), and then applies $F(k'_{\text{lin}}, \cdot)$ to get randomness r' .

We note that pk_{LF} produces an associated injective function with probability $\geq 1 - \text{negl}(\kappa) = 1 - \text{negl}(\lambda)$, so we assume this is the case in the following analysis.

The first function is a puncturable PRF by definition. It is not hard to see that the second function is indeed a puncturable PRF: the key is simply $(\text{pk}_{\text{LF}}, k'_{\text{lin}})$, and evaluation can be done by padding with zeros and successively applying $\text{LF.F}(\text{pk}_{\text{LF}}, \cdot)$ and $\text{F}(k'_{\text{lin}}, \cdot)$. Puncturing is also relatively simple: for a set S , one evaluates the set S on $\text{LF.F}(\text{pk}_{\text{LF}}, \cdot)$ to get a set of images S' before puncturing $k'_{\text{lin}}\{S'\} \leftarrow \text{Punc}(k'_{\text{lin}}, S')$. Correctness follows from the injectivity of LF.F and the correctness of F , while puncturing security follows from the injectivity of LF.F and the puncturing security of F on inputs of length m .

If we now define distributions $D = D_0 = D_1$ which take as input $y \in \{0, 1\}^d$ and randomness r and construct a coset (A_y, b_y) , it is easy to see that in Hyb_0 , the circuit $E_0(y) := D(y; \text{F}(k_{\text{lin}}, y))$ is used black-box by P , P^{-1} , and D (and all three circuits are obfuscated).

In Hyb_1 , we will instead use the circuit $E_1(y) := D(y; \text{F}(k'_{\text{lin}}, \text{LF.F}(\text{pk}_{\text{LF}}, y)))$, where pk_{LF} and k'_{lin} are freshly sampled. It follows by Lemma A.1 that the output of Hyb_0 and Hyb_1 are $(f(\kappa) - \text{poly}(\kappa), O(|\mathcal{X}|/f(\kappa)))$ -computationally indistinguishable, where $\mathcal{X}$ is the set of all possible values of y , which has size 2^r . We incur some loss due to assuming LF.F is injective, and thus it follows by Assumptions (1) and (3) that the success probability of $\mathcal{A}$ in this hybrid is

$$\varepsilon_1 \geq \varepsilon - O\left(\frac{2^r}{f(\kappa)}\right) - \text{negl}(\lambda) \geq \frac{31\varepsilon}{32}.$$

- **Hyb₂: Switching to a small number of cosets.**

E_1 now samples a lossy key $\text{pk}'_{\text{LF}} \leftarrow \text{LF.KeyGen}(1^\kappa, 1, 1^w)$ instead of an injective key $\text{pk}_{\text{LF}} \leftarrow \text{LF.KeyGen}(1^\kappa, 0, 1^w)$. By the security of the lossy function, the output of this hybrid is $(f(\kappa), \frac{1}{f(\kappa)})$ -computationally indistinguishable from the previous hybrid. Thus, Assumption (1) implies that the success probability of $\mathcal{A}$ in this hybrid is

$$\varepsilon_2 \geq \varepsilon_1 - \frac{1}{f(\kappa)} \geq \frac{15\varepsilon}{16}.$$

We will refer to the image of the lossy function $\text{LF.F}(\text{pk}'_{\text{LF}}, \cdot)$ by $\mathcal{X}$. Note that in this hybrid, the coset A_y, b_y is also an efficiently computable function of $x = \text{LF.F}(\text{pk}'_{\text{LF}}, y) \in \mathcal{X}$ (and is the same for all y 's which map to the same x), so we will refer to these cosets by A_x, b_x .

- **Hyb₃: Using obfuscated circuits inside D .**

Recall that in Hyb_2 , $D(j, y, v)$ computes x before applying $\text{F}(k_{\text{lin}}, x)$ to get pseudorandomness r_x which is used to generate a coset (A_x, b_x) ; $D(j, y, v)$ then checks membership in the dual of $\text{ColSpan}(A_x^{[j:n-r]}) := S_{x,j}$.

In Hyb_3 , we additionally sample another puncturable PRF key k_S . After computing x , we apply $\text{F}(k_S, x)$ to obtain additional pseudorandomness r'_x ; using r'_x , we generate obfuscations $O_{S_{x,j}^\perp} \leftarrow \text{iO}(1^\kappa, 1^p, C_{S_{x,j}^\perp})$ of the membership circuits for $S_{x,j}^\perp$. D now uses $O_{S_{x,j}^\perp}$ instead of $C_{S_{x,j}^\perp}$ to decide dual membership. By the correctness of the inner obfuscation scheme, D remains functionally equivalent, so the two hybrids are $(f(\kappa), \frac{\ell}{f(\kappa)})$ -computationally close. Thus, by Assumption (1), the success probability of $\mathcal{A}$ in this hybrid is

$$\varepsilon_3 \geq \varepsilon_2 - \frac{\ell}{f(\kappa)} \geq \frac{29\varepsilon}{32}.$$

- **Hyb₄: Relaxing dual verification inside D .**

Once again, we would like to apply Lemma A.1, so we now describe the two ensembles of distributions we want to use; we will use one puncturable PRF F . The first distribution $\mathcal{D}_{S, k_{\text{lin}}}$ takes as input $\mathbf{x}$ and randomness r and computes $S_{\mathbf{x}}$ from $F(k_{\text{lin}}, \mathbf{x})$ before using r to generate obfuscations $\mathcal{O}_{S_{\mathbf{x},j}}^{\perp} \leftarrow \text{iO}(1^\kappa, 1^p, C_{S_{\mathbf{x},j}}^{\perp})$. The second distribution $\mathcal{D}_{T, k_{\text{lin}}}$ takes as input $\mathbf{x}$ and randomness r and computes $S_{\mathbf{x}}$ from $F(k_{\text{lin}}, \mathbf{x})$ before using r to sample superspaces $T_{\mathbf{x},j}^{\perp}$ (with s more dimensions) of $S_{\mathbf{x},j}^{\perp}$ and generate obfuscations $\mathcal{O}_{T_{\mathbf{x},j}^{\perp}} \leftarrow \text{iO}(1^\kappa, 1^p, C_{T_{\mathbf{x},j}^{\perp}})$.

Specifically, the first part of r will be used to generate a random full column-rank matrix $\mathbf{M}_{\mathbf{x}} \in \mathbb{Z}_2^{(n-r-\ell) \times (n-r-\ell)}$ and a random matrix $\mathbf{M}'_{\mathbf{x}} \in \mathbb{Z}_2^{(n-r-\ell) \times \ell}$. This defines matrices $\mathbf{C}_{\mathbf{x}} := \begin{bmatrix} \mathbf{I}_{\ell} & \mathbf{0} \\ \mathbf{M}'_{\mathbf{x}} & \mathbf{M}_{\mathbf{x}} \end{bmatrix} \in \mathbb{Z}_2^{(n-r) \times (n-r)}$ and $\bar{\mathbf{A}}_{\mathbf{x}} := \mathbf{A}_{\mathbf{x}} \cdot \mathbf{C}_{\mathbf{x}} \in \mathbb{Z}_2^{n \times (n-r)}$. We can now define

$$T_{\mathbf{x},j}^{\perp} := \text{ColSpan} \left(\begin{bmatrix} \bar{\mathbf{A}}_{\mathbf{x}}^{[j:\ell]} & \bar{\mathbf{A}}_{\mathbf{x}}^{[\ell+s+1:n-r]} \end{bmatrix} \right)^{\perp}.$$

By Lemmas D.2 and D.6, the distributions $\mathcal{D}_{S, k_{\text{lin}}}$ and $\mathcal{D}_{T, k_{\text{lin}}}$ are $(f(n-r-s-\ell) - \text{poly}(n), \text{poly}(n)/f(n-r-s-\ell))$ -computationally indistinguishable (even though $S_{\mathbf{x},j}$ are known).

Since $|\mathcal{X}| \leq 2^w$ with probability $1 - \text{negl}(\lambda)$, in what follows, we assume this is the case. Consider the two circuits $E_{S, k_{\text{lin}}}(\mathbf{x}) := \mathcal{D}_{S, k_{\text{lin}}}(\mathbf{x}, F(k_S, \mathbf{x}))$ and $E_{T, k_{\text{lin}}}(\mathbf{x}) := \mathcal{D}_{T, k_{\text{lin}}}(\mathbf{x}, F(k_T, \mathbf{x}))$ for uniformly sampled keys k_S, k_T . Observe that $E_{S, k_{\text{lin}}}$ is used black-box in D in Hyb_3 , and D will be obfuscated. Thus, if we switch to $E_{T, k_{\text{lin}}}$, by Lemma A.1, this is $(f(n-r-s-\ell) - \text{poly}(n), 2^w \cdot \text{poly}(n)/f(n-r-s-\ell))$ -computationally indistinguishable. We conclude that Hyb_3 and Hyb_4 are $(f(n-r-s-\ell) - \text{poly}(n), 1/f(\kappa) + 2^w \cdot \text{poly}(n)/f(n-r-s-\ell))$ -computationally indistinguishable.

Therefore, after incurring some loss from the assumption that $|\mathcal{X}| \leq 2^w$, we have that by Assumptions (1) and (3), the success probability of $\mathcal{A}$ in this hybrid is

$$\varepsilon_4 \geq \varepsilon_3 - \text{negl}(\lambda) - \frac{2^w \cdot \text{poly}(n)}{f(n-r-s-\ell)} \geq \frac{7\varepsilon}{8}.$$

- **Hyb₅: Changing the success predicate.**

Recall that in Hyb_4 , we say $\mathcal{A}$ is successful if it finds a collision, i.e. it produces a pair $x_0 \neq x_1$ such that $y_0 = y_1 := \mathbf{y}$. We now say that $\mathcal{A}$ is successful only if $y_0 = y_1 := \mathbf{y}$ and $\mathbf{u}_0 - \mathbf{u}_1 \notin T_{\mathbf{x},1}$. As $|\mathcal{X}| \leq 2^w$ with probability $1 - \text{negl}(\lambda)$, in what follows, we assume this is the case.

For every $\mathbf{x} \in \mathcal{X}$, let $\varepsilon_4^{(\mathbf{x})}$ be the probability that $\mathcal{A}$ finds $x_0 \neq x_1$ such that $y_0 = y_1 := \mathbf{y}$ and $\mathbf{x} = \text{LF.F}(\text{pk}_{\text{LF}}, \mathbf{y})$; by definition, $\sum_{\mathbf{x} \in \mathcal{X}} \varepsilon_4^{(\mathbf{x})} = \varepsilon_4$. Observe that when a collision occurs, we have $\mathbf{u}_0 - \mathbf{u}_1 \in \text{ColSpan}(\mathbf{A}_{\mathbf{x}}) \setminus \{0\} = S_{\mathbf{x},1} \setminus \{0\}$. Let $\delta^{(\mathbf{x})}$ be the probability that $\mathcal{A}$ finds x_0, x_1 such that $y_0 = y_1 := \mathbf{y}$, $\mathbf{x} = \text{LF.F}(\text{pk}_{\text{LF}}, \mathbf{y})$, and $\mathbf{u}_0 - \mathbf{u}_1 \in T_{\mathbf{x},1} \setminus \{0\}$.

Define $\varepsilon_5^{(\mathbf{x})}$ to be the probability that $\mathcal{A}$ finds x_0, x_1 such that $y_0 = y_1 := \mathbf{y}$, $\mathbf{x} = \text{LF.F}(\text{pk}_{\text{LF}}, \mathbf{y})$, and $\mathbf{u}_0 - \mathbf{u}_1 \notin T_{\mathbf{x},1}$. Let L be the subset of $\mathcal{X}$ such that $\varepsilon_4^{(\mathbf{x})} \geq \frac{\varepsilon_4}{2|\mathcal{X}|}$; this means that $\sum_{\mathbf{x} \in L} \varepsilon_4^{(\mathbf{x})} \geq \varepsilon_4/2$. Our goal is to show that for every $\mathbf{x} \in L$, $\varepsilon_5^{(\mathbf{x})} \geq \frac{\varepsilon_4^{(\mathbf{x})}}{16n^2}$. By definition, we have that $\varepsilon_5^{(\mathbf{x})} \geq \varepsilon_4^{(\mathbf{x})} - \delta^{(\mathbf{x})}$, so if $\delta^{(\mathbf{x})} \leq \varepsilon_4^{(\mathbf{x})}/2$, then we are done. Thus, we assume that

$$\delta^{(\mathbf{x})} > \frac{\varepsilon_4^{(\mathbf{x})}}{2} \geq \frac{\varepsilon_4}{4|\mathcal{X}|} \geq \Omega\left(\frac{\varepsilon}{2^w}\right).$$

Let $t := n - r - s \leq n - 1$ and $m := \frac{n(t+1)}{\delta^{(\mathbf{x})}} \leq O\left(\frac{n^2 \cdot 2^w}{\varepsilon}\right)$. By Assumption (2), we have that

$$\frac{t}{\delta^{(\mathbf{x})}} \cdot (2^{t-s} + 2^{-t+\ell}) = O\left(\frac{2^w \cdot t}{\varepsilon}\right) \cdot (2^{t-s} + 2^{-t+\ell}) \leq o(1),$$

and by Assumption (3),

$$\frac{m \cdot \text{poly}(n) \cdot T_A}{f(n-r-s-\ell)} = \frac{2^w \cdot \text{poly}(n) \cdot T_A / \varepsilon}{f(n-r-s-\ell)} \leq o(1),$$

so the conditions of Lemma D.9 are fulfilled. Thus, it follows from Lemma D.9 and the security of the iO and puncturable PRF that for $\mathbf{x} \in L$,

$$\varepsilon_5^{(\mathbf{x})} \geq \frac{\delta^{(\mathbf{x})}}{16n(t+1)} - O\left(\frac{1}{f(\kappa)}\right) \geq \frac{\delta^{(\mathbf{x})}}{16n^2} - O\left(\frac{1}{f(\kappa)}\right) \geq \frac{\varepsilon_4^{(\mathbf{x})}}{32n^2} - O\left(\frac{1}{f(\kappa)}\right).$$

Therefore, after incurring some loss from the assumption that $|\mathcal{X}| \leq 2^w$, we have that by Assumption (3), the success probability of $\mathcal{A}$ in this hybrid is

$$\begin{aligned} \varepsilon_5 &= \sum_{\mathbf{x} \in \mathcal{X}} \varepsilon_5^{(\mathbf{x})} - \text{negl}(\lambda) \geq \left(\sum_{\mathbf{x} \in L} \varepsilon_5^{(\mathbf{x})} \right) - \text{negl}(\lambda) \\ &\geq \sum_{\mathbf{x} \in L} \left[\frac{\varepsilon_4^{(\mathbf{x})}}{32n^2} - O\left(\frac{1}{f(\kappa)}\right) \right] - \text{negl}(\lambda) \geq \frac{\varepsilon_4/2}{32n^2} - O\left(\frac{|\mathcal{X}|}{f(\kappa)}\right) - \text{negl}(\lambda) \\ &\geq \frac{7\varepsilon}{512n^2} - O\left(\frac{2^w}{f(\kappa)}\right) - \text{negl}(\lambda) \geq \frac{3\varepsilon}{256n^2}. \end{aligned}$$

- **Hyb₆: Using the security of the P-PRP.**

Sample all components except k_{in} for the initial permutation, thus defining subspaces $S_{\mathbf{x}, \ell+1} \subseteq \dots \subseteq S_{\mathbf{x}, 1}$ and $T_{\mathbf{x}, \ell+1} \subseteq \dots \subseteq T_{\mathbf{x}, 1}$.

Now consider the following permutation Γ over $\{0, 1\}^n$. Denote by h_0 the first r bits and h_1 the last $n-r$ bits of the input to Γ . Recall that in P , P^{-1} , and D , the value of $\mathbf{x}$ (and more specifically the matrices $\mathbf{M}_{\mathbf{x}}$ and $\mathbf{M}'_{\mathbf{x}}$ which are used to compute $\mathbf{C}_{\mathbf{x}}$) can be written as an efficient function of $H(\mathbf{x}) \in \{0, 1\}^r$. Thus, we define Γ as the (efficient) permutation which takes h_0 , computes $\mathbf{M}_{h_0}$, $\mathbf{M}'_{h_0}$, and $\mathbf{C}_{h_0}$, interprets h_1 as a vector in $\mathbb{Z}_2^{n-r}$, and multiplies h_1 by $\mathbf{C}_{h_0}$. Note that this means we will now use the PRF key k_T in P and P^{-1} as well, because it is needed to compute $\mathbf{M}_{h_0}$ and $\mathbf{M}'_{h_0}$ and hence to compute Γ .

For every value $h_0 \in \{0, 1\}^r$, multiplying by $\mathbf{C}_{h_0}$ is an affine transformation, which is $(2^{n-r} \cdot (n-r)^2, \text{poly}(n-r))$ -decomposable by Lemma A.3. Thus, Γ applies a controlled decomposable permutation and is itself a $(2^r \cdot 2^{n-r} \cdot (n-r)^2, \text{poly}(n-r) + \text{poly} \log(2^n)) = (2^n \cdot n^2, \text{poly}(n)) = (2^{\text{poly}(\lambda)}, \text{poly}(\lambda)) = (2^{\text{poly}(\kappa)}, \text{poly}(\kappa))$ -decomposable permutation by Lemma A.3.

Thus, if we switch to applying Γ to the output of Π (and Γ^{-1} to the input of Π^{-1}), since the circuits P and P^{-1} which use Π and Π^{-1} are obfuscated, this change is $(f(\kappa), O(\frac{1}{f(\kappa)}))$ -computationally indistinguishable by Lemma A.5.

- **Hyb₇: Using the security of outside iO.**

For every $h \in \mathbb{Z}_2^r$, we make the following change to P , P^{-1} , and D . Instead of composing the permutation Γ to the output of Π (by using $k_{\text{in}}^{\Gamma, 1}$), we switch to using just k_{in} which applies Π as-is. However, after generating $(\mathbf{A}_{\mathbf{x}}, \mathbf{b}_{\mathbf{x}})$, we will compute $\mathbf{M}_{\mathbf{x}}$, $\mathbf{M}'_{\mathbf{x}}$, and $\mathbf{C}_{\mathbf{x}}$ to get $\bar{\mathbf{A}}_{\mathbf{x}} := \mathbf{A}_{\mathbf{x}} \cdot \mathbf{C}_{\mathbf{x}}$. P and P^{-1} will now use $(\bar{\mathbf{A}}_{\mathbf{x}}, \mathbf{b}_{\mathbf{x}})$ instead of $(\mathbf{A}_{\mathbf{x}}, \mathbf{b}_{\mathbf{x}})$. Additionally, in D , instead of obfuscating the membership check circuits for $T_{\mathbf{x}, i}^{\perp}$, we go back to using $\bar{\mathbf{A}}_{\mathbf{x}}$ to directly check for membership.

Since P , P^{-1} , and D remain functionally equivalent, by the security of the outer iO scheme that obfuscates P , P^{-1} , and D , this change is $(f(\kappa), O(\frac{1}{f(\kappa)}))$ -computationally indistinguishable from Hyb_6 . Thus, by Assumption (1), the success probability of $\mathcal{A}$ in this hybrid is

$$\varepsilon_7 \geq \varepsilon_6 - O\left(\frac{1}{f(\kappa)}\right) \geq \varepsilon_5 - O\left(\frac{1}{f(\kappa)}\right) \geq \frac{\varepsilon}{128n^2}.$$

- **Hyb₈: Using the security of the puncturable PRF.**

Recall the process in Hyb_7 for generating our cosets in all three circuits P , P^{-1} , and D : given y we compute x with the lossy function, and then apply a puncturable PRF with two i.i.d keys, k_{lin} and k_T to obtain (A_x, b_x) and the matrix C_x . We then compute $\bar{A}_x := A_x \cdot C_x$ and use $(\bar{A}_x, b_x)$ in all circuits. Our goal is to apply Lemma A.1, so we will describe two distributions and two puncturable PRFs.

The first distribution D_2 takes as input x and randomness (r_1, r_2) , uses r_1 to generate (A_x, b_x) and r_2 to generate C_x , computes $\bar{A}_x := A_x \cdot C_x$, and outputs $(\bar{A}_x, b_x)$. The second distribution D_3 takes as input x and randomness r_1 and uses r_1 to generate (A_x, b_x) . By an identical argument as in the proof of Lemma 4.4, it is easy to see that D_2 and D_3 are statistically equivalent.

Consider the function F' which takes as input x and outputs $(r_1, r_2) := (F(k_1, x), F(k_2, x))$ for randomly sampled keys k_1, k_2 . We claim that F' is a puncturable PRF whose keys are simply a pair of keys for F . Puncturing F' is simple: $\text{Punc}'(k = (k_1, k_2), S) = (\text{Punc}(k_1, S), \text{Punc}(k_2, S))$. The correctness of F' follows directly from the correctness of F , and it is not hard to see that F' has $(f(\kappa) - \text{poly}(\kappa), O(\frac{1}{f(\kappa)}))$ -puncturing security by the security of F .

As $|\mathcal{X}| \leq 2^w$ with probability $1 - \text{negl}(\lambda)$, in what follows, we assume this is the case. We now define the circuits $E_2(x) = D_2(x; F'((k_{\text{lin}}, k_T), x))$ and $E_3(x) = D_3(x; F(k'_{\text{lin}}, x))$ for freshly sampled keys $k_{\text{lin}}, k_T, k'_{\text{lin}}$. Observing that all three circuits in Hyb_7 use E_2 black-box, we see that Lemma A.1 implies that switching to E_3 in all three circuits is $(f(\kappa) - \text{poly}(\kappa), O(\frac{2^w}{f(\kappa)}))$ -computationally indistinguishable.

Therefore, after incurring some loss from the assumption that $|\mathcal{X}| \leq 2^w$, we have that by Assumptions (1) and (3), the success probability of $\mathcal{A}$ in Hyb_8 is

$$\varepsilon_8 \geq \varepsilon_7 - O\left(\frac{2^w}{f(\kappa)}\right) - \text{negl}(\lambda) \geq \frac{\varepsilon}{128n^2} - \frac{\varepsilon}{512n^2} = \frac{3\varepsilon}{512n^2}.$$

- **Hyb₉: Switching back to injective mode of the lossy function.**

We make the same change from Hyb_1 to Hyb_2 , but in the other direction. Recall that in the previous hybrid Hyb_8 , on input y , we compute $x \leftarrow \text{LF.F}(\text{pk}_{\text{LF}}, y)$ and use x to compute all other components. We now sample an injective key $\text{pk}'_{\text{LF}} \leftarrow \text{LF.KeyGen}(1^\kappa, 0, 1^w)$ instead of the lossy key pk_{LF} . By the security of the lossy function, the output of this hybrid is $(f(\kappa), \frac{1}{f(\kappa)})$ -computationally indistinguishable from Hyb_8 . Thus, Assumption (1) implies that the success probability of $\mathcal{A}$ in this hybrid is

$$\varepsilon_9 \geq \varepsilon_8 - \frac{1}{f(\kappa)} \geq \frac{\varepsilon}{256n^2}.$$

- **Hyb₁₀: Stopping use of the lossy function.**

We make the same change from Hyb_0 to Hyb_1 , but in the other direction. Instead of applying LF.F to $y||0^{\kappa-d}$ to get x and then applying F to x to get randomness, we directly apply F (with a different key) to y to get our randomness. By an identical argument as before, this hybrid is $(f(\kappa) - \text{poly}(\kappa), O(2^r/f(\kappa)))$ -computationally indistinguishable from Hyb_9 , so by Assumptions (1) and (3), the success probability of $\mathcal{A}$ in this hybrid is

$$\varepsilon_{10} \geq \varepsilon_9 - O\left(\frac{2^r}{f(\kappa)}\right) - \text{negl}(\lambda) \geq \frac{\varepsilon}{512n^2}.$$

We conclude by noting that Hyb_{10} is exactly the distribution $(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}') \leftarrow \widetilde{\text{Setup}}(1^\lambda, n, r, \ell, s)$. $\square$

Proof of Lemma B.3. Given $\overline{\mathcal{P}}$ and $\overline{\mathcal{P}}^{-1}$ from $(\overline{\mathcal{P}}, \overline{\mathcal{P}}^{-1}, \overline{\mathcal{D}}) \leftarrow \widetilde{\text{Setup}}(1^\lambda, r+s, r, 0, 0)$ and the code of $\mathcal{A}, \mathcal{B}$ acts as follows:

- Sample a puncturable PRF key k_F that outputs a sufficient (polynomial in κ) number of bits and a P-PRP key k_Γ for a P-PRP on $\{0, 1\}^n$. On input $\mathbf{y} \in \{0, 1\}^d$, use the pseudorandomness of $F(k_F, \mathbf{y})$ to generate a vector $\mathbf{d}_y \in \mathbb{Z}_2^n$, matrix $\mathbf{C}_{y,1} \in \mathbb{Z}_2^{(n-\ell) \times \ell}$, and full-rank matrix $\mathbf{C}_{y,2} \in \mathbb{Z}_2^{(n-\ell) \times (n-\ell)}$. Denote by $\mathbf{C}_y$ the full-rank matrix $\begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{C}_{y,1} & \mathbf{C}_{y,2} \end{bmatrix} \in \mathbb{Z}_2^{n \times n}$; we write as shorthand $(\mathbf{C}_y, \mathbf{d}_y) \leftarrow F(k_F, \mathbf{y})$ to refer to this process. This defines the following circuits:

- $(\mathbf{y} \in \mathbb{Z}_2^d, \mathbf{u} \in \mathbb{Z}_2^n) \leftarrow P(x \in \mathbb{Z}_2^n)$:
 - $(\bar{x} \in \{0, 1\}^{r+s}, \tilde{x} \in \mathbb{Z}_2^{n-r-s-\ell}, x' \in \mathbb{Z}_2^\ell) \leftarrow \Pi(k_\Gamma, x)$.
 - $(\mathbf{y} \in \mathbb{Z}_2^d, \bar{\mathbf{u}} \in \mathbb{Z}_2^{r+s}) \leftarrow \overline{\mathcal{P}}(\bar{x})$.
 - $(\mathbf{C}_y \in \mathbb{Z}_2^{n \times n}, \mathbf{d}_y \in \mathbb{Z}_2^n) \leftarrow F(k_F, \mathbf{y})$.
 - $\mathbf{u} \leftarrow \mathbf{C}_y \cdot \begin{pmatrix} x' \\ \bar{\mathbf{u}} \\ \tilde{x} \end{pmatrix} + \mathbf{d}_y$.
- $(x \in \mathbb{Z}_2^n \cup \{\perp\}) \leftarrow P^{-1}(\mathbf{y} \in \mathbb{Z}_2^d, \mathbf{u} \in \mathbb{Z}_2^n)$:
 - $(\mathbf{C}_y, \mathbf{d}_y) \leftarrow F(k_F, \mathbf{y})$.
 - $\begin{pmatrix} x' \\ \bar{\mathbf{u}} \\ \tilde{x} \end{pmatrix} \leftarrow \mathbf{C}_y^{-1} \cdot (\mathbf{u} - \mathbf{d}_y)$.
 - $(\bar{x} \in \mathbb{Z}_2^{r+s} \cup \{\perp\}) \leftarrow \overline{\mathcal{P}}^{-1}(\mathbf{y}, \bar{\mathbf{u}})$
 - Output $x \leftarrow \Pi^{-1}(k_\Gamma, (\bar{x}, \tilde{x}, x'))$ if $\bar{x} \neq \perp$, else output $\perp$.
- $\{0, 1\} \leftarrow D'(j \in [\ell+1], \mathbf{y} \in \mathbb{Z}_2^d, \mathbf{v} \in \mathbb{Z}_2^n)$:
 - $(\mathbf{C}_y, \mathbf{d}_y) \leftarrow F(k_F, \mathbf{y})$.
 - Output 1 iff $\mathbf{v}^T \cdot [\mathbf{C}_y^{[j:\ell]} \quad \mathbf{C}_y^{[\ell+s+1:n-r]}] = 0^{n-r-s-j+1}$.
- Generate obfuscations $\mathcal{P} \leftarrow \text{iO}(1^\kappa, 1^p, P)$, $\mathcal{P}^{-1} \leftarrow \text{iO}(1^\kappa, 1^p, P^{-1})$, $\mathcal{D}' \leftarrow \text{iO}(1^\kappa, 1^p, D')$.

Finally, $\mathcal{B}$ runs $(x_0, x_1) \leftarrow \mathcal{A}(\mathcal{P}, \mathcal{P}^{-1}, \mathcal{D}')$, $(\bar{x}_b, \tilde{x}_b, x'_b) \leftarrow \Pi(k_\Gamma, x_b)$ and outputs $(\bar{x}_0, \bar{x}_1)$.

Define

$$\begin{aligned} \mathbf{A}_y &:= \mathbf{C}_y \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \bar{\mathbf{A}}_y & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I}_{n-r-s-\ell} \end{bmatrix} = \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{C}_{y,1} & \mathbf{C}_{y,2} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \bar{\mathbf{A}}_y & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I}_{n-r-s-\ell} \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{I}_\ell & \mathbf{0} \\ \mathbf{C}_{y,1} & \mathbf{C}_{y,2} \cdot \begin{bmatrix} \bar{\mathbf{A}}_y & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_{n-r-s-\ell} \end{bmatrix} \end{bmatrix} \in \mathbb{Z}_2^{n \times (n-r)}, \end{aligned}$$

where $\bar{\mathbf{A}}_y \in \mathbb{Z}_2^{(r+s) \times s}$ is the matrix arising from $\overline{\mathcal{P}}$ and $\overline{\mathcal{P}}^{-1}$. As $\mathbf{C}_{y,2}$ and $\bar{\mathbf{A}}_y$ have full column-rank, so does $\mathbf{A}_y^{[\ell+1:n-r]}$; on the other hand, $\mathbf{C}_{y,1}$ remains a random matrix; thus, $\mathbf{A}_y$ has full column-rank. Now, since

$(\mathbf{u}_0 - \mathbf{u}_1) \notin \text{ColSpan}([A_y^{[1:\ell]} \ A_y^{[\ell+s+1:n-r]}])$, this means that if we look at each of the two coordinates vectors $\mathbf{w}_0, \mathbf{w}_1$ (which each have $n - r$ bits) of $\mathbf{u}_0$ and $\mathbf{u}_1$, they differ somewhere between the $(\ell + 1)$ 'th and $(\ell + s)$ 'th bit. However, $(\mathbf{w}_b)_{[\ell+1:\ell+s]}$ arises from applying the permutation $\bar{\Pi}$ on $\{0, 1\}^{r+s}$ (defined by $\mathcal{P}, \mathcal{P}^{-1}$ to $\bar{x}_b$) and taking the last s bits of the output. As x_0, x_1 have the same $\mathbf{y}$'s and thus so do $\bar{x}_0$ and $\bar{x}_1$, it must be the case that $\bar{x}_0 \neq \bar{x}_1$ in order for $\mathbf{w}_0$ to differ from $\mathbf{w}_1$ in the appropriate indices.

If we define ε_B as the probability that $\mathcal{A}$ outputs x_0, x_1 such that $y_0 = y_1 := \mathbf{y}$ and $(\mathbf{u}_0 - \mathbf{u}_1) \notin \text{ColSpan}([A_y^{[1:\ell]} \ A_y^{[\ell+s+1:n-r]}])$, we wish to lower bound the probability of ε_B . We do this through a series of hybrids.

- **Hyb₀: The distribution $(\mathcal{P}, \mathcal{P}^{-1}, D') \leftarrow \mathcal{B}(\bar{\mathcal{P}}, \bar{\mathcal{P}}^{-1})$, as simulated by $\mathcal{B}$.**

This is the original execution, and thus has success probability ε_B .

- **Hyb₁: Removing the inner obfuscation.**

In this hybrid, we remove the obfuscations from $\bar{\mathcal{P}}$ and $\bar{\mathcal{P}}^{-1}$ and use the underlying plain circuits $\bar{P}$ and $\bar{P}^{-1}$. By the correctness of the inner iO scheme, we have not changed the functionality of the final circuits P, P^{-1}, D , so the security of the outer iO scheme guarantees that this change is $(f(\kappa), O(\frac{1}{f(\kappa)}))$ -computationally indistinguishable.

- **Hyb₂: Removing the inner permutation $\bar{\Pi}_{\text{in}}$ by using the security of the P-PRP.**

Let $\bar{\Pi}_{\text{in}}$ be the first P-PRP inside $\bar{P}$. In the previous hybrid, we apply the n -bit P-PRP $\Pi(k_{\Gamma}, \cdot)$ to x before applying the inner permutation $\bar{\Pi}_{\text{in}}(\bar{k}_{\text{in}}, \cdot)$ to the first $r + s$ output bits of $\Pi(k_{\Gamma}, x)$. The inner permutation can also be thought of as a permutation Γ' on $\{0, 1\}^n$ which acts as the identity on the last $n - r - s$ bits and applies $\bar{\Pi}_{\text{in}}(\bar{k}_{\text{in}}, \cdot)$ to the first $r + s$ bits.

By Theorem A.4, the inner P-PRP $\bar{\Pi}_{\text{in}}(\bar{k}_{\text{in}}, \cdot)$ is $(2^{\text{poly}(\kappa)}, \text{poly}(\kappa))$ -decomposable. This implies that Γ' is also $(2^{\text{poly}(\kappa)}, \text{poly}(\kappa))$ -decomposable.

Since the circuits P and P^{-1} which apply Π and Π^{-1} are obfuscated by the outer iO scheme, if we stop applying Γ' to the output of Π (and $(\Gamma')^{-1}$ to the input of Π), this change is $(f(\kappa), O(\frac{1}{f(\kappa)}))$ -computationally indistinguishable by Lemma A.5.

- **Hyb₃: Taking A_y to be the direct output of the PRF F .**

In the previous hybrid, we computed $A_y := C_y \cdot \begin{bmatrix} \mathbf{I} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \bar{A}_y & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I}_{n-r-s} \end{bmatrix}$ and $\mathbf{b}_y := C_y \cdot \begin{pmatrix} \mathbf{0} \\ \bar{\mathbf{b}}_y \\ \mathbf{0} \end{pmatrix} + \mathbf{d}_y$, where

$\bar{A}_y, \bar{\mathbf{b}}_y$ are defined by the output of the inner PRF $\bar{F}(\bar{k}_{\text{in}}, \mathbf{y})$ and $C_y, \mathbf{d}_y$ are computed from the output of $F(k_F, \mathbf{y})$.

Our goal will be to apply Lemma A.1, so we begin by describing two distributions and two puncturable PRFs.

The first distribution $\mathcal{D}_0$ takes as input $\mathbf{y}$ and randomness (r_1, r_2) , uses r_1 to generate $(\bar{A}_y, \bar{\mathbf{b}}_y)$ and

r_2 to generate $(C_y, \mathbf{d}_y)$, computes $A_y := C_y \cdot \begin{bmatrix} \mathbf{I} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \bar{A}_y & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{I}_{n-r-s} \end{bmatrix}$ and $\mathbf{b}_y := C_y \cdot \begin{pmatrix} \mathbf{0} \\ \bar{\mathbf{b}}_y \\ \mathbf{0} \end{pmatrix} + \mathbf{d}_y$, and outputs

$(A_y, \mathbf{b}_y)$. The second distribution $\mathcal{D}_1$ takes as input $\mathbf{y}$ and randomness r_1 and uses r_1 to generate $(A_y, \mathbf{b}_y)$. $\mathcal{D}_2$ and $\mathcal{D}_3$ are statistically equivalent by the fact that $C_{y,1}$ is a random matrix, $C_{y,2}$ and $\bar{A}_y$ are random full column-rank matrices, and $\mathbf{d}_y$ is a random vector.

Now consider the function F' which takes as input $\mathbf{y}$ and outputs $(r_1, r_2) := (\bar{F}(k_1, \mathbf{y}), F(k_2, \mathbf{y}))$ for randomly sampled keys k_1, k_2 . By an essentially identical argument as in the proof of Lemma B.2, we have that F' is a puncturable PRF.

We now define the circuits $E_0(y) = D_0(y; F'(\overline{(k_{\text{lin}}, k_F)}, y))$ and $E_1(y) = D_1(y; F(k'_F, y))$ for freshly sampled keys $\overline{k_{\text{lin}}}, k_F, k'_F$.

Observing that the circuit E_0 is only used black-box in P , P^{-1} , and D , Lemma A.1 implies that switching to E_1 is $(f(\kappa) - \text{poly}(\kappa), O(\frac{2^r}{f(\kappa)}))$ -computationally indistinguishable.

- **Hyb₄: Moving the second permutation outside $\overline{P}$ and $\overline{P}^{-1}$.**

Recall that in the previous hybrid, the second inner permutation $\overline{\Pi}^{-1}(k_{\text{out}}, \cdot)$, which acts on $\{0, 1\}^d$ to generate y , is used in $\overline{P}$ and $\overline{P}^{-1}$. We simply move the application of the permutation outside of $\overline{P}$ and $\overline{P}^{-1}$. This does not change the functionality of P , P^{-1} , or D , so this change is $(f(\kappa), O(\frac{1}{f(\kappa)}))$ -computationally indistinguishable by the security of the outer iO scheme.

- **Hyb₅: Dropping $\overline{P}$ and $\overline{P}^{-1}$.**

Having moved the permutation $\overline{\Pi}^{-1}(k_{\text{out}}, \cdot)$ to the outside of $\overline{P}$ and $\overline{P}^{-1}$, we observe that P , P^{-1} , and D never use the output of $\overline{P}$ and $\overline{P}^{-1}$, so we can simply generate P , P^{-1} , and D without using $\overline{P}$ and $\overline{P}^{-1}$ at all. This does not change the functionality of P , P^{-1} , or D , so this change is $(f(\kappa), O(\frac{1}{f(\kappa)}))$ -computationally indistinguishable by the security of the outer iO scheme.

It is easy to see that the final distribution in Hyb₅ is precisely the output distribution of $\widetilde{\text{Setup}}(1^\lambda, n, r, \ell, s)$, and the outputs of Hyb₀ and Hyb₅ are $(f(\kappa) - \text{poly}(\kappa), O(\frac{2^r}{f(\kappa)}))$ -computationally indistinguishable. By hypothesis, we have that the success probability of $\mathcal{A}$ in the final hybrid is ε , and thus we have by the lemma's assumptions that $\varepsilon_B \geq \varepsilon - O(\frac{2^r}{f(\kappa)}) \geq \varepsilon/2$, as desired. $\square$

G Improved Analysis of the [SZ25] Distinguisher

In this section, we show that the distinguisher described in [SZ25] does in fact also succeed in distinguishing between the case where the preimage state has not been measured at all and the case that (only) the first bit of the preimage (or indeed, any given bit) has been measured. Recall that [SZ25] showed the weaker statement that the distinguisher succeeds in distinguishing between the case where the preimage state has not been measured at all and the case that *all the bits* of the preimage have been measured.

We first recall the construction of [SZ25].

Construction 7 : One-Shot Signature Scheme, [SZ25]

Let $\lambda \in \mathbb{N}$ be the statistical security parameter. Define $s := 16 \cdot \lambda$, $r := s \cdot (\lambda - 1)$, and $n := r + \frac{3}{2} \cdot s$. Let $\Pi : \{0, 1\}^n \rightarrow \{0, 1\}^n$ be a random permutation and let $F : \{0, 1\}^r \rightarrow \{0, 1\}^{n-(r+1)}$ a random function. Let $H(x)$ denote the first r output bits of $\Pi(x)$, and $J(x)$ denote the remaining $n - r$ bits, which are interpreted as a vector in $\mathbb{Z}_2^{n-r}$. For each $y \in \{0, 1\}^r$, let $A_y \in \mathbb{Z}_2^{n \times (n-r)}$ be a random matrix with full column-rank, and $b_y \in \mathbb{Z}_2^n$ be uniformly random, both are generated by the output randomness of $F(y)$. Then, we let $\mathcal{P} : \{0, 1\}^n \rightarrow (\{0, 1\}^r \times \mathbb{Z}_2^n)$, $\mathcal{P}^{-1} : (\{0, 1\}^r \times \mathbb{Z}_2^n) \rightarrow \{0, 1\}^n \cup \{\perp\}$, $\mathcal{D} : (\{0, 1\}^r \times \mathbb{Z}_2^n) \rightarrow \{0, 1\}$ be the following oracles:

- $\mathcal{P}(x) = (y, A_y \cdot J(x) + b_y)$ where $y = H(x)$
- $\mathcal{P}^{-1}(y, u) = \begin{cases} \Pi^{-1}(y, z) & \exists z \in \mathbb{Z}_2^{n-r} \text{ such that } A_y \cdot z + b_y = u \\ \perp & \text{otherwise} \end{cases}$
- $\mathcal{D}(y, v) = \begin{cases} 1 & \text{if } v^T \cdot A_y = 0^{n-r} \\ 0 & \text{otherwise} \end{cases}$

We denote the above distribution of oracles by $\mathcal{O}_{n,r}$. We define our hash function as H , which can be easily computed by querying $\mathcal{P}$, considering only the first r bits and discarding the second output.

Lemma G.1. *For a given security parameter λ , let $\mathcal{O}_{n,r} := \mathcal{O}_{n(\lambda),r(\lambda)}$ be the oracle distribution of Construction 7.*

There exists a pair of QPT algorithms (S, D) such that for all $\lambda \in \mathbb{N}$,

$$\left| \Pr \left[D(|\psi_{y,b}\rangle_{\text{aux}}=1 : \begin{array}{l} (\mathcal{P}, \mathcal{P}^{-1}, D) \leftarrow \mathcal{O}_{n,r} \\ (|\psi\rangle, \text{aux}) \leftarrow S^{(\mathcal{P}, \mathcal{P}^{-1}, D)} \\ |\psi_{y,b}\rangle \leftarrow \text{Measure}_{H,1}(|\psi\rangle) \end{array} \right] - \Pr \left[D(|\psi_y\rangle_{\text{aux}}=1 : \begin{array}{l} (\mathcal{P}, \mathcal{P}^{-1}, D) \leftarrow \mathcal{O}_{n,r} \\ (|\psi\rangle, \text{aux}) \leftarrow S^{(\mathcal{P}, \mathcal{P}^{-1}, D)} \\ |\psi_y\rangle \leftarrow \text{Measure}_H(|\psi\rangle) \end{array} \right] \right| \geq \frac{1}{4}.$$

Here, $|\psi_{y,b}\rangle \leftarrow \text{Measure}_{H,1}(|\psi\rangle)$ means to measure the first bit of $|\psi\rangle$ in the computational basis after measuring $H^{(\mathcal{P}, \mathcal{P}^{-1}, D)}(\cdot)$, resulting in the outcome (b, y) , while $|\psi_y\rangle \leftarrow \text{Measure}_H$ means to measure $H^{(\mathcal{P}, \mathcal{P}^{-1}, D)}(\cdot)$, resulting in outcome y . Then the state $|\psi\rangle$ collapses to $|\psi_y\rangle$, which contains some superposition of preimages of y .

Proof. S_H computes a uniform superposition $|+\rangle^{\otimes n}$ and outputs it as $|\psi\rangle$ with no auxiliary information.

D_H , given an unknown n -qubit state $|\phi\rangle := \sum_x \alpha_x |x\rangle$, acts as follows:

1. Compute $\mathcal{P}$ to obtain the state

$$\sum_x \alpha_x |x\rangle |y_x\rangle |A_y \cdot \mathbf{w}_x + \mathbf{b}_y\rangle.$$

2. Apply $\mathcal{P}^{-1}$ to uncompute the input register holding x to obtain the state

$$\sum_x \alpha_x |y_x\rangle |A_y \cdot \mathbf{w}_x + \mathbf{b}_y\rangle.$$

3. Apply $H^{\otimes n}$ to the rightmost register (which contains coset vectors) to obtain the state

$$\sum_x \alpha_x |y_x\rangle (H^{\otimes n} |A_y \cdot \mathbf{w}_x + \mathbf{b}_y\rangle).$$

4. Apply $D(\cdot, \cdot)$ before measuring and returning the output bit register.

The case where just H is measured is the same as in [SZ25]: D_H receives the state $\sum_{x: H(x)=y} |x\rangle$ for some y , and after the second step D_H holds the state

$$|y\rangle \sum_{x: H(x)=y} |A_y \cdot \mathbf{w}_x + \mathbf{b}_y\rangle.$$

Since applying the QFT to a uniform superposition over a coset yields a uniform superposition over the dual subspace (with a phase), it follows that D_H outputs 1 with probability 1.

In the case where both H and the first preimage bit is measured, D_H receives the state $\sum_{x: H(x)=y, x_1=b} |x\rangle$ for some hash y and bit b . Define the set $S_{y,b} = \{A_y \cdot \mathbf{w}_x + \mathbf{b}_y\}_{\mathbf{w}_x: x_1=b} \subseteq \{A_y \cdot \mathbf{w}_x + \mathbf{b}_y\}_{\mathbf{w}_x \in \{0,1\}^{n-r}}$. At the end of the second step, D_H holds the state

$$\frac{1}{\sqrt{|S_{y,b}|}} |y\rangle \otimes \sum_{\mathbf{s} \in S_{y,b}} |\mathbf{s}\rangle.$$

After applying the QFT $H^{\otimes n}$, D_H holds the state

$$\frac{1}{\sqrt{2^n \cdot |S_{y,b}|}} |y\rangle \otimes \sum_{\mathbf{z} \in \mathbb{Z}_2^n} \left(\sum_{\mathbf{s} \in S_{y,b}} (-1)^{\mathbf{s} \cdot \mathbf{z}} \right) |\mathbf{z}\rangle.$$

Thus, the probability of step 4 outputting 1 is precisely

$$\begin{aligned} \frac{1}{2^n \cdot |S_{y,b}|} \sum_{z \in A_y^\perp} \left(\sum_{s \in S_{y,b}} (-1)^{s \cdot z} \right)^2 &= \frac{1}{2^n \cdot |S_{y,b}|} \sum_{z \in A_y^\perp} \left(\sum_{s \in S_{y,b}} (-1)^{b_y \cdot z} \right)^2 \\ &= \frac{1}{2^n \cdot |S_{y,b}|} \sum_{z \in A_y^\perp} |S_{y,b}|^2 = \frac{2^r \cdot |S_{y,b}|^2}{2^n \cdot |S_{y,b}|} = \frac{|S_{y,b}|}{2^{n-r}}. \end{aligned}$$

If we fix a permutation Π , we know that each hash value y is equally likely to occur, and conditioned on Π and y , the outcome b will be measured with probability $\frac{|S_{y,b}|}{2^{n-r}}$. Thus, the probability that $\mathcal{D}_H$ outputs 1 is precisely

$$\begin{aligned} \mathbb{E}_{\Pi,y} \left[\frac{|S_{y,0}|^2}{2^{2(n-r)}} + \frac{|S_{y,1}|^2}{2^{2(n-r)}} \right] &= \mathbb{E}_{\Pi,y} \left[\frac{|S_{y,0}|^2 + (2^{n-r} - |S_{y,0}|)^2}{2^{2(n-r)}} \right] \\ &= \mathbb{E}_{\Pi,y} \left[\frac{2|S_{y,0}|^2 + 2^{2(n-r)} - 2^{n-r+1}|S_{y,0}|}{2^{2(n-r)}} \right] \\ &= \mathbb{E}_{\Pi,y} \left[\frac{2|S_{y,0}|^2}{2^{2(n-r)}} \right] + \frac{2^{2(n-r)} - 2^{n-r+1} \cdot 2^{n-r-1}}{2^{2(n-r)}} \\ &= 2\mathbb{E}_{\Pi,y} \left[\left(\frac{|S_{y,0}|}{2^{n-r}} \right)^2 \right] \\ &= 2 \left[\left(\frac{2^{n-r-1}}{2^{n-r}} \right)^2 + \frac{2^{n-r}}{2^{2(n-r)}} \cdot \frac{2^{n-1}}{2^n} \cdot \frac{2^n - 2^{n-1}}{2^n} \cdot \frac{2^n - 2^{n-r}}{2^n - 1} \right] \\ &= \frac{1}{2} + \frac{1}{2^{n-r+1}} \cdot \frac{2^n - 2^{n-r}}{2^n - 1} \leq \frac{1}{2} + \frac{1}{2^{n-r+1}} \leq \frac{3}{4}, \end{aligned}$$

since $|S_{y,0}|$ is a hypergeometric variable.

We conclude that the distinguishing advantage of $(\mathcal{S}_H, \mathcal{D}_H)$ is at least $1 - \frac{3}{4} = \frac{1}{4}$, as desired. $\square$